

```
export default async function Page() {  
  // ...  
  "use cache";  
  const data = await getData();  
  return <App data={data} />;  
}  
  
// Server Components  
// App Router - RSC  
// Production ready
```

Next.js 16 

راهنمای جامع و حرفه‌ای

Next.js 16

از مفاهیم پایه تا معماری Production-Level

Next.js

React 19

TypeScript

راهنمایی برای جامعه توسعه‌دهندگان فارسی‌زبان، به‌سوی یادگیری
مفاهیم مدرن، معماری حرفه‌ای و استانداردهای واقعی صنعت.

۳۷ فصل کاربردی • مینی‌پروژه عملی • آمادگی مصاحبه

محمدرضا رضائیان

گردآوری و تدوین

با هدف کمک به جامعه توسعه‌دهندگان Front-End فارسی‌زبان



فهرست مطالب

بخش یکم - بنیادها و مفاهیم اصلی

01 معرفی Next.js و تازه‌های نسخه ۱۶

فریم‌ورک چیست و در نسخه ۱۶ چه تغییر کرده

02 شروع به کار و ساختار پروژه

نصب، Turbopack، create-next-app و آناتومی فولدرها

03 نقشه راه یادگیری

مسیر مبتدی تا حرفه‌ای و نحوه استفاده از کتاب

04 روتینگ فایل محور (App Router)

صفحات، مسیرهای پویا، گروه‌ها و موازی

05 Layout، فایل‌های ویژه و Metadata

قالب‌ها، loading، error، not-found و سئوی پایه

06 پیوند و ناوبری

useLinkStatus و Link، prefetch، useRouter

07 Client Components و Server

تفاوت، زمان استفاده و الگوی ترکیب

08 واکنشی داده و Streaming

preload و موازی، async، Suspense

09 کشینگ مدرن با Cache Components

revalidate و use cache، cacheLife، cacheTag

10 Server Actions و تغییر داده

فرم‌ها، revalidation و useActionState

11 Route Handlers (مبانی API)

ساخت اندپوینت و پارامترهای async

12 proxy.ts؛ جانشین Middleware

احراز هویت، ریدایرکت و نکات امنیتی

13 استایل‌دهی با Tailwind v4

shadcn/ui و CSS Modules، Tailwind

14 بهینه‌سازی: تصویر، فونت، لینک و سئو

sitemap و next/image، next/font

15 محیط، TypeScript و استقرار

پیکربندی، تایپ‌ها و دیپلوی پایه

16 پروژه عملی: مینی‌بلاگ کامل

ترکیب مفاهیم پایه در یک نمونه واقعی

بخش دوم - معماری، پیشرفته و فوق‌پیشرفته

17 معماری پروژه، ساختار پوشه و Clean Code

لایه‌بندی، DAL، feature-based و مقیاس‌پذیری

18 استراتژی‌های رندر: SSR، SSG، ISR و PPR

چهار مدل رندر و پیش‌رندر جزئی

19 API نویسی کامل با Route Handlers

استریم، CORS، Webhook، SSE و Runtime

20 خطاهای Hydration – مرجع کامل

علت‌ها، راه‌حل و جدول مرجع سریع

21 احراز هویت و کنترل دسترسی

session، کوکی httpOnly و RBAC

22 امنیت اپلیکیشن – فصل کامل

اعتبارسنجی، rate limit، XSS، CSRF و اسرار

23 فرم‌ها و اعتبارسنجی پیشرفته

useFormStatus، schema مشترک و بهبود تدریجی

24 مدیریت state و داده در کلاینت

هوک use()، Context و SWR

25 تسلط بر کشینگ: لایه‌ها و باطل‌سازی

گونه‌های use cache و پروفایل سفارشی

26 کارایی و Core Web Vitals

React Compiler و LCP، CLS، INP

27 Metadata و سئو – کامل

Open Graph، OG پویا و JSON-LD

28 TypeScript پیشرفته و امنیت تایپ

infer، typedRoutes، zod و satisfies

29 مدیریت خطا – کامل

سلسله مراتب خطا و notFound

30 تست نویسی

Vitest، Testing Library و Playwright

31 استقرار حرفه ای و Self-Hosting

Docker، standalone، Vercel و چک لیست

32 مهاجرت و ارتقا به نسخه ۱۶

codemod و تغییرات شکننده

بخش سوم · جمع بندی و کارگاه

33 نکات و ترفندهای طلایی

React.cache، after، useOptimistic و ...

34 کارگاه مینی پروژه ها

هشت پروژه کوچک صفر تا صد

35 بهترین شیوه ها، اشتباهات رایج و منابع

جمع بندی حرفه ای و مسیر ادامه

بخش چهارم · پیوست ها

36 پرسش های مصاحبه (Junior تا Senior)

پرکاربردترین سؤالات با پاسخ دقیق

37 واژه نامه فارسی-انگلیسی

مرجع سریع اصطلاحات کلیدی

01 معرفی Next.js و تازه‌های نسخه ۱۶

در این فصل می‌بینیم Next.js دقیقاً چیست، چه مشکلی را در توسعه React حل می‌کند، و چرا نسخه ۱۶ یک نقطه عطف محسوب می‌شود.

Next.js چیست و چه مشکلی را حل می‌کند؟

React به‌تنهایی فقط یک کتابخانه برای ساخت رابط کاربری است؛ یعنی مسائلی مثل روتینگ، رندر سمت سرور، بهینه‌سازی تصویر، کشینگ و ساختار پروژه را به خودتان واگذار می‌کند. **Next.js** یک فریم‌ورک کامل روی React است که همه این لایه‌ها را به‌صورت یکپارچه و آماده برای Production فراهم می‌کند. به بیان ساده: React جعبه ابزار است، Next.js کارگاه کامل آماده‌به‌کار.

سه قابلیت بنیادی که Next.js در اختیار شما می‌گذارد:

- ♦ **رندر سمت سرور (SSR) و تولید استاتیک (SSG):** صفحه روی سرور آماده می‌شود، نه فقط در مرورگر کاربر؛ نتیجه‌اش سرعت بارگذاری بالاتر و سئوی بهتر است.
- ♦ **روتینگ فایل‌محور:** ساختار فولدرها مستقیماً نقشه آدرس‌های سایت شما را می‌سازد؛ بدون نیاز به پیکربندی دستی روتر.
- ♦ **full-stack بودن:** می‌توانید بک‌اند (API، اتصال به دیتابیس، احراز هویت) را در همان پروژه فرانت بنویسید.

معماری پیشنهادی: App Router

Next.js دو سیستم روتینگ دارد: **Pages Router** قدیمی و **App Router** مدرن. تمام این آموزش بر پایه **App Router** است؛ همان معماری‌ای که برای هر پروژه جدید در سال ۱۴۰۵ توصیه می‌شود و قابلیت‌هایی مثل **Server Components** و **Cache Components** فقط در آن وجود دارند.

چه چیزی در نسخه ۱۶ تغییر کرد؟

نسخه ۱۶ مهم‌ترین به‌روزرسانی چند سال اخیر است. اگر از نسخه‌های ۱۴ یا ۱۵ می‌آیید، این تغییرات را باید بشناسید:

قابلیت	توضیح کوتاه
Turbopack پیش‌فرض	باندلر جدید مبتنی بر Rust جایگزین Webpack شد؛ بیلد production تا چند برابر سریع‌تر و Fast Refresh به شکل چشمگیری روان‌تر است.
Cache Components	مدل کشینگ کاملاً بازنویسی شد. حالا همه چیز به صورت پیش‌فرض «پویا» است و شما با دایرکتیو <code>use cache</code> به صراحت انتخاب می‌کنید چه چیزی کش شود.
React 19.2	قابلیت‌هایی مثل View Transitions، <code><Activity></code> و پشتیبانی پایدار از React Compiler.
async های API	<code>cookies()</code> ، <code>searchParams</code> ، <code>params</code> و <code>headers()</code> همگی async شدند و باید <code>await</code> شوند.
proxy.ts	فایل <code>middleware.ts</code> به <code>proxy.ts</code> تغییر نام داد تا نقش واقعی آن (لایه شبکه) شفاف‌تر باشد.
+Node.js ۲۰.۹	حداقل نسخه Node بالا رفت؛ روی Node 18 اجرا نمی‌شود.
حذف‌ها	<code>next lint</code> و پشتیبانی از AMP حذف شدند و چند prop قدیمی از <code>next/image</code> کنار رفت.

مهم‌ترین تغییر ذهنی

در نسخه‌های قبل، Next.js به صورت پیش‌فرض و پنهانی همه چیز را کش می‌کرد و شما باید با تنظیمات مختلف از کش «خارج» می‌شدید. در نسخه ۱۶ این منطق برعکس شد: **هیچ چیز به صورت خودکار کش نمی‌شود** و شما با `use cache` آگاهانه انتخاب می‌کنید چه بخشی کش شود. این تغییر، رفتار اپلیکیشن‌های مهاجرت‌یافته از نسخه ۱۴/۱۵ را عوض می‌کند.

پیش‌نیازهای این آموزش

- تسلط نسبی به **JavaScript مدرن** (async/await، destructuring، modules) و آشنایی پایه با **React** (کامپوننت، props، state، hooks).
- آشنایی با **TypeScript**؛ تمام مثال‌ها TS هستند چون عملاً استاندارد پروژه‌های جدی شده است.
- **Node.js نسخه ۲۰.۹ یا بالاتر** و یک ویرایشگر مثل VS Code.

02 شروع به کار و ساختار پروژه

از صفر یک پروژه Next.js 16 می‌سازیم، سرور توسعه را بالا می‌آوریم و معماری فولدرها را می‌شناسیم.

ساخت پروژه جدید

ابتدا نسخه Node را بررسی کنید؛ باید ۲۰.۹ یا بالاتر باشد. سپس با دستور رسمی، یک پروژه آماده TypeScript و Tailwind بسازید:

```
Terminal

# ✔ Check Node version first (must be ≥ 20.9)
node -v

# 🚀 Scaffold a new Next.js 16 app (answers prompts non-interactively)
npx create-next-app@latest doctor-booking \
  --typescript --tailwind --eslint \
  --app --src-dir --import-alias "@/*" --use-npm

cd doctor-booking
npm run dev # ⚡ Turbopack dev server boots in well under a second
```

پس از اجرای `npm run dev`، اپلیکیشن روی `http://localhost:3000` در دسترس است. چون Turbopack حالا پیش‌فرض است، بالا آمدن سرور و Fast Refresh به شکل محسوسی سریع‌تر از قبل است.

معنای فلگ‌ها

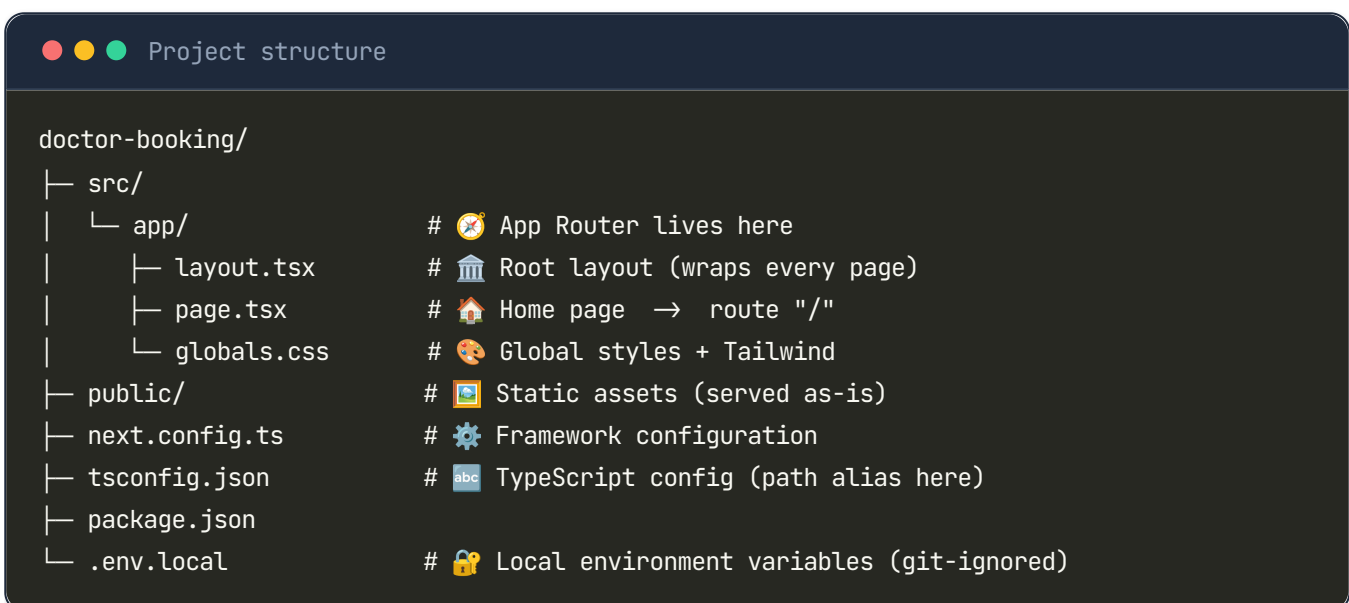
`--app` معماری App Router را فعال می‌کند، `--src-dir` کدها را داخل پوشه `src` مرتب می‌کند، و `--import-alias "@/*"` اجازه می‌دهد به جای مسیرهای نسبی طولانی، از `@/components/...` استفاده کنید.

دستورهای اصلی

دستور	کاربرد
<code>npm run dev</code>	سرور توسعه با Hot Reload
<code>npm run build</code>	ساخت نسخه بهینه Production
<code>npm run start</code>	اجرای خروجی build روی سرور
<code>npx tsc --noEmit</code>	بررسی صحت تایپ‌ها بدون تولید فایل خروجی

آناتومی پروژه

ساختار یک پروژه تازه‌ساخته چنین است:



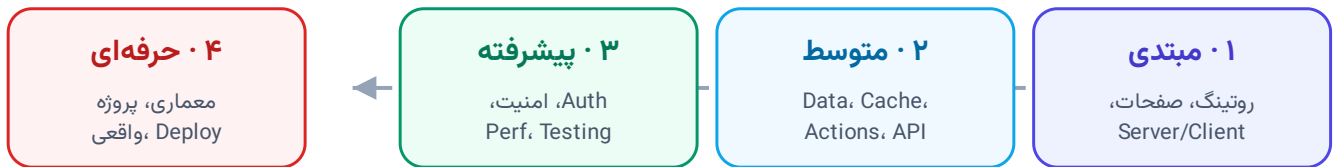
قلب اپلیکیشن، پوشه `src/app` است. هر فولدر داخل آن یک «سگمنت مسیر» می‌سازد و فایل‌های با نام‌های ویژه (مثل `page.tsx` و `layout.tsx`) رفتار آن مسیر را تعیین می‌کنند. در فصل بعد دقیقاً همین قواعد را باز می‌کنیم.

پوشه public

هر فایل‌ای که در `public` بگذارید، مستقیماً از ریشه دامنه سرو می‌شود. مثلاً `public/logo.svg` با آدرس `/logo.svg` در دسترس است.

03 نقشه راه یادگیری

این کتاب یک مسیر پیوسته از مبتدی تا حرفه‌ای است. این فصل نشان می‌دهد چه چیزی، به چه ترتیبی و با چه هدفی یاد می‌گیرید تا آماده بازار کار شوید.



خروجی: توانایی ساخت و استقرار یک اپلیکیشن Production واقعی با Next.js 16

آماده برای مصاحبه فنی و ورود به بازار کار به عنوان توسعه‌دهنده Front-End

چهار مرحله، چه چیزی و چرا

مرحله	فصل‌ها	به چه توانایی می‌رسید
مبتدی	بنیادها (رویتینگ، Layout، پیوند، Server/Client)	ساخت صفحات و ناوبری، درک مدل رندر سمت سرور
متوسط	Data، Cache Components، Server Actions، Route Handlers، استایل، سئو	واکشی و تغییر داده، کشینگ آگاهانه، ساخت API
پیشرفته	رندر، Hydration، Auth، امنیت، Perf، خطا، تست	اپ امن، سریع، پایدار و تست‌شده
حرفه‌ای	معماری، پروژه نهایی، استقرار، مصاحبه	معماری تمیز، پروژه واقعی، دیپلوی، آمادگی شغلی

چطور از این کتاب بیشترین بهره را ببرید

♦ **خطی بخوانید، اما دست نگه ندارید:** هر فصل مثال عملی دارد؛ همان‌جا کد را در یک پروژه واقعی تایپ و اجرا کنید، نه فقط بخوانید.

- ♦ **هر فصل یک مینی پروژه دارد:** به جای یک پروژه بزرگ نهایی، هر مبحث با یک مینی پروژه مستقل و کامل تثبیت می شود؛ همان جا پیاده اش کنید تا مفهوم در دستتان بماند.
- ♦ **روی «اشتباهات رایج» و «پرسش های مصاحبه» تمرکز کنید:** این ها دقیقاً همان چیزهایی اند که در کار و مصاحبه به کارتان می آیند.
- ♦ **هر فصل را با یک تمرین ببندید:** در پایان فصل های کلیدی یک «تمرین» آمده؛ تا انجامش ندهید، سراغ فصل بعد نروید.

پیش نیاز صادقانه

این کتاب فرض می کند JavaScript مدرن و مبانی React (کامپوننت، props، state، hooks) را می دانید. اگر این ها لرزانند، ابتدا چند روز روی همان ها کار کنید؛ بازدهی این کتاب چند برابر می شود.

04 روتینگ فایل محور (App Router)

در Next.js «ساختار پوشه‌ها» همان «نقشه آدرس‌های سایت» است. این فصل تمام انواع مسیر – از ساده تا موازی و رهگیری‌کننده – را عمیق و با مثال واقعی آموزش می‌دهد.

۱ | مفهوم چیست؟

روتینگ یعنی «کدام آدرس، چه صفحه‌ای را نشان دهد». در App Router این کار را با **ساختار پوشه** انجام می‌دهید: هر پوشه داخل `app` یک سگمنت از آدرس می‌سازد، و قرار دادن فایل `page.tsx` در آن پوشه، آن مسیر را «قابل دسترسی» می‌کند. بدون `page.tsx`، پوشه فقط یک گروه‌بندی است و آدرسی نمی‌سازد.

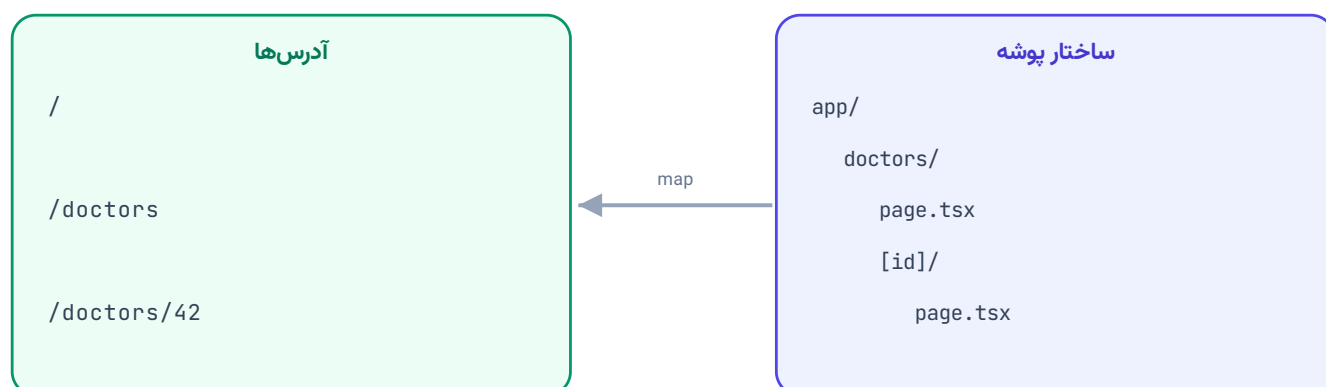
۲ | چرا فایل محور؟ چه مشکلی را حل می‌کند؟

در روترهای سنتی (مثل React Router) باید مسیرها را **دستی** در یک فایل پیکربندی تعریف و نگهداری می‌کردید؛ با بزرگ‌شدن اپ، آن فایل به گلوگاه و منبع خطا تبدیل می‌شد. روتینگ فایل محور این دردها را حل می‌کند:

- ♦ **بدون پیکربندی:** ساختار پوشه = ساختار آدرس. برای دیدن نقشه سایت کافی است به پوشه‌ها نگاه کنید.
- ♦ **هم مکانی (Colocation):** صفحه، layout، حالت بارگذاری و خطای هر مسیر کنار خودش زندگی می‌کند.
- ♦ **مقیاس‌پذیری:** افزودن مسیر یعنی افزودن پوشه؛ نه ویرایش یک فایل مرکزی پرتنش.

۳ | Mental Model – مدل ذهنی

درخت پوشه‌ها را دقیقاً مثل درخت آدرس‌ها تصور کنید: از ریشه (`/` = `app/`) هر پوشه یک قدم در مسیر است، و `page.tsx` نقطه توقفی است که «اینجا چیزی برای نمایش هست».



۴ | نحوه کار داخلی (Behind the Scenes)

- ◆ هنگام بیلد، Next.js درخت `app` را پیمایش می‌کند و برای هر `page.tsx` یک مسیر می‌سازد.
- ◆ وقتی درخواستی می‌آید، آدرس را با این درخت تطبیق می‌دهد: سگمنت‌های ثابت اولویت دارند، سپس پویا (`[id]`)، و در آخر `catch-all`.
- ◆ سپس زنجیره `layout` های مسیر را از ریشه تا برگ روی هم می‌چیند و `page` را داخل آن‌ها رندر می‌کند.

انواع مسیر (کامل)

الف) مسیر ثابت (Static): ساده‌ترین حالت؛ یک پوشه با `page.tsx`. صفحه فقط یک کامپوننت است:

```
Routing map + a static page

app/page.tsx      → /
app/about/page.tsx → /about
app/doctors/page.tsx → /doctors
app/doctors/[id]/page.tsx → /doctors/42    (dynamic)
app/shop/[...slug]/page.tsx → /shop/a/b/c    (catch-all)

src/app/about/page.tsx

// 📄 A page is just a default-exported component
export default function AboutPage() {
  return <h1>درباره ما</h1>;
}
```

ب) مسیر پویا (Dynamic): وقتی بخشی از آدرس متغیر است، نام پوشه را در گروه بگذارید: `[id]`. مقدار از طریق `params` می‌رسد.

تغییر کلیدی نسخه ۱۶: `params` حالا `async` است

در نسخه ۱۶، `params` یک Promise است و باید `await` شود. این رایج‌ترین خطای مهاجرت از نسخه‌های قدیمی است.

src/app/doctors/[id]/page.tsx

```
// ⚠️ params is a Promise in v16 → type it and await it
type PageProps = { params: Promise<{ id: string }> };

export default async function DoctorPage({ params }: PageProps) {
  const { id } = await params; // ⌚ must await in Next.js 16
  const doctor = await getDoctor(id); // 🌐 fetch on the server
  if (!doctor) notFound(); // 🚫 renders the nearest not-found.tsx
  return <h1>{doctor.name}</h1>;
}
```

ج) **Catch-all**: برای گرفتن چند سگمنت، از `[. slug]` استفاده کنید (آرایه‌ای از سگمنت‌ها می‌دهد). نسخه اختیاری `[. slug]` حالت «بدون سگمنت» را هم می‌گیرد – مناسب صفحات داکيومنت یا دسته‌بندی چندلایه.

د) **مسیر تودرتو (Nested)**: پوشه داخل پوشه = مسیر داخل مسیر؛ مثلاً `app/doctors/[id]/reviews/page.tsx` می‌سازد: `/doctors/42/reviews`. هر لایه می‌تواند `layout` خودش را داشته باشد.

گروه‌بندی مسیرها (Route Groups)

چرا؟ گاهی می‌خواهید چند مسیر یک layout مشترک داشته باشند یا صرفاً سازمان‌دهی شوند، بدون آنکه نام پوشه در آدرس ظاهر شود. نام پوشه را در پرانتز بگذارید: `(marketing)`.

Route groups – same URL, separate layouts

```
app/
├─ (marketing)/
│  ├─ layout.tsx      # 🍷 public layout (navbar/footer)
│  └─ page.tsx        → /      (the "()" name is invisible)
├─ pricing/page.tsx   → /pricing
└─ (dashboard)/
   ├─ layout.tsx      # 🛡️ app layout (sidebar) – a totally different shell
   └─ settings/page.tsx → /settings
```

کاربرد واقعی

الگوی طلایی برای جدا کردن «سایت عمومی» از «پنل کاربری»: هر گروه layout مستقل خودش را دارد، اما آدرس‌ها تمیز و بدون پیشوند می‌مانند.

مسیرهای موازی (Parallel Routes)

چه مشکلی را حل می‌کنند؟ گاهی می‌خواهید چند بخش مستقل را هم‌زمان در یک صفحه نشان دهید که هرکدام ناوبری، بارگذاری و خطای خودشان را دارند (مثل یک داشبورد با دو پنل مجزا). با «اسلات‌ها» (پوشه‌های @) این بخش‌ها به‌صورت prop به layout می‌رسند و مستقل رندر می‌شوند.

parallel routes structure

```
app/
├─ layout.tsx          # receives @team and @analytics as props
├─ page.tsx            → the main content (children)
├─ @team/page.tsx      # 🌱 an independent slot
├─ @team/default.tsx   # 🚧 fallback when this slot has no match (e.g. on refresh)
└─ @analytics/page.tsx # 🌱 another independent slot
```

src/app/layout.tsx

```
// 🌱 each @slot arrives as a named prop, rendered independently
export default function DashboardLayout({
  children,
  team,
  analytics,
}: {
  children: React.ReactNode;
  team: React.ReactNode;
  analytics: React.ReactNode;
}) {
  return (
    <div className="space-y-4">
      {children}
      <div className="grid grid-cols-2 gap-4">
        {team}      {/* 📊 renders @team, with its own loading/error */}
        {analytics} {/* 📈 renders @analytics, independently */}
      </div>
    </div>
  )
}
```

```

    </div>
  );
}

```

نقش default.tsx

وقتی کاربر مستقیماً یک URL را باز/رفرش می‌کند، ممکن است یک اسلات برای آن آدرس محتوایی نداشته باشد. فایل `default.tsx` می‌گوید «در این حالت چه چیزی نشان بده» و از خطای «صفحه پیدا نشد» جلوگیری می‌کند.

مسیرهای رهگیری‌کننده (Intercepting Routes)

چه مشکلی را حل می‌کنند؟ کلاسیک‌ترین کاربرد: باز کردن یک محتوا در **مودال روی صفحه فعلی** هنگام کلیک، اما نمایش همان محتوا به صورت **صفحه کامل** اگر کاربر آدرس را مستقیم باز/رفرش کند (تا لینک قابل اشتراک بماند). با پیشوندهای `(.)` (هم‌سطح)، `(. )` (یک سطح بالاتر) و `(. .)` (از ریشه) یک مسیر را «رهگیری» می‌کنید.

intercepting + parallel = shareable modal

```

app/
├─ feed/page.tsx           # the list; photos link to /photo/123
├─ photo/[id]/page.tsx     # 🖼️ FULL page (direct visit / refresh)
└─ feed/
   ├─ @modal/(.)photo/[id]/page.tsx # 🪟 intercepted → shows as a modal over the feed
   └─ @modal/default.tsx         # 🚫 nothing when no modal is open

```

src/app/feed/@modal/(.)photo/[id]/page.tsx

```

import { Modal } from "@components/modal";

// 🪟 Same data, but presented inside a modal over the feed
export default async function PhotoModal({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const photo = await getPhoto(id);
  return (
    <Modal>
      <img src={photo.url} alt={photo.title} className="rounded-lg" />
    </Modal>
  );
}

```

۵ | چه زمانی استفاده کنیم؟ / ۶ | چه زمانی نه؟ (Trade-offs)

نیاز	ابزار درست
صفحه معمولی	مسیر ثابت
آدرس با شناسه متغیر	مسیر پویا <code>[id]</code>
layoutهای متفاوت بدون پیشوند در URL	Route Group <code>(...)</code>
چند پنل مستقل در یک صفحه	Parallel Routes <code>@slot</code>
مودال قابل اشتراک (لینک‌پذیر)	Intercepting <code>(.)</code> + Parallel

هشدار پیچیدگی

Parallel و Intercepting قدرتمندند، اما پیچیدگی ذهنی و نگهداری اضافه می‌کنند. اگر یک مودال ساده با `useState` کافی است، سراغ Intercepting نروید. این ابزارها را فقط وقتی به «قابل اشتراک بودن URL» یا «پنل‌های واقعاً مستقل» نیاز دارید به کار ببرید – نه به‌خاطر جذابیت.

۷ | مثال واقعی: چیدمان یک اپ کامل

سناریو: یک اپ رزرو با سایت عمومی و پنل کاربری که هرکدام پوسته متفاوت دارند، و پرونده پویای پزشک با زیرمسیر نظرات:

```
real-world app routing

app/
├─ (site)/                                # public shell (navbar + footer)
│  ├─ layout.tsx
│  ├─ page.tsx                            → /
│  └─ doctors/
│     ├─ page.tsx                        → /doctors          (list)
│     └─ [id]/
│        ├─ page.tsx                    → /doctors/42        (profile)
│        ├─ loading.tsx                 # ⌚ shown while the profile loads
│        └─ reviews/page.tsx           → /doctors/42/reviews (nested)
└─ (panel)/                              # authenticated shell (sidebar)
   ├─ layout.tsx
   └─ appointments/page.tsx             → /appointments
```

چرا این چیدمان؟

گروه‌های (site) و (panel) دو پوسته کاملاً جدا می‌سازند بدون آلوده کردن آدرس‌ها. `loading.tsx` کنار پرونده پزشک، تجربه بارگذاری همان بخش را می‌سازد. زیرمسیر `reviews` نشان می‌دهد چطور مسیرها به‌طور طبیعی تودرتو می‌شوند.

❌ اشتباهات رایج / ✅ روش درست

اشتباه ❌	درست ✅
خواندن <code>params</code> بدون <code>await</code> (سبک نسخه قدیم)	در v16 حتماً <code>const { id } = await params</code>
استفاده از تگ <code><a></code> برای ناوبری داخلی	کامپوننت <code><Link></code> (فصل پیوند و ناوبری)
گذاشتن فایل غیرصفحه‌ای به نام <code>page.tsx</code> در هر پوشه	فقط جایی <code>page.tsx</code> که واقعاً باید آدرس بسازد
استفاده بی‌مورد از Intercepting برای مودال ساده	مودال ساده = <code>useState</code> ؛ Intercepting فقط برای URL قابل اشتراک
فراموش کردن <code>default.tsx</code> در اسلات‌های موازی	برای هر <code>@slot</code> یک <code>default.tsx</code> بگذارید

✅ Best Practices و نکات طلایی

(۱) ساختار پوشه را «خوانا» نگه دارید؛ کسی باید با نگاه به `app/` نقشه سایت را بفهمد. (۲) فایل‌های ویژه (`loading` / `error` / `not-found`) را نزدیک همان سگمنت بگذارید. (۳) از Route Groups برای جدا کردن پوسته‌ها استفاده کنید. (۴) `[id]` را همیشه تایپ کنید و `notFound()` را برای داده ناموجود صدا بزنید. (۵) Parallel/Intercepting را فقط با نیاز واقعی به کار ببرید.

🚀 مینی پروژه: سیستم پروفایل کاربران + گالری با مودال قابل اشتراک

۱) مسیر `app/(site)/users/[username]/page.tsx` را بسازید که پروفایل را روی سرور واکنشی کند و زیرمسیر `/posts/page.tsx` ... برای پست‌ها داشته باشد. ۲) یک گالری بسازید که کلیک روی هر عکس آن را در یک مودال باز کند (`(.)` Intercepting + اسلات `@modal`)، اما باز کردن مستقیم `/photo/[id]` صفحه کامل نشان دهد. ۳) `not-found.tsx` برای کاربر/عکس ناموجود اضافه کنید. هدف: تسلط عملی بر Intercepting و Nested, Parallel.

📌 خلاصه فصل

پوشه = سگمنت آدرس، و `page.tsx` = صفحه قابل دسترسی. مسیر پویا با `[id]` (و `params` در v16 یک Promise است). Route Groups با `(.)` پوشتهای مستقل بدون پیشوند می‌سازند. Parallel Routes با `@slot` پنلهای مستقل همزمان، و Intercepting با `(.)` مودال قابل اشتراک می‌سازند. ابزارهای پیشرفته را فقط با نیاز واقعی به کار ببرید.

📌 تمرین

۱) یک ساختار بسازید که `/blog/[slug]`، `/blog`، `/` و `/blog/[slug]/comments` را پوشش دهد و برای `[slug]`، `notFound()` را روی داده‌ی ناموجود فعال کن. ۲) دو گروه `(site)` و `(panel)` با دو layout متفاوت بساز و تفاوت پوشته را هنگام ناوبری ببین. ۳) یک اسلات `@modal` بساز و رفتار آن را با و بدون `default.tsx` هنگام رفرش مقایسه کن.

🎤 سؤالات مصاحبه (با پاسخ)

Junior – چطور یک صفحه جدید و یک مسیر پویا می‌سازید؟ یک پوشه داخل `app` با فایل `page.tsx` می‌سازم؛ نام پوشه سگمنت آدرس است. برای مسیر پویا نام پوشه را در گروه می‌گذارم (`[id]`) و مقدار را از `params` می‌خوانم (که در v16 باید `await` شود).

Mid – Route Group چیست و چه فرقی با پوشه عادی دارد؟ پوشه‌ای با نام داخل پرانتز (`(marketing)`) که در آدرس ظاهر نمی‌شود؛ برای سازمان‌دهی و مهم‌تر، دادن **layout مستقل** به گروهی از مسیرها بدون افزودن پیشوند به URL. مثلاً جدا کردن سایت عمومی از پنل.

Senior – Parallel و **Intercepting Routes** چه مشکلی را حل می‌کنند و Trade-off شان چیست؟ Parallel ((`@slot`)) چند بخش مستقل با ناوبری/بارگذاری/خطای جدا را در یک layout رندر می‌کند (داشبوردهای پیچیده). Intercepting (`(.)`) اجازه می‌دهد یک مسیر هنگام ناوبری داخلی به‌صورت مودال و هنگام باز کردن مستقیم به‌صورت صفحه کامل نمایش داده شود (مودال قابل اشتراک). Trade-off: پیچیدگی ذهنی و نگهداری بالاتر؛ فقط با نیاز واقعی به کار می‌روند، وگرنه یک مودال `useState` ساده‌تر و بهتر است.

05 Layout، فایل‌های ویژه و Metadata

فایل‌های با نام مخصوص، رفتار هر سگمنت را کنترل می‌کنند: قالب مشترک، حالت بارگذاری، خطا، صفحه پیداننده و متادیتای سئو.

Layout؛ پوسته‌ای که باقی می‌ماند

رابط کاربری مشترکی است که چندین صفحه را در بر می‌گیرد و هنگام جابه‌جایی بین صفحه‌ها `layout.tsx` `re-render` نمی‌شود (state آن حفظ می‌شود). یک `layout` حتماً باید `children` را بگیرد و رندر کند.

هر اپلیکیشن دقیقاً یک **Root Layout** دارد که تگ‌های `<html>` و `<body>` در آن تعریف می‌شوند. برای رابط فارسی، اینجا جای درست تنظیم `lang` و `dir` است:

```
src/app/layout.tsx

import type { Metadata } from "next";
import "../globals.css";

// 🌐 App-wide default metadata (great for SEO defaults)
export const metadata: Metadata = {
  title: { default: "دکتر رزرو", template: "%s | دکتر رزرو" },
  description: "رزرو آنلاین نوبت پزشک",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  // 🚫 RTL + Persian: set lang and dir on the html element
  return (
    <html lang="fa" dir="rtl">
      <body>{children}</body>
    </html>
  );
}
```

قالب عنوان (title template)

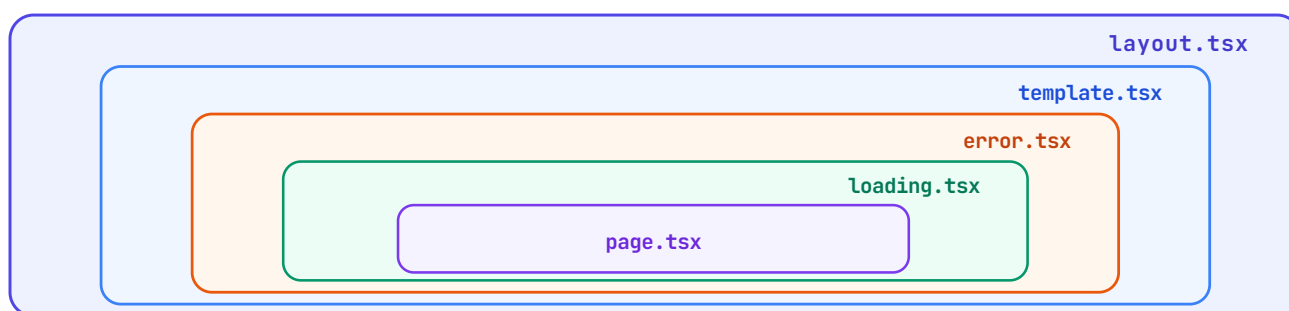
با `template: "%s | دکتر رزرو"` هر صفحه فقط بخش اختصاصی عنوان خود را تعریف می‌کند و بقیه به‌صورت خودکار اضافه می‌شود؛ مثلاً صفحه پزشکان عنوانش می‌شود «پزشکان | دکتر رزرو».

فایل‌های ویژه هر سگمنت

کنار `page.tsx` می‌توانید این فایل‌ها را بگذارید تا Next.js به صورت خودکار آن‌ها را در جای درست نمایش دهد:

فایل	نقش
<code>loading.tsx</code>	UI بارگذاری؛ به صورت خودکار صفحه را در یک <code><Suspense></code> می‌پیچد.
<code>error.tsx</code>	گرفتن خطاهای زمان اجرا در آن سگمنت (باید Client Component باشد).
<code>not-found.tsx</code>	UI حالت ۴۰۴ برای آن بخش.
<code>template.tsx</code>	مثل layout اما در هر نوبری از نو ساخته می‌شود (state حفظ نمی‌شود).

این فایل‌ها به صورت تودرتو یک صفحه را «می‌پیچند». دانستن این ترتیب کلیدی است: نزدیک‌ترین مرز، خطا یا بارگذاری را می‌گیرد.



src/app/doctors/loading.tsx

```
// ⌚ Shown instantly while the page's data is loading
export default function Loading() {
  return <p className="p-6 text-gray-500">در حال بارگذاری پزشکان</p>;
}
```

src/app/doctors/error.tsx

```
"use client"; // 🚨 error boundaries must be Client Components

export default function Error({
  error,
  reset,
}): {
```

```

error: Error & { digest?: string };
reset: () => void;
}) {
  return (
    <div className="p-6">
      <p>مشکلی پیش آمد: {error.message}</p>
      {/* 🔄 reset() re-renders the segment to retry */}
      <button onClick={() => reset()}>تلاش دوباره</button>
    </div>
  );
}

```

نکته Production: خطا را امن نمایش دهید

در Production نمایش مستقیم `error.message` می‌تواند جزئیات داخلی را لو دهد. یک پیام عمومی به کاربر نشان دهید و برای پیگیری، `error.digest` (شناسه خطای سمت سرور) را لاگ کنید. همچنین برای گرفتن خطای خودِ **Root Layout** از `global-error.tsx` در ریشه `app` استفاده کنید (این فایل باید تگ‌های `<html>` / `<body>` خودش را داشته باشد).

Metadata و سئو

برای سئوی هر صفحه دو راه دارید. اگر اطلاعات ثابت‌اند، یک آبجکت `metadata` اکسپورت کنید. اگر باید بر اساس داده (مثلاً نام پزشک) ساخته شوند، از تابع `generateMetadata` به نام `async` استفاده کنید:

```

src/app/doctors/[id]/page.tsx

import type { Metadata } from "next";

// 🚀 Dynamic, data-driven metadata per page
export async function generateMetadata({
  params,
}: {
  params: Promise<{ id: string }>;
}): Promise<Metadata> {
  const { id } = await params;
  const doctor = await getDoctor(id); // your data source
  return {
    title: doctor.name, // → "دکتر احمدی | دکتر رزرو"
    description: `رزرو نوبت با ${doctor.name}`,
  };
}

```

فایل‌های ویژه تصویر

کافی است فایل‌هایی با نام‌های `apple-icon.png`، `icon.png` یا `opengraph-image.png` را داخل سگمنت بگذارید؛ Next.js به صورت خودکار تگ‌های مربوطه را در `<head>` تولید می‌کند. برای OG داینامیک هم می‌توانید `opengraph-image.tsx` بنویسید. نکته `Production`: در `Root metadata` حتماً `metadataBase: new URL(process.env.NEXT_PUBLIC_BASE_URL!)` را تنظیم کنید تا URL‌های تصویر OG به صورت مطلق و درست تولید شوند.

🚀 مینی‌پروژه: پوسته سایت + متادیتای پویا

یک (Root) `app/layout.tsx` با `lang="fa" dir="rtl"` و متادیتای پیش‌فرض قالب‌دار بسازید. سپس برای `app/doctors/[id]/page.tsx` با `generateMetadata` عنوان و توضیح وابسته به نام پزشک تولید کنید، و کنار همان صفحه `loading.tsx` و `error.tsx` بگذارید. در آخر یک `opengraph-image.tsx` ساده اضافه کنید و در ابزار پیش‌نمایش لینک، نتیجه را ببینید.

🎤 سؤالات مصاحبه (با پاسخ)

Junior – layout.tsx چه کار می‌کند؟ رابط کاربری مشترکی که چند صفحه را در بر می‌گیرد و هنگام ناوبری بین آن صفحه‌ها re-render نمی‌شود (state حفظ می‌شود). حتماً باید `children` را رندر کند؛ Root Layout هم تگ‌های `<body>` / `<html>` را دارد.

Mid – تفاوت layout و template؟ متادیتای ثابت در برابر پویا؟ `layout` بین ناوبری‌ها باقی می‌ماند و state را نگه می‌دارد؛ `template` در هر ناوبری از نو ساخته می‌شود (برای انیمیشن ورود یا ریست state مفید است). برای سئو، اگر اطلاعات ثابت‌اند آبجکت `metadata` اکسپورت می‌کنیم؛ اگر به داده وابسته‌اند از `generateMetadata` (async) استفاده می‌کنیم.

Senior – سلسله‌مراتب فایل‌های ویژه چطور کار می‌کند و error.tsx چرا باید Client باشد؟ Next.js صفحه را در مرزهای `page` → `loading(Suspense)` → `error(ErrorBoundary)` → `template` → `layout` می‌پیچد؛ هر سگمنت می‌تواند نسخه خودش را داشته باشد و نزدیک‌ترین مرز، خطا/بارگذاری را می‌گیرد. `error.tsx` باید Client باشد چون Error Boundary‌ها به state و رویداد سمت کلاینت (تابع `reset`) نیاز دارند.

06 پیوند و ناوبری (Linking & Navigating)

ناوبری در Next.js فقط «رفتن به صفحه بعد» نیست؛ یک سیستم بهینه‌شده با prefetch، کش و گذارهای سمت کلاینت است. در این فصل کامل می‌بینیم چطور کار می‌کند و چطور کنترلش کنیم.

ناوبری چطور کار می‌کند؟

وقتی یک `<Link>` در دید کاربر قرار می‌گیرد، Next.js مسیر مقصد را به صورت خودکار prefetch می‌کند (در پس‌زمینه از قبل می‌گیرد). با کلیک، به جای بارگذاری کامل صفحه، یک گذار سمت کلاینت رخ می‌دهد: فقط بخش‌های تغییر یافته رندر می‌شوند و layoutهای مشترک حفظ می‌مانند. نتیجه: ناوبری تقریباً آنی.

کامپوننت Link

برای هر پیوند داخلی از `Link` استفاده کنید، نه تگ خام `<a>`:

```
basic links

import Link from "next/link";

export function Menu() {
  return (
    <nav className="flex gap-4">
      <Link href="/">خانه</Link>
      <Link href="/doctors">پزشکان</Link>
      { /* dynamic href */ }
      <Link href={` /doctors/${42}`}>پروفایل پزشک</Link>
    </nav>
  );
}
```

کنترل prefetch و اسکرول

prefetch خودکار عالی است، اما گاهی می‌خواهید آن را خاموش کنید (مثلاً برای صدها لینک در یک جدول بزرگ که هزینه شبکه ایجاد می‌کنند). همچنین می‌توانید پرش اسکرول به بالای صفحه را غیرفعال کنید:

● ● ● prefetch + scroll control

```
// 🚫 disable prefetch for low-priority / numerous links
<Link href={` /items/${id}`} prefetch={false}>{name}</Link>

// 📄 keep the scroll position instead of jumping to top
<Link href="/feed?tab=new" scroll={false}>تب جدید</Link>
```

ناوبری برنامه‌نویسی‌شده با useRouter

برای ناوبری از داخل کد (مثلاً بعد از ثبت موفق فرم) از هوک `useRouter` استفاده کنید. حتماً از `next/navigation` ایمپورت شود، نه `next/router` قدیمی:

● ● ● src/components/save-button.tsx

```
"use client";
import { useRouter } from "next/navigation";

export function SaveButton() {
  const router = useRouter();
  async function onSave() {
    await save();
    router.push("/dashboard"); // ➡ go to a new route
    // other methods:
    // router.replace("/x"); // navigate without adding history entry
    // router.back(); // go back
    // router.refresh(); // re-fetch the current route's server data
    // router.prefetch("/heavy"); // manually warm a route
  }
  return <button onClick={onSave}>ذخیره</button>;
}
```

خواندن مسیر و پارامترهای فعلی

در Client Component ها برای دانستن آدرس فعلی یا کوئری‌استرینگ از این هوک‌ها استفاده کنید:

● ● ● usePathname + useSearchParams

```
"use client";
import { usePathname, useSearchParams } from "next/navigation";
```

```
export function Breadcrumb() {
  const pathname = usePathname(); // e.g. "/doctors/42"
  const tab = useSearchParams().get("tab"); // e.g. "reviews"
  return <span>{pathname}{tab && ` · ${tab}`}</span>;
}
```

نکته Suspense و Production: useSearchParams

کامپوننتی که از `useSearchParams` استفاده می‌کند باید درون یک `<Suspense>` قرار گیرد؛ وگرنه هنگام بیلد با هشدار مواجه می‌شوید و آن بخش به رندر سمت کلاینت مجبور می‌شود. این رایج‌ترین دام ناوبری در پروژه‌های واقعی است.

لینک فعال (Active Link)

یک الگوی پرکاربرد: متمایز کردن لینک صفحه جاری در منو. نکته Production: مقایسه دقیق (===) در مسیرهای تودرتو کافی نیست – با ورود به `/doctors/42` باید لینک والد `/doctors` هم فعال شود. پس برای خانه مقایسه دقیق و برای بقیه `startsWith`، به همراه `aria-current` برای دسترس‌پذیری:

src/components/nav-link.tsx

```
"use client";
import Link from "next/link";
import { usePathname } from "next/navigation";

export function NavLink({ href, children }: { href: string; children: React.ReactNode }) {
  const pathname = usePathname();
  // ✅ exact for home, prefix match for section roots (/doctors/42 → /doctors active)
  const active = href === "/" ? pathname === "/" : pathname.startsWith(href);
  return (
    <Link
      href={href}
      aria-current={active ? "page" : undefined} // ⓘ a11y: mark the current page
      className={active ? "font-bold text-blue-600" : "text-gray-600"}
    >
      {children}
    </Link>
  );
}
```

نمایش وضعیت «در حال رفتن» با `useLinkStatus`

اگر مقصد کند است، خوب است هنگام کلیک یک نشانگر بارگذاری نشان دهید. هوک `useLinkStatus` (از `next/link`) وضعیت ناوبری نزدیک‌ترین `Link` والد را می‌دهد:

inline pending indicator

```
"use client";
import Link from "next/link";
import { useLinkStatus } from "next/link";

function Pending() {
  const { pending } = useLinkStatus(); // ⌚ true while navigating to the parent Link
  return pending ? <span className="animate-spin">⌚</span> : null;
}

export function SlowLink() {
  return (
    <Link href="/reports">
      گزارش‌ها <Pending />
    </Link>
  );
}
```

`useLinkStatus` در برابر `loading.tsx`

`useLinkStatus` برای بازخورد فوری کنار خود لینک است (مثل اسپینر روی دکمه). فایل `loading.tsx` برای نمایش اسکلت کل آن سگمنت هنگام بارگذاری داده است. این دو مکمل‌اند: لینک، حس «کلیک ثبت شد» را می‌دهد و `loading`، اسکلت صفحه مقصد را.

ریدایرکت از سمت سرور

در `Server Component` ها، `Server Action` ها و `Route Handler` ها برای هدایت کاربر از توابع آماده استفاده کنید (نه `useRouter` که فقط کلاینتی است):

server-side redirects

```
import { redirect, permanentRedirect } from "next/navigation";

// 🚫 temporary (307) – e.g. guard an unauthenticated user
```

```
if (!user) redirect("/login");

// ∞ permanent (308) – e.g. an old URL moved for good (good for SEO)
permanentRedirect("/new-home");
```

اشتباهات رایج / روش درست

اشتباه ❌	درست ✅
تگ خام <code><a></code> برای پیوند داخلی	<code><Link></code> (گذار سمت کلاینت + prefetch)
ایمپورت <code>useRouter</code> از <code>next/router</code>	از <code>next/navigation</code>
Active Link با <code>router</code> مسیر تودرتو	خانه دقیق، بقیه <code>aria-current</code> + <code>startsWith</code>
<code>useSearchParams</code> بدون <code>Suspense</code>	پیچیدن در <code><Suspense></code>
ریدایرکت با <code>useRouter</code> در کد سرور	<code>redirect</code> / <code>permanentRedirect</code>

مینوپروژه: نوار ناوبری حرفه‌ای 🚀

یک `<Navbar />` با چند `NavLink` بسازید که لینک فعال را درست (شامل مسیرهای تودرتو) و با `aria-current` مشخص کند. سپس روی یک لینک کند، با `useLinkStatus` یک اسپینر «در حال رفتن» اضافه کنید و برای یک لینک کم‌اهمیت `prefetch={false}` بگذارید.

سؤالات مصاحبه (با پاسخ) 🎤

Junior – چرا `<Link>` به جای `<a>`؟ چون گذار سمت کلاینت انجام می‌دهد (بدون رفرش کامل)، layoutهای مشترک و state را حفظ می‌کند و مسیر مقصد را خودکار prefetch می‌کند؛ نتیجه ناوبری تقریباً آنی است.

Mid – «گذار نرم» (soft navigation) یعنی چه و prefetch چطور کار می‌کند؟ فقط بخش‌های تغییریافته درخت دوباره رندر می‌شوند و layoutهای مشترک باقی می‌مانند. Next.js وقتی `Link` وارد دید کاربر می‌شود مقصد را در پس‌زمینه prefetch می‌کند (فقط در Production).

Senior – یک `Active Link` واقعاً Production چه ویژگی‌هایی دارد؟ تطبیق درست مسیرهای تودرتو (خانه دقیق، بخش‌ها با `startsWith`)، دسترس‌پذیری با `aria-current="page"`، آگاهی از query در صورت نیاز، و پیچیدن هر مصرف‌کننده `useSearchParams` در `Suspense` برای جلوگیری از CSR اجباری.

جمع‌بندی ناوبری

برای پیوندها `Link` (با prefetch خودکار)؛ برای ناوبری برنامه‌نویسی‌شده `useRouter`؛ برای دانستن مکان فعلی `usePathname` / `useSearchParams`؛ برای بازخورد کلیک `useLinkStatus`؛ و برای هدایت سمت سرور `redirect` / `permanentRedirect`. همین چند ابزار، تمام نیازهای ناوبری یک اپ واقعی را پوشش می‌دهند.

07 Client Components و Server Components

این مهم‌ترین تغییر ذهنی Next.js نسبت به React سنتی است. اگر این یک مفهوم را عمیق بفهمید، بقیه فریم‌ورک ساده می‌شود؛ پس این فصل را کامل و با تمرکز بخوانید.

۱ | مفهوم چیست؟

در App Router، هر کامپوننتی که می‌نویسید پیش‌فرض روی سرور اجرا می‌شود و اسمش **Server Component** است؛ کدش هرگز به مرورگر کاربر نمی‌رود. فقط بخش‌هایی که واقعاً به تعامل نیاز دارند (کلیک، تایپ، انیمیشن) را با نوشتن `"use client"` به **Client Component** تبدیل می‌کنید تا در مرورگر زنده شوند. به بیان ساده: «همه چیز روی سرور، مگر جزیره‌های کوچک تعاملی».

۲ | چرا ساخته شد و چه مشکلی را حل می‌کند؟

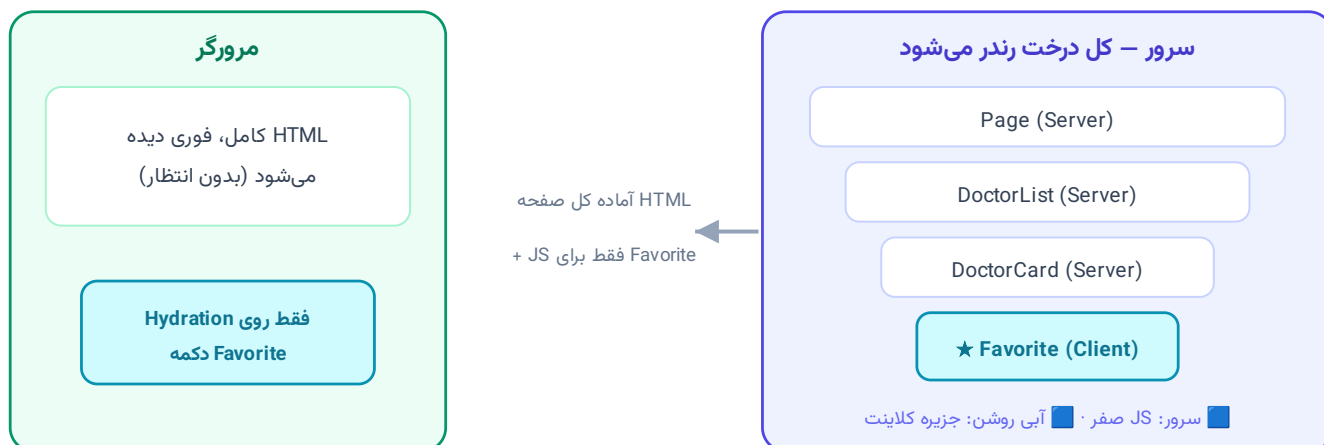
در React سنتی (SPA) کل برنامه در مرورگر اجرا می‌شد. این چهار درد را داشت:

- ♦ **باندل سنگین JS:** کد هر کامپوننت – حتی یک متن ثابت – به مرورگر می‌رفت و صفحه را کند می‌کرد.
- ♦ **آبشار واکنشی سمت کلاینت:** مرورگر اول HTML خالی می‌گرفت، بعد JS، بعد تازه `useEffect` داده را می‌گرفت؛ کاربر مدت‌ها اسپینر می‌دید.
- ♦ **لورفتن منطق و اسرار:** برای گرفتن داده باید یک API می‌ساختید و کلیدها را با احتیاط پنهان می‌کردید.
- ♦ **سئوی ضعیف:** محتوای مهم بعد از اجرای JS ساخته می‌شد.

Server Components هر چهار مشکل را از ریشه حل می‌کند: کامپوننت روی سرور اجرا و به HTML تبدیل می‌شود، کدش در مرورگر بارگذاری نمی‌شود (باندل سبک)، مستقیماً به دیتابیس/اسرار دسترسی دارد (بدون API میانی)، و محتوا از همان ابتدا در HTML حاضر است (سئوی عالی).

۳ | Mental Model – مدل ذهنی

صفحه را مثل یک «سند سرور» تصور کنید که بیشترش ثابت و غیرتعاملی است، و فقط چند «جزیره تعاملی» (Client Island) کوچک دارد. سرور کل درخت را رندر می‌کند؛ اما فقط برای همان جزیره‌ها JavaScript به مرورگر می‌فرستد و فقط همان‌ها در مرورگر «زنده» (hydrate) می‌شوند.



۴ | نحوه کار داخلی (Behind the Scenes)

جریان یک درخواست دقیقاً این‌طور است:

- ♦ مرورگر صفحه را می‌خواهد؛ درخواست به سرور می‌رسد.
- ♦ سرور درخت Server Component ها را اجرا می‌کند (داده می‌گیرد، HTML می‌سازد). خروجی یک «RSC Payload» است: توصیفی فشرده از UI به‌علاوه نشانه‌گذاری جایگاه Client Component ها.
- ♦ سرور HTML اولیه + این Payload را می‌فرستد؛ مرورگر بلافاصله صفحه کامل را نشان می‌دهد.
- ♦ سپس فقط Client Component JavaScript ها بارگذاری می‌شود و React آن جزیره‌ها را «hydrate» می‌کند (رویدادها را وصل می‌کند). بقیه صفحه هیچ JS لای ندارد.

چرا «serializable» بودن props مهم است؟

وقتی یک Server Component به یک Client Component props می‌دهد، آن props باید از سرور به کلاینت «سریالایز» شود. برای همین می‌توانید داده (رشته، عدد، آبجکت، آرایه) پاس دهید، اما نمی‌توانید تابع پاس دهید (تابع سریالایز نمی‌شود). این محدودیت مستقیماً از همین معماری می‌آید.

۵ | چه زمانی Server Component؟ / ۶ | چه زمانی Client Component؟

قاعده تصمیم ساده است: تا وقتی به تعامل نیاز نداری، Server بمان.

انتخاب	اگر نیاز داری به...
Server Component	واکشی داده، کوئری دیتابیس، خواندن اسرار
Server Component	محتوای ثابت/نمایشی، کاهش باندل، سئو
Client Component	Context ، useEffect ، useState
Client Component	رویدادها (onClick ، onChange)
Client Component	API های مرورگر (localStorage ، window)
Client Component	کتابخانه‌ای که فقط در مرورگر کار می‌کند

Trade-off: Client Component بد نیست – برای تعامل ضروری است. اشتباه، «Client کردن بیش از حد» است: هر `"use client"` یعنی JS بیشتر و رندر اولیه سنگین‌تر. پس Client را فقط برای همان جزء تعاملی نگه دارید، نه کل صفحه.

۷ | مثال عمیق: فهرست پزشکان + دکمه ذخیره

سناریو: صفحه فهرست پزشکان که داده‌اش روی سرور واکشی می‌شود (سبک و سئوپذیر)، اما هر کارت یک دکمه «ذخیره» تعاملی دارد. کل صفحه Server است و فقط دکمه یک جزیره Client.

```

file structure

src/
├─ app/doctors/page.tsx      # Server: fetches + renders the list
├─ components/
│   ├─ doctor-card.tsx      # Server: presentational
│   └─ save-button.tsx      # Client: the only interactive island

```

صفحه یک Server Component است که داده را روی سرور می‌گیرد و فهرست را می‌سازد:

```

src/app/doctors/page.tsx

// 🖱️ Server Component: fetch on the server, render the list (ships zero JS)
import { DoctorCard } from "@components/doctor-card";

async function getDoctors(): Promise<Doctor[]> {
  const res = await fetch("https://api.example.com/doctors");

```

```

    if (!res.ok) throw new Error("خطا در دریافت فهرست پزشکان");
    return res.json();
  }

export default async function DoctorsPage() {
  const doctors = await getDoctors(); // ⌚ awaited on the server
  return (
    <ul className="grid gap-4">
      {doctors.map((d) => <DoctorCard key={d.id} doctor={d} />)}
    </ul>
  );
}

```

کارت پزشک یک Server Component نمایشی است؛ فقط دکمه‌ی داخلش Client است:

```

src/components/doctor-card.tsx

// 🟦 Server Component: pure presentation → no JS shipped for this
import { SaveButton } from "../save-button";

export function DoctorCard({ doctor }: { doctor: Doctor }) {
  return (
    <li className="rounded-xl border p-4">
      <h3 className="font-bold">{doctor.name}</h3>
      <p className="text-sm text-gray-500">{doctor.specialty}</p>
      { /* 🖱️ only this small island becomes interactive on the client */ }
      <SaveButton doctorId={doctor.id} />
    </li>
  );
}

```

```

src/components/save-button.tsx

"use client"; // 🖱️ this is the ONLY interactive island in the card

import { useState } from "react";

export function SaveButton({ doctorId }: { doctorId: string }) {
  const [saved, setSaved] = useState(false);
  return (
    <button onClick={() => setSaved((s) => !s)} className="mt-2 text-sm text-blue-600">
      {saved ? "★ ذخیره ☆" : "ذخیره شد"}
    </button>
  );
}

```

```

    </button>
  );
}

```

چرا این تقسیم درست است؟

کل فهرست (که سنگین و پرتعداد است) روی سرور به HTML تبدیل می‌شود و هیچ JS ای برای آن به کلاینت نمی‌رود؛ فقط کد کوچک `SaveButton` بارگذاری می‌شود. اگر کل صفحه را Client می‌کردیم، باندل چند برابر و رندر اولیه کندتر می‌شد.

الگوی ترکیب: جا دادن Server داخل Client

یک سوءتفاهم رایج: «اگر والد Client باشد، همه فرزندان هم Client می‌شوند.» درست است — اما فقط اگر آن‌ها را `import` کنید. اگر Server Component را به صورت `children` (یا prop) به Client Component بدهید، همچنان روی سرور می‌ماند:

```

src/app/page.tsx

import { Modal } from "@components/modal"; // a Client Component
import { DoctorList } from "@components/doctor-list"; // a Server Component

export default function Page() {
  // 🧩 DoctorList is passed as children → it stays a Server Component
  return (
    <Modal>
      <DoctorList />
    </Modal>
  );
}

```

اشتباهات رایج / روش درست

اشتباه ❌	روش درست ✅
گذاشتن <code>"use client"</code> بالای کل صفحه	فقط جزء تعاملی را Client کنید (پایین درخت)
پاس دادن یک تابع از Server به Client به عنوان prop	داده را پاس دهید؛ برای اکشن‌ها از Server Action استفاده کنید
خواندن <code>process.env.SECRET</code> در Client	اسرار فقط در Server Component/DAL
کردن یک Server Component داخل فایل <code>import</code> Client	آن را به صورت <code>children</code> پاس دهید
استفاده از <code>useState</code> در Server Component	آن بخش را جدا و Client کنید

Best Practices و نکات طلایی ✅

۱) پیش‌فرض را Server بگذارید؛ Client تصمیم آگاهانه باشد نه عادت. ۲) مرز Client را تا حد ممکن پایین ببرید (الگوی «برگ‌های تعاملی»). ۳) props ارسالی به Client را کوچک و serializable نگه دارید. ۴) اگر فقط برای یک `onClick` کوچک کل کامپوننت را Client می‌کنید، احتمالاً می‌شود همان دکمه را جدا کرد. ۵) کتابخانه‌های سنگین فقط کلاینتی را داخل Client Component و در صورت امکان با `dynamic(..., { ssr:false })` بارگذاری کنید.

مینی‌پروژه: کارت پروفایل کاربر + دکمه فالو 🚀

یک `app/users/[id]/page.tsx` بسازید که پروفایل کاربر را روی سرور واکنشی و رندر کند (نام، بیو، تعداد دنبال‌کننده). سپس یک `<FollowButton userId=... />` به صورت Client Component بسازید که با `useState` وضعیت «دنبال‌شده/نشده» را نگه دارد. هدف: تجربه عملی مرز Server/Client – صفحه سروری و سبک، فقط دکمه تعاملی.

خلاصه فصل 📌

پیش‌فرض همه چیز Server Component است (JS صفر، دسترسی مستقیم به داده/اسرار، سئوی عالی). فقط برای تعامل، state و API‌های مرورگر `"use client"` بزنید و آن را پایین درخت نگه دارید. Server را می‌توان به صورت `children` داخل Client جا داد. props بین سرور و کلاینت باید serializable باشد (تابع نه).

تمرین

۱) یک صفحه بسازید که یک متن ثابت + یک شمارنده دارد؛ کاری کنید که فقط شمارنده Client باشد و در DevTools ببینید که JS صفحه چقدر کم است. ۲) تلاش کنید یک تابع را از Server به Client پاس دهید و خطای آن را ببینید و تفسیر کنید. ۳) یک Server Component را یکبار با `import` و یکبار با `children` داخل یک Client بگذارید و تفاوت رفتار را توضیح دهید.

سوالات مصاحبه (با پاسخ)

Junior – تفاوت Server و Client Component؟ Server Component پیش‌فرض است، فقط روی سرور اجرا می‌شود، به مرورگر ارسال نمی‌شود و می‌تواند مستقیم داده بگیرد؛ اما state/رویداد ندارد. Client Component با `"use client"` مشخص می‌شود و برای تعامل، state و API‌های مرورگر است.

Mid – چرا Server Components باندل را کوچک می‌کند؟ چون کد Server Component ها هرگز به مرورگر ارسال نمی‌شود؛ فقط خروجی HTML/RSC-Payload می‌رود. فقط Client Component ها JS همراه دارند، پس هرچه Client کمتر باشد، باندل سبک‌تر است.

Senior – مرز Client را کجا می‌گذارید و چرا نمی‌توان تابع را به Client پاس داد؟ مرز را تا حد ممکن پایین درخت و فقط دور جزء تعاملی می‌گذارم تا بیشترین بخش Server (و بدون JS) بماند. تابع را نمی‌توان پاس داد چون props عبوری از مرز سرور → کلاینت باید serializable باشد؛ برای عملیات سمت سرور به جای پاس دادن تابع، از Server Action استفاده می‌کنم.

08 واکنشی داده و Streaming

داده، خونِ هر اپلیکیشن است. این فصل نشان می‌دهد در Next.js 16 داده را کجا، چطور و با چه استراتژی‌ای بگیرید تا اپ هم سریع باشد، هم ساده – از واکنشی سروری تا استریم تدریجی صفحه.

۱ | مفهوم چیست؟

«واکنشی داده» یعنی گرفتن اطلاعات (از API یا دیتابیس) و رساندن آن به UI. در App Router، چون کامپوننت‌ها پیش‌فرض روی سرور اجرا می‌شوند، واکنشی هم روی سرور و مستقیماً داخل خود کامپوننت انجام می‌شود: کامپوننت را `async` می‌کنید و داده را `await` می‌زنید – درست مثل نوشتن کد بک‌اند. **Streaming** هم یعنی به‌جای انتظار برای آماده‌شدن کل صفحه، بخش‌های آماده را فوری بفرستید و بخش‌های کند را به‌تدریج «جاری» کنید.

۲ | چرا این روش ساخته شد و چه مشکلی را حل می‌کند؟

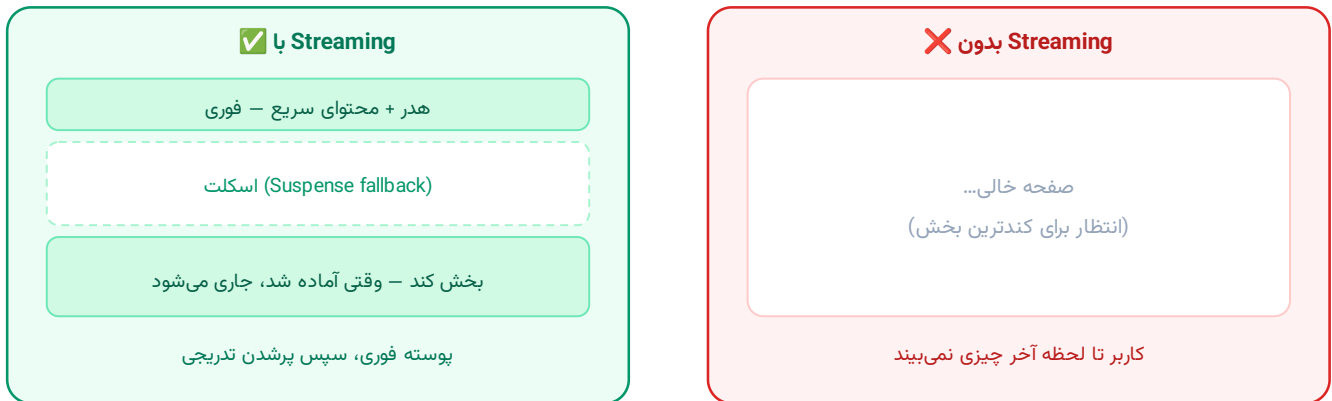
در React سنتی، واکنشی سمت کلاینت با `useEffect` انجام می‌شد و این «آبشار» دردناک را می‌ساخت:

- ♦ مرورگر HTML خالی می‌گرفت → بعد باندل JS را دانلود می‌کرد → بعد `useEffect` اجرا و تازه درخواست داده ارسال می‌شد → کاربر مدت‌ها اسپینر می‌دید.
 - ♦ برای هر واکنشی به state بارگذاری/خطا و اغلب یک کتابخانه (مثل React Query) نیاز بود.
 - ♦ محتوا بعد از اجرای JS ساخته می‌شد، پس سئو ضعیف بود.
- واکنشی سروری هر سه را حل می‌کند: داده همان لحظه که HTML ساخته می‌شود آماده است (بدون آبشار)، بدون state/کتابخانه اضافه، و محتوا از ابتدا در HTML حاضر است (سئوی عالی). **Streaming** هم مشکل «صفحه خالی تا آماده‌شدن کندترین بخش» را حل می‌کند.

۳ | Mental Model – مدل ذهنی

سرور را مثل آشپزی تصور کنید که به‌جای آوردن کل سفارش یک‌جا (و معطل‌کردن مشتری)، ابتدا نان و پیش‌غذا (پوسته صفحه) را فوری می‌آورد و غذای اصلی (بخش کند) را وقتی آماده شد سر می‌آورد. **Streaming** دقیقاً همین است: پوسته صفحه فوری، بخش‌های کند به‌تدریج.

Streaming در برابر با Streaming بدون



۴ نحوه کار داخلی (Behind the Scenes)

- سرور رندر درخت را آغاز می‌کند. به یک مرز `<Suspense>` که می‌رسد و داده‌اش هنوز آماده نیست، **منتظر نمی‌ماند**؛ به جایش `fallback` را در HTML اولیه می‌گذارد.
- HTML پوسته (بخش‌های آماده + fallback ها) فوری با *chunked transfer* به مرورگر می‌رود؛ کاربر بلافاصله چیزی می‌بیند.
- وقتی داده هر مرز آماده شد، سرور همان تکه را «استریم» می‌کند و React آن را جای `fallback` می‌نشاند – بدون رفرش، بدون درخواست جدید از مرورگر.

واکشی در Server Component

چون Server Component ها روی سرور اجرا می‌شوند، همان‌جا `await` می‌زنید؛ نه `useEffect`، نه `state` بارگذاری، نه کتابخانه جانبی:

```
src/app/doctors/page.tsx

// Async Server Component: fetch right inside the component
async function getDoctors(): Promise<Doctor[]> {
  const res = await fetch("https://api.example.com/doctors");
  if (!res.ok) throw new Error("خطا در دریافت فهرست پزشکان"); // handled by error.tsx
  return res.json();
}

export default async function DoctorsPage() {
  const doctors = await getDoctors(); // awaited on the server
  return (
    <ul>
```

```

    {doctors.map((d) => <li key={d.id}>{d.name}</li>)}
  </ul>
);
}

```

اگر بک‌اند جدا ندارید و مستقیم با دیتابیس کار می‌کنید (Mongoose/Prisma)، همان کوئری را داخل Server Component بزنید – بدون API میانی:

```

src/app/doctors/page.tsx

import { db } from "@/lib/db";

export default async function DoctorsPage() {
  // 🗄️ Talk to the database directly from a Server Component
  const doctors = await db.doctor.findMany({ orderBy: { name: "asc" } });
  return <DoctorGrid items={doctors} />;
}

```

رفتار جدید fetch در نسخه ۱۶

در نسخه ۱۶ کشینگ خودکار حذف شده است؛ یعنی `fetch` پیش‌فرض هر بار تازه اجرا می‌شود (مثل `{ cache: "no-store" }`). اگر می‌خواهید نتیجه کش شود، باید آگاهانه از مدل Cache Components (فصل بعد) استفاده کنید. این تغییر یعنی رفتار «پیش‌بینی‌پذیر» شده: چیزی پشت سرتان کش نمی‌شود مگر خودتان بخواهید.

۵ | چه زمانی از این ابزارها استفاده کنیم؟ / ۶ | چه زمانی نه؟

موقعیت	انتخاب درست
واکشی داده برای نمایش اولیه صفحه	✓ واکشی سروری در Server Component
بخشی از صفحه کند است ولی بقیه آماده	✓ همان بخش را در <code><Suspense></code> بپیچید (Streaming)
کل صفحه به یک واکشی وابسته است	✓ فایل <code>loading.tsx</code>
داده به تعامل کاربر بستگی دارد (جستجوی زنده، صفحه‌بندی بی‌نهایت)	👉 واکشی سمت کلاینت با SWR (فصل مدیریت state)
هر بخش کوچک را در <code>Suspense</code> جدا می‌پیچید	✗ نکنید؛ <code>Suspense</code> ‌های بیش‌ازحد ریز، تجربه را «لرزان» می‌کند

Suspense با Streaming

قدرت واقعی اینجاست: بخش‌های سریع را فوری نشان دهید و بخش‌های کند را داخل `<Suspense>` بپیچید تا به‌تدریج استریم شوند:

```
src/app/dashboard/page.tsx

import { Suspense } from "react";

// 🐢 A slow component that fetches heavy data
async function Appointments() {
  const data = await getAppointments(); // takes a while
  return <AppointmentTable rows={data} />;
}

export default function Dashboard() {
  return (
    <section>
      <h1>داشبورد</h1>
      { /* ⚡ Header shows instantly; the table streams in when ready */ }
      <Suspense fallback={<p>در حال بارگذاری نوبت‌ها</p>}>
        <Appointments />
      </Suspense>
    </section>
  );
}
```

```

    </section>
  );
}

```

Suspense در برابر loading.tsx

اگر کل صفحه باید منتظر داده بماند، فایل `loading.tsx` ساده‌ترین راه است (Next.js خودش صفحه را در `Suspense` می‌پیچد). اما اگر می‌خواهید فقط بخشی از صفحه منتظر بماند و بقیه فوری بیاید، از `<Suspense>` دستی دور همان بخش استفاده کنید.

۷ مثال عمیق: داشبورد با بخش‌های موازی و استریم‌شونده

سناریو: داشبورد یک سیستم رزرو که سه بخش دارد: کارت‌های آمار (سریع)، فهرست نوبت‌های امروز (متوسط)، و نمودار درآمد ماه (کند). می‌خواهیم پوسته صفحه و آمار فوری بیایند و دو بخش دیگر مستقل و موازی استریم شوند — تا کندی نمودار، فهرست را معطل نکند.

```

src/app/(dashboard)/page.tsx

import { Suspense } from "react";
import { StatsCards } from "@features/dashboard/stats-cards";
import { TodayAppointments } from "@features/dashboard/today-appointments";
import { RevenueChart } from "@features/dashboard/revenue-chart";
import { CardSkeleton, ChartSkeleton } from "@features/dashboard/skeletons";

export default async function DashboardPage() {
  // ⚡ Fast, awaited inline → part of the instant shell
  return (
    <main className="space-y-6 p-6">
      <h1 className="text-2xl font-bold">داشبورد</h1>

      {/* 📊 stats are quick → render as part of the shell */}
      <StatsCards />

      {/* 🌱 each slow section streams INDEPENDENTLY (own Suspense boundary) */}
      <Suspense fallback={<CardSkeleton rows={5} />}>
        <TodayAppointments />
      </Suspense>

      <Suspense fallback={<ChartSkeleton />}>
        <RevenueChart />
      </Suspense>
    </main>
  );
}

```

```

    </main>
  );
}

```

src/features/dashboard/revenue-chart.tsx

```

import { getMonthlyRevenue } from "../queries";

// 🐢 Slow server component – but it no longer blocks the rest of the page
export async function RevenueChart() {
  const points = await getMonthlyRevenue(); // heavy aggregation query
  return <Chart data={points} />;
}

```

چرا این معماری؟ هر `<Suspense>` یک «مرز مستقل» است؛ پس فهرست نوبت‌ها و نمودار درآمد هم‌زمان روی سرور شروع می‌شوند و هرکدام هر وقت آماده شد جاری می‌شود. اگر هر دو را در یک `Suspense` می‌گذاشتیم، کاربر باید تا آماده‌شدن کندترین‌شان صبر می‌کرد.

واکشی ترتیبی در برابر موازی (آبشار درخواست‌ها)

اگر چند منبع مستقل را پشت‌سرهم `await` کنید، «آبشار» می‌سازید: هر درخواست تا پایان قبلی شروع نمی‌شود و تأخیرها جمع می‌شوند:

sequential waterfall (slow)

```

export default async function ProfilePage() {
  const user = await getUser(); // ⏰ wait...
  const appointments = await getAppointments(); // ⏰ ...then wait again (waterfall)
  return <Profile user={user} appointments={appointments} />;
}

```

چون این دو به هم وابسته نیستند، باید هم‌زمان شروع شوند؛ با `Promise.all`:

parallel (fast)

```

export default async function ProfilePage() {
  // 🚀 kick off both at once → total time = the slower one (not the sum)
  const [user, appointments] = await Promise.all([getUser(), getAppointments()]);
  return <Profile user={user} appointments={appointments} />;
}

```

```
}
```

چه زمانی ترتیبی درست است؟

وقتی یک درخواست به نتیجه قبلی نیاز دارد (اول کاربر را می‌گیرید تا `userId` را برای واکنشی نوبت‌هایش داشته باشید)، ترتیبی بودن اجتناب‌ناپذیر و درست است. آبشار فقط وقتی «بد» است که درخواست‌ها مستقل‌اند ولی بی‌دلیل پشت‌سرهم `await` شده‌اند.

الگوی preload برای جلوگیری از آبشار والد-فرزند

گاهی فرزند به داده‌ای نیاز دارد اما والد پیش از رسیدن به آن کاری (مثل بررسی دسترسی) انجام می‌دهد. با `preload` واکنشی را زودتر و بدون `await` شروع می‌کنید تا وقتی فرزند رسید، داده آماده باشد. این الگو روی `React.cache` سوار می‌شود:

src/lib/get-item.ts

```
import { cache } from "react";
import "server-only";
import { db } from "@lib/db";

// 🧠 cache() dedupes this call within one request
export const getItem = cache(async (id: string) => db.item.findUnique({ where: { id } }));

// 🔥 preload: start the request WITHOUT awaiting (fire-and-forget)
export const preload = (id: string) => { void getItem(id); };
```

src/app/items/[id]/page.tsx

```
import { preload, getItem } from "@lib/get-item";

export default async function Page({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  preload(id); // 🔥 start fetching NOW

  const allowed = await checkAccess(); // 🕒 runs in parallel with the item fetch
  if (!allowed) return <p>دسترسی ندارید</p>;

  return <Item id={id} />;
}
```

```

async function Item({ id }: { id: string }) {
  const item = await getItem(id); // ✅ already in-flight/cached → instant
  return <h1>{item?.name}</h1>;
}

```

❌ اشتباهات رایج / ✅ روش درست

اشتباه ❌	درست ✅
واکشی داده با <code>useEffect</code> در Client Component	واکشی مستقیم در Server Component
<code>await</code> پشت سرهم منابع مستقل	<code>Promise.all</code> برای موازی سازی
پیچیدن کل صفحه در یک <code>Suspense</code>	مرز <code>Suspense</code> مستقل دور هر بخش کند
فراموش کردن مدیریت خطای <code>fetch</code>	بررسی <code>res.ok</code> و پرتاب خطا (تا <code>error.tsx</code> بگیرد)
انتظار کش شدن خودکار <code>fetch</code> (مثل نسخه های قبل)	در v16 آگاهانه از <code>use cache</code> استفاده کنید

📌 Best Practices و نکات طلایی

۱) داده را نزدیک به محل مصرف واکشی کنید، نه همه را در بالای صفحه. ۲) منابع مستقل → همیشه
 ۳. `Promise.all` درخواست تکراری در یک رندر → ۴. `React.cache` بخش کند → `Suspense` مستقل
 با یک اسکلت هم/اندازه تا صفحه نپرد. ۵) برای واکشی وابسته به تعامل کاربر (جستجوی زنده) سمت کلاینت با
 SWR بروید، نه سرور.

🚀 مینی پروژه: صفحه محصولات با استریم بخش ها

یک `app/products/page.tsx` بسازید که ۱) فهرست محصولات را روی سرور با `Promise.all` همزمان
 با «دسته بندی ها» واکشی کند، و ۲) یک بخش «پیشنهاد ویژه» (کند) را در یک `<Suspense>` مستقل با
 اسکلت قرار دهد تا جدا استریم شود. فقط دکمه «افزودن به سبد» را Client کنید. نتیجه: صفحه سریع،
 سئوپذیر، و بدون آبخار.

خلاصه فصل 📌

در App Router داده را روی سرور و مستقیم داخل کامپوننت `async` واکنشی کنید. در `fetch` v16 پیش‌فرض تازه است (کش دستی است). منابع مستقل را با `Promise.all` موازی کنید و از آبشار بپرهیزید؛ برای آبشار والد-فرزند از `preload` استفاده کنید. بخش‌های کند را با `<Suspense>` مستقل استریم کنید تا پوسته صفحه فوری بیاید.

تمرین 📝

۱) یک صفحه بسازید که دو منبع مستقل را یک‌بار ترتیبی و یک‌بار با `Promise.all` بگیرد و اختلاف زمان بارگذاری را با یک `console.time` مقایسه کن. ۲) یک بخش کند بساز و آن را با/بدون `Suspense` امتحان کن و تفاوت لحظه «اولین دیدن محتوا» را توصیف کن. ۳) یک `error.tsx` اضافه کن و با `throw` در واکنشی، آن را فعال کن.

سؤالات مصاحبه (با پاسخ) 🗣️

Junior – چطور در Server Component داده می‌گیرید و چرا به `useEffect` نیاز نیست؟ کامپوننت را `async` می‌کنم و مستقیم `await` می‌زنم. چون کد روی سرور اجرا می‌شود، داده پیش از ساخت HTML آماده است؛ پس نیازی به واکنشی سمت کلاینت با `useEffect` و state بارگذاری نیست.

Mid – Streaming و Suspense چه مشکلی را حل می‌کنند و آبشار چیست؟ Streaming اجازه می‌دهد پوسته صفحه فوری بیاید و بخش‌های کند به‌تدریج جاری شوند، پس کاربر صفحه خالی نمی‌بیند. آبشار وقتی است که منابع مستقل پشت‌سرهم `await` شوند و تأخیرها جمع شود؛ راه‌حل، `Promise.all` و مرزهای `Suspense` مستقل است.

Senior – رفتار `fetch` در v16 چه تغییری کرده و `preload` کی لازم است؟ در v16 کش خودکار حذف شده و `fetch` پیش‌فرض تازه است؛ کش باید آگاهانه با `Cache Components` انتخاب شود، که رفتار را پیش‌بینی‌پذیر می‌کند. `preload` وقتی لازم است که والد پیش از رندر فرزند داده‌بر، کار دیگری انجام می‌دهد؛ با شروع زود هنگام واکنشی (روی `React.cache`)، از آبشار والد-فرزند جلوگیری می‌کنیم.

09 کشینگ مدرن با Cache Components

بزرگ‌ترین تغییر نسخه ۱۶. مدل کشینگ از «پیش‌فرض پنهان» به «انتخاب صریح» تغییر کرد. این فصل یک مدل ذهنی روشن به شما می‌دهد تا دقیقاً بدانید چه چیزی، چطور و چقدر کش کنید.

۱ | مفهوم چیست؟

«کش» یعنی نتیجه یک کار پرهزینه (واکشی داده یا رندر) را ذخیره کنیم تا دفعه بعد به‌جای تکرار، همان نتیجه آماده را فوری تحویل دهیم. Cache Components سازوکار نسخه ۱۶ برای این کار است: با دایرکتیو `use cache` صراحتاً می‌گویید «خروجی این تابع/کامپوننت/صفحه را کش کن».

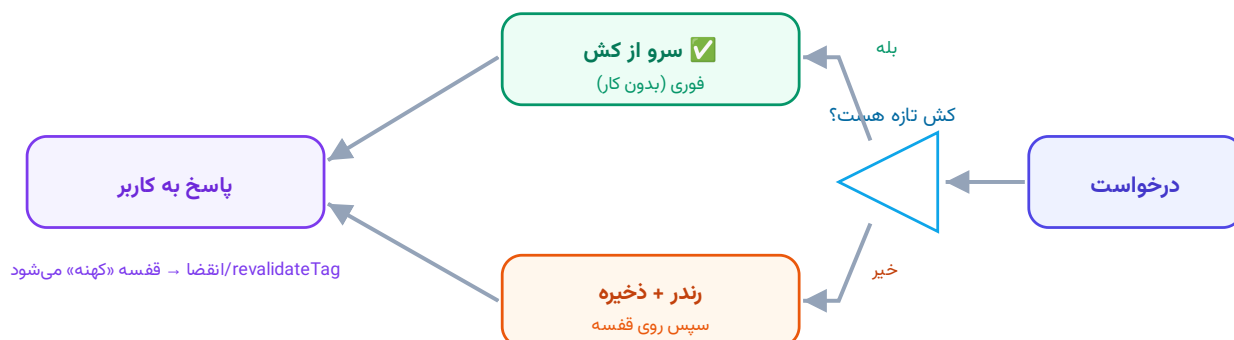
۲ | چرا ساخته شد و چه مشکلی را حل می‌کند؟

در نسخه‌های قبل، Next.js تلاش می‌کرد خودکار همه چیز را کش کند. نتیجه، رفتاری گیج‌کننده بود: توسعه‌دهنده نمی‌دانست چرا داده تازه‌اش نمایش داده نمی‌شود، یا برعکس چرا چیزی کش نشده. نسخه ۱۶ این را وارونه کرد:

- ◆ **پیش‌فرض = پویا:** هیچ چیز خودکار کش نمی‌شود؛ رفتار قابل‌پیش‌بینی است.
- ◆ **کش = تصمیم صریح شما:** فقط چیزی کش می‌شود که خودتان با `use cache` علامت بزنید.
- ◆ **کنترل کامل عمر و باطل‌سازی:** با `cacheLife` و `cacheTag` دقیقاً تعیین می‌کنید کش چه مدت زنده بماند و کی تازه شود.

۳ | Mental Model – مدل ذهنی

کش را مثل یک «قفسه برچسب‌دار» تصور کنید. هر درخواست می‌پرسد: «آیا برای این کلید، نتیجه‌ای تازه روی قفسه هست؟» اگر بله، همان را فوری برمی‌دارد (سریع)؛ اگر نه، کار را انجام می‌دهد، نتیجه را روی قفسه می‌گذارد و تحویل می‌دهد. برچسب‌ها (`cacheTag`) اجازه می‌دهند بعداً یک قفسه خاص را «کهنه» اعلام کنید.



۴ | نحوه کار داخلی (Behind the Scenes)

- وقتی تابعی `use cache` دارد، Next.js یک «کلید کش» از نام تابع + آرگومان‌ها + مقادیر بسته‌شده (closure) می‌سازد. ورودی‌های متفاوت = ورودی‌های کش جدا.
- خروجی سریالایز و ذخیره می‌شود؛ درخواست بعدی با همان کلید، بدون اجرای دوباره، همان خروجی را می‌گیرد.
- `cacheLife` «تازگی» را تعیین می‌کند و `cacheTag` یک برچسب می‌چسباند تا `revalidateTag` بتواند آن ورودی را باطل کند.

فعال‌سازی و دایرکتیو `use cache`

ابتدا قابلیت را روشن کنید:

```

next.config.ts

import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  cacheComponents: true, // 🔒 enable the `use cache` directive (stable in v16)
};

export default nextConfig;
  
```

`use cache` را در سه سطح می‌توان به کار برد. ۱) سطح داده (کش کردن یک تابع واکنشی):

src/lib/queries.ts

```
export async function getSpecialties() {
  "use cache"; // 🧠 cache this function's return value (keyed by its args)
  const res = await fetch("https://api.example.com/specialties");
  return res.json();
}
```

۲) سطح کامپوننت/صفحه (کش کردن کل خروجی رندر) به همراه تعیین عمر:

src/app/specialties/page.tsx

```
import { cacheLife } from "next/cache";

export default async function SpecialtiesPage() {
  "use cache"; // 📁 cache the whole rendered page
  cacheLife("hours"); // 🕒 stays fresh for ~hours

  const list = await getSpecialties();
  return <ul>{list.map((s: { id: string; title: string }) => <li key={s.id}>{s.title}</li>)}</ul>;
}
```

سه قانون مهم `use cache`

۱) هر تابع دارای `use cache` باید `async` باشد. ۲) آرگومان‌ها و مقادیر closure خودکار بخشی از «کلید کش» می‌شوند. ۳) داخل `scope` کش‌شده مستقیم `cookies()` / `headers()` را نخوانید؛ بیرون بخوانید و به‌عنوان آرگومان پاس دهید.

عمر کش با `cacheLife` و باطل‌سازی با `cacheTag`

مناسب برای	پرو فایل <code>cacheLife</code>
داده‌های نیمه‌زنده مثل موجودی نوبت	"seconds" / "minutes"
محتوای کم‌تغییر مثل تخصص‌ها یا مقالات	"hours" / "days"
محتوای تقریباً ثابت	"weeks" / "max"

زمان‌بندی همیشه کافی نیست؛ برای تازه‌کردن «درست بعد از یک تغییر»، برچسب بزنید و بعد باطل کنید:

src/lib/posts.ts

```
import { cacheTag } from "next/cache";

export async function getPosts() {
  "use cache";
  cacheTag("posts"); // 🏷 label this cache entry
  return db.post.findMany();
}
```

inside a Server Action

```
"use server";
import { revalidateTag } from "next/cache";

export async function createPost(data: PostInput) {
  await db.post.create({ data });
  revalidateTag("posts"); // 🔄 invalidate the "posts" cache AFTER the mutation
}
```

کش خصوصی برای هر کاربر

گاهی محتوا برای هر کاربر متفاوت است (داشبورد شخصی) اما باز هم می‌خواهید کش شود. از

`use cache` استفاده کنید که کش را per-user نگه می‌دارد. حالت پیش‌فرض `"use cache: private"`

اشتراکی است؛ پس داده شخصی را به اشتباه اشتراکی کش نکنید.

۵ | چه زمانی کش کنیم؟ / ۶ | چه زمانی نه؟ (Trade-off تازگی در برابر سرعت)

نوع داده	تصمیم
عمومی و کم‌تغییر (تخصص‌ها، مقالات)	<code>use cache</code> + <code>cacheLife</code> ✓
تغییر با یک عملیات مشخص	<code>cacheTag</code> + <code>revalidateTag</code> ✓
شخصی هر کاربر ولی کش‌پذیر	<code>use cache: private</code> ✓
واقعاً زنده (موجودی لحظه‌ای، قیمت لحظه‌ای)	✗ کش نکنید؛ در <code><Suspense></code> بپیچید
داده حساس/محرمانه کاربر	✗ هرگز اشتراکی کش نکنید

✗ اشتباهات رایج / ✓ روش درست

اشتباه ✗	درست ✓
کش‌کردن داده شخصی با <code>use cache</code> اشتراکی	<code>use cache: private</code> یا اصلاً کش نکردن
خواندن <code>cookies()</code> داخل scope کش‌شده	بیرون بخوانید و به‌عنوان آرگومان پاس دهید
اتکای صرف به <code>cacheLife</code> برای داده تغییرپذیر	بعد از تغییر، <code>revalidateTag</code> بزنید
انتظار کش خودکار مثل نسخه‌های قبل	در v16 کش، تصمیم صریح شماست

Best Practices و نکات طلایی ✓

۱) پیش‌فرض را پویا نگه دارید؛ فقط چیزی را کش کنید که واقعاً کم‌تغییر و عمومی است. ۲) هر داده کش‌شده را هم با `cacheLife` (سقف تازگی) و هم با `cacheTag` (باطل‌سازی رویدادی) پوشش دهید. ۳) قانون طلایی: «داده کاربر را هرگز اشتراکی کش نکن». ۴) برای داده زنده، کش نکنید و `Suspense` را جایگزین کنید. ۵) کلید کش = آرگومان‌ها؛ پس توابع کش‌شده را «خالص» و ورودی‌محور نگه دارید.

🚀 مینی پروژه: صفحه تخصص‌ها با کش ساعتی + دکمه تازه‌سازی مدیر

یک `getSpecialties()` با `cacheLife("hours")` ، `use cache` و `cacheTag("specialties")` بسازید و در صفحه رندر کنید. سپس یک Server Action «افزودن تخصص» بنویسید که بعد از ثبت، `revalidateTag("specialties")` را صدا بزند. نتیجه: صفحه سریع و کش‌شده، اما بلافاصله پس از افزودن، تازه می‌شود. (اتصال Action در فصل بعد).

📌 خلاصه فصل

در v16 پیش‌فرض پویاست؛ کش یک تصمیم صریح با `use cache` است. کلید کش از آرگومان‌ها ساخته می‌شود. عمر را با `cacheLife` و باطل‌سازی رویدادی را با `cacheTag + revalidateTag` کنترل کنید. داده شخصی → `private`؛ داده زنده → کش نکنید و Suspense بزنید.

📝 تمرین

۱) یک تابع کش‌شده با آرگومان بسازید و ببینید ورودی‌های مختلف، ورودی‌های کش جدا می‌سازند. ۲) یک صفحه را با `cacheLife("minutes")` کش کنید و رفتار تازه‌شدن را مشاهده کنید. ۳) یک `cacheTag` بزنید و با یک Action و `revalidateTag` آن را باطل کنید و لحظه تازه‌شدن را ببینید.

🎤 سؤالات مصاحبه (با پاسخ)

Junior – کش یعنی چه و چرا مهم است؟ ذخیره نتیجه یک کار پرهزینه تا دفعه بعد به‌جای تکرار، همان نتیجه آماده سریع تحویل داده شود؛ باعث سرعت بیشتر و بار کمتر روی سرور/دیتابیس می‌شود.

Mid – چرا در v16 پیش‌فرض پویاست و `use cache` چه کار می‌کند؟ چون کش خودکار نسخه‌های قبل گیج‌کننده بود؛ حالا رفتار قابل‌پیش‌بینی است و فقط چیزی که با `use cache` علامت بزنید کش می‌شود. کلید کش از آرگومان‌ها ساخته می‌شود.

Senior – چه چیزی را باید/نباید کش کرد و Trade-off را چطور مدیریت می‌کنید؟ داده عمومی کم‌تغییر را کش می‌کنم (با `cacheLife` برای سقف تازگی و `cacheTag` برای باطل‌سازی رویدادی)، داده شخصی را `private` و داده زنده حساس را اصلاً کش نمی‌کنم و به Suspense می‌سپارم. Trade-off همان تعادل «تازگی در برابر سرعت» است که با ترکیب عمر + برچسب کنترلش می‌کنم.

10 Server Actions و تغییر داده

راهی برای «تغییر داده» بدون ساختن دستی API: توابعی که مستقیماً روی سرور اجرا می‌شوند و می‌توان آن‌ها را به فرم وصل کرد — امن، ساده و با پشتیبانی از کارکرد بدون JavaScript.

۱ | مفهوم چیست؟

Server Action یک تابع `async` است که با دایرکتیو `"use server"` علامت خورده و فقط روی سرور اجرا می‌شود، اما می‌توانید آن را مستقیماً از دل یک کامپوننت کلاینت (مثلاً یک فرم) صدا بزنید. Next.js ارتباط شبکه‌ای بین کلاینت و سرور را پشت‌صحنه مدیریت می‌کند.

۲ | چرا ساخته شد و چه مشکلی را حل می‌کند؟

پیش از این، برای هر «تغییر داده» (ثبت فرم، حذف، ویرایش) باید یک Route Handler می‌ساختید، در کلاینت `fetch` می‌زدید، حالت `loading/error` را دستی مدیریت می‌کردید و بدنه را سریالایز می‌کردید. Server Actions این تشریفات را حذف می‌کند:

- بدون API دستی: تابع سرور را مستقیماً به `<form action={...}>` وصل می‌کنید.
- Progressive Enhancement: فرم حتی پیش از بارگذاری JavaScript هم کار می‌کند.
- یکپارچگی با کش: بعد از تغییر، با `revalidatePath/Tag` داده تازه می‌شود.

۳ | Mental Model + جریان تغییر داده

یک Server Action را مثل یک «فراخوانی مستقیم تابع سرور از کلاینت» تصور کنید. کاربر فرم را می‌فرستد → تابع روی سرور اجرا می‌شود → ورودی اعتبارسنجی و داده تغییر می‌کند → کش باطل و UI تازه می‌شود.



۴ نحوه کار داخلی (Behind the Scenes)

- ◆ Next.js برای هر اکشن یک شناسه امن می‌سازد؛ هنگام ارسال فرم، یک درخواست `POST` به همان اکشن ارسال می‌شود (شما این را نمی‌بینید).
- ◆ بدنه به صورت خودکار سریالایز/دیسریالایز می‌شود؛ خروجی اکشن می‌تواند `UI` را از سرور به روزرسانی کند.
- ◆ یک محافظت داخلی در برابر `CSRF` وجود دارد (بررسی `Origin`)، اما این جایگزین اعتبارسنجی و بررسی دسترسی نیست.

ساخت یک Server Action

عبارت `"use server"` در بالای فایل یا تابع، آن را به اکشن سرور تبدیل می‌کند. اعتبارسنجی ورودی همیشه روی سرور:

```
src/app/contact/actions.ts

"use server"; // 🔒 everything in this file runs on the server

import { z } from "zod";
import { revalidatePath } from "next/cache";

// 🛡️ Validate on the server – never trust the client
const schema = z.object({
  name: z.string().min(2, "نام را کامل وارد کنید"),
  phone: z.string().regex(/^09\d{9}$/, "شماره موبایل معتبر نیست"),
});

export async function submitContact(_prev: unknown, formData: FormData) {
  const parsed = schema.safeParse({
    name: formData.get("name"),
    phone: formData.get("phone"),
  });

  if (!parsed.success) return { ok: false, error: parsed.error.issues[0].message };

  await db.contact.create({ data: parsed.data }); // 💾 persist
  revalidatePath("/contact"); // 🔄 refresh the page data
  return { ok: true };
}
```

اتصال به فرم + مدیریت وضعیت با useState

برای نمایش خطا/موفقیت و وضعیت «در حال ارسال»، در یک Client Component از هوک `useState` استفاده کنید. این الگوی استاندارد فرم‌های Production است:

```
src/components/contact-form.tsx

"use client";
import { useState } from "react";
import { submitContact } from "@app/contact/actions";

export function ContactForm() {
  // 📦 state = last result • formAction = wrapped action • pending = in-flight
  const [state, formAction, pending] = useState(submitContact, null);

  return (
    <form action={formAction} className="space-y-3">
      <input name="name" placeholder="نام" className="border p-2" />
      <input name="phone" placeholder="موبایل" className="border p-2" />
      <button type="submit" disabled={pending}>
        {pending ? "ارسال..." : "در حال ارسال"}
      </button>
      {state?.error && <p className="text-red-600">{state.error}</p>}
      {state?.ok && <p className="text-green-600">✅ با موفقیت ثبت شد</p>}
    </form>
  );
}
```

امنیت Server Actions

هر Server Action یک اندپوینت واقعی و از بیرون قابل فراخوانی است. پس همیشه ورودی را روی سرور اعتبارسنجی کنید و احراز هویت/دسترسی کاربر را داخل اکشن بررسی کنید. محدودیت الی هیچ ارزش امنیتی ندارد.

۵) چه زمانی Server Action؟ / ۶) چه زمانی Route Handler؟

نیاز	ابزار درست
تغییر داده داخلی همین اپ از فرم	Server Action
API عمومی برای اپ موبایل/سرویس ثالث	Route Handler (فصل API)
Webhook، استریم، کنترل کامل HTTP	Route Handler
خواندن ساده برای نمایش	هیچ‌کدام؛ در Server Component مستقیم واکنشی کنید

revalidatePath در برابر revalidateTag

داده یک مسیر مشخص را باطل می‌کند؛ `revalidatePath("/contact")`
 همه کش‌های دارای آن برچسب را – فارغ از صفحه. برای تغییرات گسترده، tag `revalidateTag("posts")` انعطاف بیشتری می‌دهد.

اشتباهات رایج / روش درست

اشتباه	درست
اعتماد به ورودی فرم بدون اعتبارسنجی سرور	اعتبارسنجی با zod داخل اکشن
نیود بررسی دسترسی داخل اکشن	<code>requireUser()</code> / بررسی نقش در ابتدای اکشن
فراموش کردن تازه‌سازی کش بعد از تغییر	<code>revalidatePath</code> / <code>revalidateTag</code>
مدیریت دستی loading با useState	<code>useActionState</code> برای وضعیت و خطا

Best Practices و نکات طلایی

۱) هر اکشن با یک schema اعتبارسنجی و یک بررسی دسترسی شروع شود. ۲) خروجی ساخت‌یافته برگردانید (`{ ok, error }`) تا UI تمیز واکنش دهد. ۳) بعد از هر تغییر، دقیقاً همان چیزی را که کش شده باطل کنید (tag یا path). ۴) برای «حس آبی»، `useOptimistic` را با اکشن ترکیب کنید (فصل ترفندها). ۵) اکشن‌ها را کوچک و تک‌مسئولیت نگه دارید.

🚀 مینی پروژه: فرم ثبت درخواست با اعتبارسنجی و بازخورد

یک فرم «ثبت درخواست مشاوره» بسازید: یک Server Action با schema (نام، موبایل، متن)، ذخیره در دیتابیس، و `revalidatePath`. سپس با `useActionState` پیام خطا/موفقیت و وضعیت «در حال ارسال» را نمایش دهید. در نهایت یک بررسی دسترسی (فقط کاربر وارد شده) به ابتدای اکشن اضافه کنید.

📌 خلاصه فصل

Server Action = تابع سرور با `"use server"` که مستقیم به فرم وصل می‌شود؛ بدون API دستی و با Progressive Enhancement. همیشه ورودی را اعتبارسنجی و دسترسی را بررسی کنید، خروجی ساخت‌یافته برگردانید، و بعد از تغییر کش را با `path/tag` تازه کنید. برای API عمومی به جای اکشن، Route Handler به کار ببرید.

📝 تمرین

۱) یک اکشن «حذف آیتم» با بررسی مالکیت رکورد بنویسید و بعد از حذف، فهرست را با `revalidatePath` تازه کنید. ۲) همان فرم تماس را یکبار بدون (Progressive Enhancement) `useActionState` و یکبار با آن پیاده کنید و تفاوت تجربه را ببینید. ۳) یک اکشن با `revalidateTag` بنویسید و اثرش را روی چند صفحه مختلف که همان tag را دارند مشاهده کنید.

🎤 سؤالات مصاحبه (با پاسخ)

Junior – Server Action چیست؟ تابعی با `"use server"` که روی سرور اجرا می‌شود و می‌توان آن را مستقیم به یک فرم وصل کرد تا داده را تغییر دهد، بدون ساختن API دستی.

Mid – چرا Server Action بهتر از fetch دستی به یک API است و `useActionState` چه می‌کند؟ تشریفات ساختن endpoint، سریالایز بدنه و مدیریت دستی loading/error را حذف می‌کند و Progressive Enhancement می‌دهد. `useActionState` نتیجه اکشن، تابع پیچیده شده و وضعیت pending را در اختیار می‌گذارد تا UI تمیز واکنش دهد.

Senior – امنیت و مرزبندی Server Action در برابر Route Handler را توضیح دهید. هر اکشن یک endpoint واقعی است؛ پس اعتبارسنجی ورودی و بررسی دسترسی داخل اکشن الزامی است (محافظت داخلی CSRF کافی نیست). Server Action برای تغییر داده داخلی از فرم مناسب است؛ برای API عمومی، Webhook، استریم یا کنترل کامل HTTP باید Route Handler به کار برد.

11 Route Handlers (ساخت API)

وقتی به یک اندپوینت واقعی نیاز دارید (مثلاً webhook، خروجی JSON برای موبایل، یا یک API عمومی)، از Route Handler استفاده می‌کنید.

چرا و مدل ذهنی

چرا وجود دارد؟ پیش از این برای یک API نیاز به سرور جداگانه (مثل Express) داشتید؛ Route Handler اجازه می‌دهد API و UI در یک پروژه، یک زبان و یک دیپلوی کنار هم باشند. **مدل ذهنی:** هر `route.ts` یک «دروازه» است – درخواست HTTP وارد می‌شود، شما آن را می‌خوانید و اعتبارسنجی می‌کنید، و یک `Response` استاندارد برمی‌گردانید. توابع صادرشده هم‌نام متد HTTP اند.

این فصل در برابر فصل API

اینجا «مبانی» Route Handler است. برای نسخه Production کامل با MongoDB/Mongoose، CRUD، صفحه‌بندی/جستجو/فیلتر، احراز هویت و دیاگرام جریان، به فصل «API نویسی کامل» مراجعه کنید.

اصول اولیه

یک فایل با نام `route.ts` داخل پوشه `app` بسازید. توابعی که با نام متدهای HTTP اکسپورت می‌کنید (مثل `GET` و `POST`) به آن اندپوینت پاسخ می‌دهند. توجه: در یک پوشه نمی‌توانید هم `page.tsx` و هم `route.ts` داشته باشید.

```
src/app/api/doctors/route.ts

import { NextResponse } from "next/server";

// GET /api/doctors
export async function GET() {
  const doctors = await db.doctor.findMany();
  return NextResponse.json(doctors); // JSON response
}

// POST /api/doctors
export async function POST(request: Request) {
  const body = await request.json();
```

```
const created = await db.doctor.create({ data: body });
return NextResponse.json(created, { status: 201 });
}
```

پارامترهای پویا و async

برای اندپوینت‌های پویا مثل `/api/doctors/[id]`، پارامتر دوم تابع شامل `params` است که – مثل صفحات – در نسخه ۱۶ یک Promise است و باید `await` شود:

```
src/app/api/doctors/[id]/route.ts

import { NextResponse } from "next/server";

export async function GET(
  _req: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params; // ⌚ async params in Next.js 16
  const doctor = await db.doctor.findUnique({ where: { id } });

  if (!doctor) {
    return NextResponse.json({ error: "یافت نشد" }, { status: 404 });
  }
  return NextResponse.json(doctor);
}
```

خواندن کوئری، کوکی و هدر

پارامترهای query از روی `request` در دسترس‌اند. برای کوکی و هدر هم توابع `cookies()` و `headers()` را – که حالا async هستند – `await` کنید:

```
reading request data

import { cookies, headers } from "next/headers";

export async function GET(request: Request) {
  // 🔍 query string: /api/search?q=cardio
  const q = new URL(request.url).searchParams.get("q");

  const cookieStore = await cookies(); // ⌚ async in v16
  const token = cookieStore.get("token")?.value;
}
```

```
const headerList = await headers(); // ⏰ async in v16
const lang = headerList.get("accept-language");

return Response.json({ q, hasToken: Boolean(token), lang });
}
```

چه زمانی Route Handler و چه زمانی Server Action؟

اگر هدف، تغییر داده از داخل خود اپلیکیشن است (فرم‌ها، دکمه‌ها)، **Server Action** ساده‌تر و امن‌تر است. Route Handler را برای موارد «بیرونی» نگه دارید: webhooks، API عمومی، یا مصرف‌کننده‌های غیر-React مثل اپ موبایل.

❌ اشتباهات رایج / ✅ روش درست

اشتباه ❌	درست ✅
خواندن <code>cookies()</code> / <code>params</code> بدون <code>await</code>	در v16 همه <code>async</code> ؛ همیشه <code>await</code>
اعتماد به بدنه درخواست بدون اعتبارسنجی	ورودی را با <code>zod</code> اعتبارسنجی کنید
همیشه بازگرداندن ۲۰۰	کد وضعیت معنادار: ۲۰۱، ۴۰۰، ۴۰۴، ۴۲۲
ساخت API برای تغییر داده داخلی فرم	از Server Action استفاده کنید

✅ Best Practices و نکات طلایی

(۱) هر handler را با اعتبارسنجی ورودی و — در صورت حساس بودن — بررسی دسترسی شروع کنید. (۲) خروجی و کد وضعیت را استاندارد نگه دارید. (۳) خطای داخلی را لو ندهید؛ پیام عمومی به کاربر، لاگ کامل روی سرور. (۴) برای مصرف عمومی، نسخه‌بندی (`... /api/v1/`) را از ابتدا در نظر بگیرید.

🚀 مینی‌پروژه: اندپوینت جستجوی پزشک

یک `GET /api/doctors/search?q=...` بسازید که پارامتر `q` را از query بخواند، با `zod` اعتبارسنجی کند، نتیجه را برگرداند و برای ورودی خالی کد `400` بدهد. سپس یک `GET /api/doctors/[id]` با پاسخ `404` برای شناسه ناموجود اضافه کنید.

خلاصه فصل

فایل `route.ts` یک اندپوینت می‌سازد و توابع هم‌نام متد HTTP به آن پاسخ می‌دهند. در v16 همه `params` / `cookies()` / `headers()` برای تغییر داده داخلی از Server Action، و برای API بیرونی/عمومی از Route Handler استفاده کنید. عمق (MongoDB/CRUD/Auth) Production در فصل API آمده است.

تمرین

۱) یک `POST /api/contact` بنویسید که بدنه را با `zod` اعتبارسنجی کند و برای ورودی نامعتبر 422 برگرداند. ۲) کوکی و `accept-language` را در یک handler بخوانید و در پاسخ JSON برگردانید. ۳) همان قابلیت «ثبت تماس» را یک‌بار با Route Handler و یک‌بار با Server Action پیاده کنید و تفاوت را بسنجید.

سؤالات مصاحبه (با پاسخ)

Junior – Route Handler کجا تعریف می‌شود و چطور به متدها پاسخ می‌دهد؟ در فایل `route.ts` داخل `app`؛ با اکسپورت توابعی هم‌نام متد HTTP (مثل `GET`، `POST`) در یک پوشه نمی‌توان هم `page.tsx` و هم `route.ts` داشت.

Mid – در v16 هنگام خواندن `params` / `cookies()` / `headers()` چه نکته‌ای هست؟ همه `async` شده‌اند و باید `await` شوند؛ فراموش کردنش رایج‌ترین خطای مهاجرت است.

Senior – Route Handler در برابر Server Action را مرزبندی کنید. Server Action برای تغییر داده داخلی از دل فرم‌ها (ساده‌تر، با محافظت داخلی CSRF)؛ Route Handler وقتی یک قرارداد HTTP عمومی می‌خواهید: webhook، مصرف‌کننده غیر-React، استریم، یا کنترل کامل هدر/وضعیت. در هر دو حالت، اعتبارسنجی و بررسی دسترسی روی سرور الزامی است.

12 proxy.ts؛ جانشین Middleware

کدی که پیش از رسیدن درخواست به مسیرها اجرا می‌شود – برای احراز هویت، ریدایرکت و بازنویسی. در نسخه ۱۶ نام آن از middleware.ts به proxy.ts تغییر کرد.

مفهوم و جای فایل

فایل `proxy.ts` را در ریشه پروژه (کنار `src` یا داخل آن، هم‌سطح `app`) بگذارید. تابع پیش‌فرض آن روی هر درخواست منطبق با `matcher` اجرا می‌شود و می‌تواند درخواست را عبور دهد، ریدایرکت کند، یا بازنویسی کند.

```
src/proxy.ts

import { NextResponse, type NextRequest } from "next/server";

export function proxy(request: NextRequest) {
  const token = request.cookies.get("token")?.value;
  const isAuthPage = request.nextUrl.pathname.startsWith("/login");

  // 🛡️ Protect the dashboard: redirect guests to /login
  if (!token && request.nextUrl.pathname.startsWith("/dashboard")) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  // 🔄 Logged-in users shouldn't see the login page
  if (token && isAuthPage) {
    return NextResponse.redirect(new URL("/dashboard", request.url));
  }

  return NextResponse.next(); // ✅ allow the request through
}

// 🎯 Only run on these paths (keeps it fast)
export const config = {
  matcher: ["/dashboard/:path*", "/login"],
};
```

هشدار امنیتی مهم نسخه ۱۶

در سال ۱۴۰۵ چند آسیب‌پذیری مهم در لایه middleware/proxy وصله شد (دور زدن احراز هویت از طریق مسیرهای prefetch و تزریق پارامتر). دو نتیجه عملی: **اول**، همیشه روی آخرین نسخه پایدار بمانید. **دوم**، `proxy.ts` را تنها خط دفاعی احراز هویت قرار ندهید؛ بررسی دسترسی را در لایه داده (/ Server Action Route Handler) هم تکرار کنید. proxy برای ریدایرکت سریع خوب است، نه به‌عنوان قفل نهایی.

کاربردهای رایج

- ♦ **محافظت از مسیرها:** ریدایرکت کاربر مهمان به صفحه ورود (نمونه بالا).
- ♦ **افزودن هدر:** مثلاً هدرهای امنیتی یا شناسه درخواست.
- ♦ **بازنویسی (rewrite):** نمایش محتوای یک مسیر زیر آدرسی دیگر بدون تغییر URL کاربر.
- ♦ **بین‌المللی‌سازی:** تشخیص زبان و هدایت به نسخه زبانی مناسب.

مهاجرت از نسخه قبل

اگر از نسخه ۱۵ می‌آیید: فایل `middleware.ts` را به `proxy.ts` و نام تابع `middleware` را به `proxy` تغییر دهید. codemod رسمی این کار را خودکار انجام می‌دهد: `npx @next/codemod@latest upgrade`.

13 استایل دهی با Tailwind v4

Next.js چند روش استایل دهی را پشتیبانی می‌کند؛ در پروژه‌های مدرن، Tailwind CSS انتخاب پیش فرض و سریع‌ترین مسیر است.

گزینه‌های موجود

روش	مناسب برای
Tailwind CSS	سرعت بالا، طراحی یکدست، استاندارد پروژه‌های جدید
CSS Modules	استایل scope شده و ایزوله برای یک کامپوننت خاص
Global CSS	متغیرها، ریست‌ها و استایل‌های سراسری

Tailwind v4 و تنظیمات

اگر هنگام ساخت پروژه فلگ `-- tailwind` را زده باشید، همه‌چیز آماده است. در نسخه ۴، پیکربندی ساده‌تر شده و کافی است در فایل CSS سراسری، Tailwind را وارد کنید:

```
src/app/globals.css

@import "tailwindcss";

/* 🎨 Design tokens as CSS variables (v4 style) */
:root {
  --brand: oklch(0.62 0.19 255);
}

/* 🌐 RTL-friendly base for a Persian UI */
body {
  font-family: var(--font-vazir), system-ui, sans-serif;
}
```

سپس مستقیماً در JSX از کلاس‌های یوتیلیتی استفاده می‌کنید. در رابط راست‌چین، از خاصیت‌های منطقی Tailwind (مثل `ms-` و `me-` به جای `ml-` و `mr-`) استفاده کنید تا فاصله‌ها در RTL درست بیفتند:

RTL-aware utility classes

```
export function Badge({ label }: { label: string }) {  
  // ✅ ms-/me- (logical) adapt to RTL automatically  
  return (  
    <span className="inline-flex items-center rounded-full bg-blue-50 px-3 py-1 text-blue-  
700 ms-2">  
      {label}  
    </span>  
  );  
}
```

shadcn/ui

برای کامپوننت‌های آماده و باکیفیت (دکمه، دیالوگ، فرم و ...) که روی Tailwind ساخته می‌شوند و کاملاً قابل سفارشی‌سازی‌اند، [shadcn/ui](#) انتخاب عالی است. کامپوننت‌ها مستقیماً داخل پروژه شما کپی می‌شوند، پس کنترل کامل دارید و وابستگی سنگین اضافه نمی‌کنید.

CSS Modules برای استایل ایزوله

اگر بخواهید استایل یک کامپوننت کاملاً محدود به خودش بماند، از فایل `*.module.css` استفاده کنید؛ نام کلاس‌ها به صورت خودکار یکتا می‌شوند و با کامپوننت‌های دیگر تداخل نمی‌کنند:

card.module.css + usage

```
/* card.module.css */  
.card { border-radius: 12px; padding: 16px; }  
  
/* component */  
import styles from "./card.module.css";  
export function Card() {  
  return <div className={styles.card}>محتوا</div>; // 🔒 scoped, collision-free  
}
```

14 بهینه‌سازی: تصویر، فونت، لینک و سئو

Next.js ابزارهای داخلی قدرتمندی برای کارایی دارد. استفاده از آن‌ها تفاوت بزرگی در Core Web Vitals و تجربه کاربر می‌سازد.

تصویر با next/image

کامپوننت `Image` به صورت خودکار تصویر را در فرمت‌های مدرن، اندازه مناسب و به صورت lazy بارگذاری می‌کند و از پرش چیدمان (layout shift) جلوگیری می‌کند. همیشه `width` و `height` (یا `fill`) را مشخص کنید:

```
src/components/doctor-avatar.tsx

import Image from "next/image";

export function DoctorAvatar({ src, name }: { src: string; name: string }) {
  return (
    <Image
      src={src}
      alt={name}
      width={96}
      height={96}
      className="rounded-full"
      // 🖼️ Use this only for the above-the-fold hero image (LCP)
      priority={false}
    />
  );
}
```

تغییر نسخه ۱۶

چند prop قدیمی `next/image` حذف شدند (مثل `lazyBoundary`، `onLoadingComplete` و `lazyRoot`). اگر از نسخه قدیمی مهاجرت می‌کنید، با codemod رسمی آن‌ها را به روز کنید. برای دامنه‌های خارجی هم باید آدرس را در `next.config.ts` در بخش `images.remotePatterns` مجاز کنید.

فونت فارسی با next/font

فونت‌ها را در زمان build بهینه و خودمیزبان (self-host) می‌کند؛ نتیجه‌اش حذف درخواست به سرور خارجی و جلوگیری از پرش متن است. برای رابط فارسی، فونت Vazirmatn را این‌گونه بارگذاری کنید:

```
src/app/layout.tsx

import { Vazirmatn } from "next/font/google";

// Self-hosted, optimized Persian font; expose it as a CSS variable
const vazir = Vazirmatn({
  subsets: ["arabic"],
  variable: "--font-vazir",
  display: "swap",
});

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="fa" dir="rtl" className={vazir.variable}>
      <body className="font-sans">{children}</body>
    </html>
  );
}
```

لینک، prefetch و سئو

کامپوننت `Link` مسیرهای داخل دید کاربر را به‌صورت خودکار prefetch می‌کند تا ناوبری تقریباً آبی شود. برای سئو هم دو فایل ویژه بسازید: `sitemap.ts` و `robots.ts` که Next.js خروجی استانداردشان را تولید می‌کند:

```
src/app/sitemap.ts

import type { MetadataRoute } from "next";

// Generates /sitemap.xml automatically
export default function sitemap(): MetadataRoute.Sitemap {
  return [
    { url: "https://doctorrezro.ir", priority: 1 },
    { url: "https://doctorrezro.ir/doctors", priority: 0.8 },
  ];
}
```

15 محیط، TypeScript و استقرار

آخرین قطعات قبل از Production: مدیریت متغیرهای محیطی، تایپ‌های امن، و انتشار اپلیکیشن.

متغیرهای محیطی

مقادیر محرمانه (مثل رشته اتصال دیتابیس) را در فایل `.env.local` بگذارید. یک قانون حیاتی: متغیرها به صورت پیش‌فرض فقط روی سرور در دسترس‌اند. اگر متغیری باید در مرورگر هم خوانده شود، نامش باید با `NEXT_PUBLIC_` شروع شود:

```
.env.local

# 🗝 Server-only (never sent to the browser)
DATABASE_URL="mongodb+srv://..."
JWT_SECRET="super-secret-value"

# 🌐 Exposed to the browser (bundled into client JS)
NEXT_PUBLIC_BASE_URL="https://doctorrezro.ir"
```

دو اشتباه پرتکرار و پرهزینه

۱) هرگز یک کلید محرمانه را با `NEXT_PUBLIC_` اکسپورت نکنید؛ مستقیماً داخل جاوااسکریپت کلاینت قرار می‌گیرد و برای همه قابل مشاهده است. ۲) مراقب باشید `NEXT_PUBLIC_BASE_URL` در محیط Production به اشتباه روی `localhost` نماند؛ این یکی از رایج‌ترین دلایل خراب شدن لینک‌های sitemap و متادیتای سئو در دیپلوی است.

تایپ‌های امن در TypeScript

چون `params` و `searchParams` در نسخه ۱۶ Promise هستند، آن‌ها را همیشه به درستی تایپ کنید. این کار خطاهای زمان اجرا را به خطاهای زمان کامپایل تبدیل می‌کند:

```
typing page props

type PageProps = {
  params: Promise<{ id: string }>;
  searchParams: Promise<{ [key: string]: string | string[] | undefined }>;
};
```

```
export default async function Page({ params, searchParams }: PageProps) {
  const { id } = await params;
  const { sort } = await searchParams;
  return <DoctorView id={id} sort={sort} />;
}
```

عادت حرفه‌ای

قبل از هر تحویل یا دیپلوی، یکبار `npx tsc --noEmit` بزنید تا مطمئن شوید هیچ خطای تایپی وجود ندارد. این کار سریع است و جلوی بسیاری از باگ‌های Production را می‌گیرد.

استقرار (Deployment)

اول یک بیلد Production بگیرید و خروجی را به‌صورت محلی تست کنید:

Terminal

```
npm run build # 🛠 produce the optimized output
npm run start # ✅ run that build locally to verify before deploy
```

سپس یکی از دو مسیر را انتخاب کنید:

- ♦ **Vercel** (سازنده Next.js): ساده‌ترین راه؛ مخزن Git را وصل می‌کنید و هر push به‌صورت خودکار بیلد و دیپلوی می‌شود. فراموش نکنید متغیرهای محیطی (به‌خصوص مقادیر Production مثل `NEXT_PUBLIC_BASE_URL`) را در داشبورد Vercel هم تنظیم کنید، نه فقط در `.env.local`.
- ♦ **میزبانی شخصی**: روی هر سروری که **Node.js نسخه ۲۰.۹ یا بالاتر** دارد، خروجی build را با `npm run start` اجرا کنید. معمولاً پشت یک reverse proxy (مثل Nginx) و با یک Process Manager (مثل PM2) آن را پایدار نگه می‌دارند.

نکته زیرساخت

اگر سرویس‌های جانبی شما (مثل سرویس پیامک یا درگاه) فقط از داخل ایران در دسترس‌اند، اما اپ روی زیرساخت خارج از ایران میزبانی می‌شود، ممکن است آن درخواست‌ها مسدود شوند. راه‌حل رایج، یک معماری relay است: یک سرویس کوچک داخل ایران که Next.js درخواست را به آن می‌سپارد و آن با relay با سرویس داخلی صحبت می‌کند.

خروجی standalone و Docker

برای میزبانی شخصی بهینه، حالت `standalone` را روشن کنید. Next.js یک سرور کاملاً مستقل و کوچک در `.next/standalone` می‌سازد که فقط وابستگی‌های لازم را دارد؛ ایده‌آل برای ساختن یک ایميج Docker سبک:

```
next.config.ts

const nextConfig = {
  output: "standalone", // 📦 self-contained server in .next/standalone
};
export default nextConfig;
```

سپس یک Dockerfile چندمرحله‌ای می‌نویسید که فقط خروجی standalone را به ایميج نهایی منتقل می‌کند:

```
Dockerfile

# 🛠️ build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# 🚀 tiny runtime image (only the standalone output)
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
EXPOSE 3000
CMD ["node", "server.js"] # 🟢 standalone entrypoint
```

خروجی کاملاً استاتیک (Static Export)

اگر سایت شما هیچ قابلیت سمت‌سرور زنده‌ای لازم ندارد (مثل یک سایت معرفی یا بلاگ کاملاً استاتیک)، می‌توانید آن را به HTML/CSS/JS خالص تبدیل کنید و روی هر هاست استاتیک (حتی GitHub Pages یا یک باکت) بگذارید:

next.config.ts

```
const nextConfig = {  
  output: "export", // 📦 generates a fully static site into the "out/" folder  
  images: { unoptimized: true }, // static export can't run the image optimizer  
};  
export default nextConfig;
```

محدودیت‌های خروجی استاتیک

در حالت `export` هیچ سرور زنده‌ای وجود ندارد، پس این‌ها کار نمی‌کنند: **Server Actions**، **Route**، **Handler**های پویا، **ISR/revalidate**، **proxy.ts** و رندر پویا. مسیرهای پویا هم حتماً باید با `generateStaticParams` از پیش ساخته شوند. اگر به هرکدام از این‌ها نیاز دارید، به جای `static export` از حالت `standalone` (سرور Node) استفاده کنید.

16 پروژه عملی: مینی بلاگ کامل

حالا همه مفاهیم را در یک نمونه واقعی و کوچک کنار هم می‌گذاریم: فهرست مقالات (کش‌شده)، صفحه تک‌مقاله (پویا)، و افزودن مقاله با Server Action.

۱ | ساختار

structure

```
src/app/blog/
├─ page.tsx           # 📄 cached list of posts
├─ [slug]/page.tsx    # 📄 single post (dynamic, async params)
├─ new/page.tsx       # 📝 create-post form
└─ actions.ts         # 🛠️ server action + data helpers
```

۲ | داده و اکشن سمت سرور

src/app/blog/actions.ts

```
"use server";
import { z } from "zod";
import { cacheTag, revalidateTag } from "next/cache";
import { redirect } from "next/navigation";

// 📄 Cached list – tagged so we can invalidate it after a create
export async function getPosts() {
  "use cache";
  cacheTag("posts");
  return db.post.findMany({ orderBy: { createdAt: "desc" } });
}

export async function getPost(slug: string) {
  return db.post.findUnique({ where: { slug } });
}

const schema = z.object({
  title: z.string().min(3, "عنوان کوتاه است"),
  body: z.string().min(10, "متن کوتاه است"),
});
```

```

export async function createPost(_prev: unknown, formData: FormData) {
  const parsed = schema.safeParse({
    title: formData.get("title"),
    body: formData.get("body"),
  });
  if (!parsed.success) return { error: parsed.error.issues[0].message };

  const slug = parsed.data.title.trim().replace(/\s+/g, "-");
  await db.post.create({ data: { ...parsed.data, slug } });

  revalidateTag("posts"); // 🔄 list cache is now stale → refresh it
  redirect(`/blog/${slug}`); // 🚀 go straight to the new post
}

```

۳ | فهرست مقالات (کش شده)

```

src/app/blog/page.tsx

import Link from "next/link";
import { getPosts } from "../actions";

export default async function BlogPage() {
  const posts = await getPosts(); // 🧠 served from cache until invalidated
  return (
    <main className="mx-auto max-w-2xl p-6">
      <div className="flex items-center justify-between">
        <h1 className="text-2xl font-bold">مقالات</h1>
        <Link href="/blog/new" className="text-blue-600">مقاله جدید</Link>
      </div>
      <ul className="mt-4 space-y-2">
        {posts.map((p) => (
          <li key={p.slug}>
            <Link href={`/${p.slug}`} className="hover:underline">
              {p.title}
            </Link>
          </li>
        ))}
      </ul>
    </main>
  );
}

```

src/app/blog/[slug]/page.tsx

```
import { notFound } from "next/navigation";
import { getPost } from "../actions";

export default async function PostPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params; // ⌚ async params
  const post = await getPost(slug);
  if (!post) notFound(); // 🚫 renders not-found.tsx

  return (
    <article className="mx-auto max-w-2xl p-6">
      <h1 className="text-2xl font-bold">{post.title}</h1>
      <p className="mt-4 leading-8">{post.body}</p>
    </article>
  );
}
```

src/app/blog/new/page.tsx

```
"use client";
import { useActionState } from "react";
import { createPost } from "../actions";

export default function NewPostPage() {
  const [state, action, pending] = useActionState(createPost, null);
  return (
    <form action={action} className="mx-auto max-w-2xl space-y-3 p-6">
      <input name="title" placeholder="عنوان" className="w-full border p-2" />
      <textarea name="body" placeholder="متن" className="w-full border p-2 rows={6} />
      <button disabled={pending} className="rounded bg-blue-600 px-4 py-2 text-white">
        {pending ? "انتشار" : "در حال ذخیره..."}
      </button>
      {state?.error && <p className="text-red-600">{state.error}</p>}
    </form>
  );
}
```

چه چیزی اینجا اتفاق افتاد؟

فهرست مقالات با `use cache` کش و با `cacheTag("posts")` برچسب‌گذاری شد؛ صفحه تک‌مقاله از `async params` و `notFound()` استفاده کرد؛ و فرم ساخت با `Server Action`، اعتبارسنجی `zod`، باطل‌کردن کش با `revalidateTag` و ریدایرکت کامل شد. این الگوی کوچک، ستون فقرات اغلب اپلیکیشن‌های واقعی است.

سؤالات مصاحبه (با پاسخ)

Mid – صفحه‌ای LCP ضعیفی دارد چون تصویر hero دیر می‌آید؛ چه می‌کنید؟ تصویر LCP را با `next/image` و پرچم `priority` بارگذاری می‌کنم تا preload شود، `sizes` درست می‌دهم تا نسخه مناسب دستگاه بیاید، فرمت مدرن (AVIF/WebP) خودکار) را می‌گیرم و با `width/height` یا `fill` فضای آن را رزرو می‌کنم تا CLS ایجاد نشود.

Senior – تیم می‌خواهد یک فونت فارسی سفارشی اضافه کند اما نمره CLS افت کرده؛ راهکار؟ فونت را با `self-host next/font/local` می‌کنم (حذف درخواست به سرور ثالث و FOUT)، `display:swap` با یک فونت fallback هم‌متریک تعریف می‌کنم تا هنگام سوییچ، جهش چیدمان کم شود، و فقط وزن‌های بحرانی را preload می‌کنم. رزرو فضای متن هم از پرش جلوگیری می‌کند.

Senior – تعادل بین کیفیت رسانه و کارایی را در یک صفحه پرتصویر چطور مدیریت می‌کنید؟ فقط المان LCP را `priority` می‌کنم و بقیه را lazy می‌گذارم، ابعاد/aspect-ratio را صریح می‌دهم تا CLS صفر بماند، برای هر breakpoint اندازه درست سرو می‌کنم و یک «بودجه تصویر» تعیین می‌کنم؛ سنجش نهایی با داده میدانی Core Web Vitals، نه فقط لوکال.

Senior – بعد از افزودن `next/image`، بیلد روی پلتفرم Self-Host به‌خاطر بهینه‌سازی تصویر کند/پرهزینه شده؛ دیباگ؟ بهینه‌سازی زمان‌اجرا به یک بهینه‌ساز نیاز دارد؛ در Self-Host یا باید `sharp` نصب باشد یا یک loader خارجی (CDN تصویر) تعریف شود. بررسی می‌کنم که آیا تصاویر بی‌جهت on-the-fly بهینه می‌شوند؛ برای دارایی‌های ثابت از loader سفارشی یا CDN استفاده می‌کنم و کش تصویر را روی یک لایه پایدار می‌گذارم تا هر درخواست دوباره پردازش نشود.

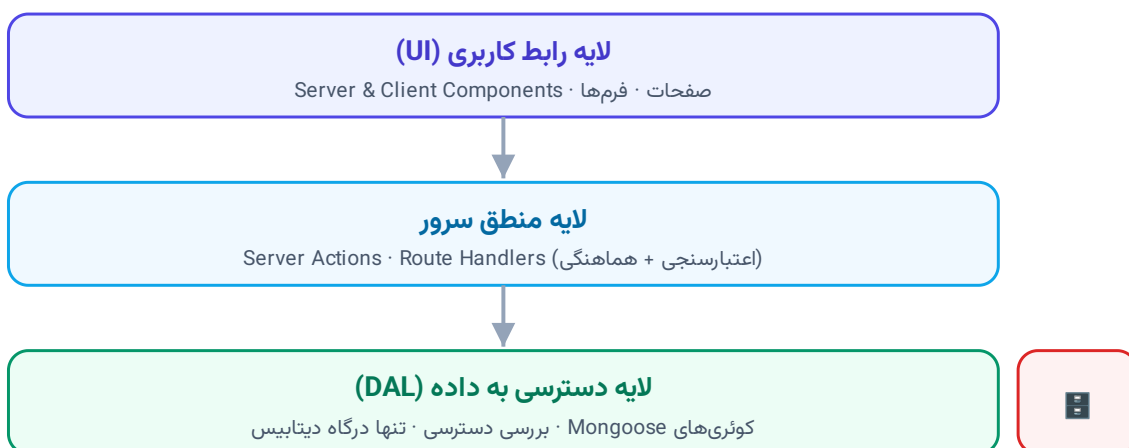
Staff – می‌خواهید در یک اپلیکیشن بزرگ با چند تیم، «رگرسیون کارایی رسانه» هرگز وارد Production نشود؛ چه سیستمی طراحی می‌کنید؟ یک «بودجه کارایی» را در CI اجباری می‌کنم: Lighthouse-CI روی هر PR با آستانه LCP/CLS، لینت سفارشی که استفاده از `<img>` خام یا نبود `sizes` را رد می‌کند، یک کامپوننت تصویر پوششی داخلی (wrapper) که `aspect-ratio / sizes / priority` را استاندارد و اجباری می‌کند، و مانیتورینگ میدانی (RUM) با هشدار روی رگرسیون. این‌طور کیفیت به‌جای اتکا به انضباط فردی، در سطح پلتفرم تضمین می‌شود.

17 معماری پروژه، ساختار پوشه و Clean Code

تفاوت یک پروژه آماتور و حرفه‌ای در کد نیست، در معماری است. این فصل نشان می‌دهد چطور پروژه را طوری بسازید که با رشد تیم و امکانات، فرو نریزد.

معماری لایه‌ای: هر چیز سر جای خودش

یک اپ Next.js حرفه‌ای سه لایه روشن دارد. رابط کاربری هرگز مستقیم با دیتابیس حرف نمی‌زند؛ همه چیز از میان یک لایه دسترسی به داده (DAL) عبور می‌کند که تنها جایی است که کوئری‌ها و بررسی دسترسی در آن جمع می‌شوند:



```

src/
├─ app/                                # 🚦 routing only (thin: pages compose features)
│  └─ (dashboard)/appointments/page.tsx
│     └─ api/appointments/route.ts
├─ features/                          # 🧩 the heart of the app – one folder per feature
│  └─ appointments/
│     │   └─ components/              # UI for this feature
│     │   └─ actions.ts               # server actions
│     │   └─ queries.ts               # DAL: db reads for this feature
│     │   └─ schema.ts                # zod validation schemas
│     │   └─ types.ts
│     └─ auth/
│        └─ actions.ts session.ts guard.ts
├─ lib/                               # 🔧 shared, feature-agnostic utils
│  └─ db.ts env.ts utils.ts
├─ components/ui/                     # 🎨 design-system primitives (shadcn/ui)
└─ proxy.ts

```

قاعده طلایی

پوشه `app/` را «نازک» نگه دارید: صفحه‌ها فقط featureها را کنار هم می‌چینند. منطق واقعی (کوئری، اعتبارسنجی، اکشن) داخل `features/` زندگی می‌کند. این‌طور هر قابلیت را می‌توان مستقل فهمید، تست کرد و حتی حذف کرد.

لایه دسترسی به داده (DAL) در عمل

به جای پخش کردن کوئری‌ها در همه‌جا، هر خواندن داده را در یک تابع متمرکز کنید که **خودش** دسترسی کاربر را بررسی می‌کند. این هم امنیت را متمرکز می‌کند، هم کد را تمیز:

```
src/features/appointments/queries.ts
```

```

import "server-only";
import { cache } from "react";
import { dbConnect } from "@lib/mongoose";
import { AppointmentModel } from "@models/appointment";
import { requireUser } from "@features/auth/guard";

// 🛡️ One gate to the data: auth is checked HERE, not in the UI
export const getMyAppointments = cache(async () => {

```

```
const user = await requireUser(); // throws/redirects if not logged in
await dbConnect();
return AppointmentModel
  .find({ patientId: user.id }) // 🛡️ scoped to the current user
  .sort({ startsAt: -1 }) // newest first
  .lean(); // ⚡ plain objects, faster for read-only
});
```

الگوهای مقیاس‌پذیری (Scaling)

چالش رشد	الگوی حرفه‌ای
منطق تکراری بین اکشن‌ها	استخراج به توابع DAL و helperهای مشترک در <code>lib/</code>
اعتبارسنجی پراکنده	یک <code>schema.ts</code> با zod per-feature؛ هم روی سرور، هم برای تایپ‌ها
کش ناهماهنگ بین چند نمونه	Cache Handler مشترک (Redis) در self-host
وابستگی مستقیم UI به دیتابیس	عبور اجباری از DAL؛ UI هرگز <code>db</code> را مستقیم import نکند
پکیج‌های سنگین در باندل	<code>server-only</code> برای کد سرور + بارگذاری تنبل کلاینت

Next.js در Clean Code

- نام‌گذاری روشن: `getMyAppointments` بهتر از `fetchData`؛ `createAppointmentAction` بهتر از `submit`.
- هر فایل یک مسئولیت: کوئری در `queries.ts`، اکشن در `actions.ts`، اعتبارسنجی در `schema.ts`.
- Server-first:** `"use client"` فقط روی کوچک‌ترین جزء تعاملی؛ منطق و داده روی سرور.
- خطاها را بلعیده نکنید: یا مدیریت کنید یا بگذارید بالا بروند تا مرز خطا بگیردشان.
- مرزها را تایپ کنید: خروجی توابع DAL و ورودی اکشن‌ها را با تایپ zod مشخص کنید.

تمرین

یک پروژه خالی بسازید و ساختار `features/` بالا را برای دو قابلیت فرضی («نوبت‌ها» و «پروفایل») پیاده کنید؛ برای هرکدام فقط فایل‌های خالی `queries.ts`، `actions.ts`، `components/` و `schema.ts` را بسازید. همین تمرین ساده، معماری را در ذهنتان جا می‌اندازد.

سؤالات مصاحبه (با پاسخ)

Junior – چرا ساختار **feature-based** بهتر از «بر اساس نوع فایل» است؟ چون کد هر قابلیت کنار هم جمع می‌شود (colocation)؛ پیدا کردن، فهمیدن، تست و حتی حذف یک قابلیت ساده می‌شود و پروژه با رشد، شلوغ و درهم نمی‌شود.

Mid – DAL چیست و چه مشکلی را حل می‌کند؟ لایه دسترسی به داده؛ تنها درگاهی که با دیتابیس حرف می‌زند و بررسی دسترسی را همان‌جا (نه در UI) متمرکز می‌کند. مزیت: امنیت یکجا، حذف کوئری‌های پراکنده، و کد تمیز و قابل‌تست.

Senior – «قاعده وابستگی» را توضیح دهید و بگویید **Service** و **Repository** کجای یک اپ **Next.js** می‌نشینند. وابستگی‌ها یک‌طرفه و رو به پایین‌اند (DB → DAL/Repository → Service → UI)؛ لایه‌های پایین هرگز به بالا وابسته نمی‌شوند و UI هرگز مستقیم **db** را `import` نمی‌کند. لایه **Service** (Server Actions/Route Handlers) منطق کسب‌وکار را هماهنگ می‌کند و **Repository** (DAL) فقط دسترسی به داده و کنترل دسترسی را انجام می‌دهد. این جداسازی، تست‌پذیری و تعویض ابزار داده (**Prisma ↔ Mongoose**) را ممکن می‌کند.

18 استراتژی‌های رندر: SSR، SSG، ISR و PPR

مهم‌ترین سؤال برای هر صفحه: HTML آن «کجا» و «کی» ساخته شود؟ پاسخ درست، تفاوت بین یک سایت کند و یک سایت برق‌آسا است. این فصل چهار استراتژی و نحوه کنترلشان را عمیق و دیگرام‌محور آموزش می‌دهد.

۱) مفهوم چیست؟

«رندر» یعنی تولید HTML نهایی صفحه. دو محور تصمیم داریم: **کجا** ساخته شود (سرور یا کلاینت) و **کی** ساخته شود (زمان بیلد، زمان هر درخواست، یا ترکیبی). هر ترکیب یک «استراتژی رندر» است.

۲) چرا چند استراتژی؟ چه مشکلی را حل می‌کنند؟

هیچ حالتی در همه معیارها برنده نیست؛ یک «مثلث تعادل» بین سه چیز وجود دارد و باید آگاهانه انتخاب کنید:

♦ **سرعت:** صفحه از پیش ساخته (استاتیک) فوری از CDN سرو می‌شود.

♦ **تازگی داده:** رندر در زمان درخواست همیشه آخرین داده را دارد، اما کندتر است.

♦ **هزینه سرور:** رندر هر درخواست بار می‌آورد؛ کش کردن بار را کم می‌کند.

استراتژی‌های مختلف به شما اجازه می‌دهند برای هر صفحه، تعادل درست را بین این سه انتخاب کنید.

۳) Mental Model – چه زمانی HTML ساخته می‌شود؟

هر حالت را روی یک محور زمانی تصور کنید: از «زمان بیلد» (یک‌بار، برای همه) تا «زمان هر درخواست» (تازه، برای همان کاربر).



۴ نحوه کار داخلی (Behind the Scenes)

- ◆ **SSG**: در زمان بیلد HTML ساخته و روی CDN گذاشته می‌شود؛ هر بازدید همان فایل آماده را می‌گیرد.
- ◆ **SSR**: در هر درخواست، سرور صفحه را از نو رندر می‌کند.
- ◆ **ISR**: نسخه کش‌شده سرو می‌شود؛ پس از انقضای عمر، نسخه تازه در پس‌زمینه ساخته و جایگزین می‌شود (بازدیدکننده فعلی معطل نمی‌ماند).
- ◆ **PPR**: پوسته استاتیک فوری سرو می‌شود و «حفره‌های پویا» به تدریج استریم و پر می‌شوند.

چهار حالت رندر – نمای کلی

حالت	زمان رندر	مناسب برای
SSG (استاتیک)	زمان build	محتوای ثابت: صفحات بازاریابی، مستندات
SSR (سروری)	هر درخواست	محتوای شخصی یا کاملاً زنده
ISR	build + بازتولید زمان‌بندی‌شده	محتوای نیمه‌ثابت: بلاگ، فروشگاه
PPR	پوسته استاتیک + استریم پویا	صفحات ترکیبی (بخش ثابت + بخش پویا)

مدل نسخه ۱۶

در نسخه ۱۶ همه‌چیز به‌صورت پیش‌فرض **پویا (SSR)** است. برای استاتیک‌سازی از `use cache` و `generateStaticParams` استفاده می‌کنید؛ و با روشن‌بودن Next.js، `cacheComponents` یک پوسته استاتیک می‌سازد و بخش‌های پویا را استریم می‌کند – همان مدل PPR.

استاتیک با (SSG) generateStaticParams

برای پیش‌ساخت صفحات یک مسیر پویا در زمان build:

```
src/app/blog/[slug]/page.tsx
```

```
// 🛠️ Build-time static generation for known dynamic paths
export async function generateStaticParams() {
  const posts = await getAllSlugs();
  return posts.map((slug) => ({ slug })); // each becomes a static page
}
```

```
// ❌ control behavior for slugs NOT returned above:
export const dynamicParams = true; // true: render on-demand | false: 404
```

بازتولید تدریجی (ISR) با cacheLife

صفحه را کش کنید و به آن «عمر» بدهید؛ پس از انقضا، نسخه تازه در پس‌زمینه ساخته می‌شود و کاربر بعدی نسخه به‌روز را می‌بیند:

```
src/app/products/page.tsx

import { cacheLife } from "next/cache";

export default async function Products() {
  "use cache";
  cacheLife("hours"); // 🕒 ISR-style: regenerate after ~1h, serve stale meanwhile
  const items = await getProducts();
  return <Grid items={items} />;
}
```

پیش‌رندر جزئی (PPR)

قدرت اصلی نسخه ۱۶: یک صفحه پوسته استاتیک فوری دارد و بخش‌های پویا به‌تدریج «داخل حفره‌ها» استریم می‌شوند. کاربر بلافاصله چیزی می‌بیند و بخش‌های شخصی/زنده کمی بعد پر می‌شوند:



```
src/app/product/[id]/page.tsx

import { Suspense } from "react";

// 🧱 Static shell: cached, served instantly
async function ProductInfo({ id }: { id: string }) {
```

```

"use cache";
const p = await getProduct(id);
return <header><h1>{p.name}</h1><p>{p.price}</p></header>;
}

// 🌀 Dynamic part: streams in, personalized per request
async function Recommendations({ id }: { id: string }) {
  const recs = await getRecsFor(id);
  return <Carousel items={recs} />;
}

export default async function Page({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  return (
    <main>
      <ProductInfo id={id} />
      <Suspense fallback=<Skeleton />>
        <Recommendations id={id} /> { /* 🌀 the dynamic hole */ }
      </Suspense>
    </main>
  );
}

```

کنترل دستی پویا/استاتیک

گاهی می‌خواهید صراحتاً اعلام کنید یک صفحه پویاست (چون به داده هر درخواست وابسته است). تابع `connection()` رندر را تا زمان درخواست به تعویق می‌اندازد:

```

● ● ● force dynamic rendering

import { connection } from "next/server";

export default async function Page() {
  await connection(); // 🌀 opt into per-request dynamic rendering
  const now = new Date().toISOString(); // safe: evaluated at request time
  return <p>زمان سرور: {now}</p>;
}

```

۵) چه زمانی؟ / ۶) چه زمانی نه؟ (قاعده انتخاب)

وضعیت داده	استراتژی
ثابت و یکسان برای همه	(SSG) <code>generateStaticParams</code>
نیمه ثابت، تحمل چند دقیقه/ساعت کهنگی	(ISR) <code>use cache</code> + <code>cacheLife</code>
شخصی یا کاملاً زنده	SSR پویا (+ <code>Suspense</code>)
ترکیبی: پوسته ثابت + بخش زنده	PPR (کش پوسته + <code>Suspense</code> برای بخش پویا)

دام رایج

یک بخش پویای کوچک (مثل تاریخ فعلی یا داده کاربر) کل صفحه را پویا می‌کند و مزیت استاتیک را از بین می‌برد. راه حل: آن بخش را جدا و در `Suspense` بپیچید تا بقیه صفحه استاتیک بماند (همان روح PPR).

اشتباهات رایج / ✓ روش درست

اشتباه ✗	درست ✓
پویا کردن کل صفحه به خاطر یک ویجت کوچک	آن ویجت را در <code>Suspense</code> ایزوله کنید (PPR)
کش کردن داده شخصی به صورت استاتیک	بخش شخصی را پویا و استریم کنید
انتظار رفتار نسخه قدیم (استاتیک پیش فرض)	در v16 پیش فرض پویاست؛ استاتیک را صریح انتخاب کنید
ISR با عمر خیلی کوتاه روی صفحه پرتراфик	عمر متناسب با نرخ تغییر واقعی داده

Best Practices و نکات طلایی ✓

۱) پیش فرض را پویا بگذارید و فقط چیزی را که کم تغییر است کش کنید. ۲) هر صفحه را با این سؤال بسنجید: «کدام بخش‌ها می‌توانند برای همه یکسان باشند؟» آن‌ها را پوسته استاتیک کنید. ۳) بخش‌های زنده را در `Suspense` بپیچید تا استاتیک را آلوده نکنند. ۴) عمر ISR را با نرخ تغییر واقعی داده هماهنگ کنید، نه دلبخواه. ۵) برای تست سرعت واقعی، روی `next build && next start` بسنجید، نه `dev`.

🚀 مینی پروژه: صفحه محصول با PPR

یک `app/product/[id]/page.tsx` بسازید که: (۱) اطلاعات پایه محصول (نام/قیمت) را با `use cache` به صورت پوستر استاتیک نشان دهد، و (۲) بخش «پیشنهادهای شخصی» را که به هر کاربر بستگی دارد در `<Suspense>` با اسکلت بارگذاری استریم کند. سپس با `next build` ببینید کدام بخش استاتیک و کدام پویا علامت خورده است.

📌 خلاصه فصل

رندر یعنی «کجا و کی HTML ساخته شود». SSG در بیلد (سریع ترین، برای محتوای ثابت)، SSR در هر درخواست (تازه ترین، برای محتوای زنده/شخصی)، ISR کش با عمر (تعادل)، و PPR پوستر استاتیک + استریم پویا (بهترین هر دو). در v16 پیش فرض پویاست؛ با `use cache` استاتیک می سازید و بخش های زنده را در `Suspense` استریم می کنید.

📝 تمرین

(۱) یک صفحه بلاگ با `generateStaticParams` بسازید و `dynamicParams` را یکبار `true` و یکبار `false` کنید و تفاوت را ببینید. (۲) یک صفحه را با `cacheLife("minutes")` کش کنید و رفتار «سرو کهنه + بازتولید پس زمینه» را مشاهده کنید. (۳) یک ویجت «تاریخ فعلی» را به یک صفحه استاتیک اضافه کنید و ببینید چطور کل صفحه پویا می شود؛ سپس با `Suspense` ایزوله اش کنید.

🎤 سؤالات مصاحبه (با پاسخ)

Junior – تفاوت SSG و SSR چیست؟ SSG صفحه را در زمان build یکبار می سازد و همان را به همه سرو می کند (سریع، برای محتوای ثابت)؛ SSR صفحه را در هر درخواست از نو می سازد (تازه، برای محتوای زنده/شخصی).

ISR – Mid چطور کار می کند و چه Trade-off ی دارد؟ نسخه کش شده فوری سرو می شود و پس از انقضای عمر، نسخه تازه در پس زمینه ساخته و جایگزین می شود. Trade-off: کاربر ممکن است تا لحظه بازتولید، نسخه کمی «کهنه» ببیند؛ در ازای سرعت بالا و بار کم سرور.

PPR – Senior چه مشکلی را حل می کند و چطور در v16 پیاده می شود؟ مشکل «یک بخش پویا کل صفحه را پویا می کند» را حل می کند: پوستر استاتیک فوری از CDN می آید و حفره های پویا استریم می شوند. در v16 با روشن کردن `cacheComponents`، کش کردن پوستر با `use cache` و پیچیدن بخش های پویا در `Suspense` پیاده می شود – بهترین تعادل سرعت اولیه و تازگی داده.

19 API نویسی کامل با Route Handlers و MongoDB

این فصل شما را از «API چیست؟» تا یک API واقعی CRUD با MongoDB و Mongoose می‌رساند: اعتبارسنجی، صفحه‌بندی، جستجو، فیلتر، مرتب‌سازی، احراز هویت و امنیت – همراه با مثال کامل، اشتباهات رایج و سؤالات مصاحبه.

توضیح ساده: HTTP، API و REST چیست؟

API یک «قرارداد» است: کلاینت درخواستی می‌فرستد و سرور پاسخی برمی‌گرداند. ارتباط از طریق HTTP انجام می‌شود؛ هر درخواست یک متد دارد که «قصد» را می‌رساند. REST یک سبک رایج طراحی API است: هر چیز یک «منبع» است (مثلاً پزشکان) و متدهای HTTP روی آن عمل می‌کنند.

متد	قصد	مثال	کد موفقیت
GET	خواندن	فهرست/یک پزشک	۲۰۰
POST	ساختن	افزودن پزشک	۲۰۱
PUT / PATCH	به‌روزرسانی	ویرایش پزشک	۲۰۰
DELETE	حذف	حذف پزشک	۲۰۰

Mental Model: جریان یک درخواست API

هر درخواست از یک مسیر روشن عبور می‌کند. Route Handler «دروازه» است: ورودی را اعتبارسنجی می‌کند، با دیتابیس حرف می‌زند و پاسخ استاندارد برمی‌گرداند.



Route Handler چیست و چرا وجود دارد؟

در App Router، هر فایل `route.ts` یک اندپوینت API می‌سازد. چرا؟ پیش از این برای یک API نیاز به سرور جداگانه (مثل Express) داشتید؛ حالا API و UI در یک پروژه، با یک زبان و یک دیپلوی کنار هم‌اند. توابع صادرشده هم‌نام متدهای HTTP اند (`GET` ، `POST` ، ...). اسکلت استاندارد یک اندپوینت – اعتبارسنجی، مدیریت خطا، کد وضعیت درست:

```
src/app/api/doctors/route.ts – skeleton

import { NextRequest, NextResponse } from "next/server";
import { z } from "zod";

const createSchema = z.object({
  name: z.string().min(2),
  specialty: z.string().min(2),
});

export async function POST(req: NextRequest) {
  let body: unknown;
  try { body = await req.json(); }
  catch { return NextResponse.json({ error: "نامعتبر JSON بدنه" }, { status: 400 }); }

  const parsed = createSchema.safeParse(body); // 🛡 validate input
  if (!parsed.success) {
    return NextResponse.json(
      { error: parsed.error.issues[0].message },
      { status: 422 }, // 422 = validation failed
    );
  }
  // ...persist (see MongoDB section) then:
  return NextResponse.json({ data: parsed.data }, { status: 201 }); // 201 Created
}
```

اتصال به MongoDB با Mongoose (الگوی درست Next.js)

مشکل: در محیط سرورلس و Hot Reload، اگر هر بار اتصال جدید بسازید، خیلی زود به سقف اتصالات می‌خورید. **راه‌حل:** اتصال را در یک متغیر سراسری «کش» کنید تا در طول عمر پروسه فقط یک بار برقرار شود:

src/lib/mongoose.ts

```
import mongoose from "mongoose";

const MONGODB_URI = process.env.MONGODB_URI!;
if (!MONGODB_URI) throw new Error("MONGODB_URI تعریف نشده است");

// 🌐 cache the connection across hot reloads / serverless invocations
type Cache = { conn: typeof mongoose | null; promise: Promise<typeof mongoose> | null };
const g = globalThis as unknown as { _mongoose?: Cache };
const cached: Cache = g._mongoose ?? (g._mongoose = { conn: null, promise: null });

export async function dbConnect() {
  if (cached.conn) return cached.conn; // ♻️ reuse existing
  cached.promise ??= mongoose.connect(MONGODB_URI, { bufferCommands: false });
  cached.conn = await cached.promise; // 🖱️ connect once
  return cached.conn;
}
```

.env.local

```
MONGODB_URI="mongodb+srv://user:pass@cluster.mongodb.net/doctor-booking"
```

طراحی Schema و Model

Schema شکل و قواعد سند را تعریف می‌کند؛ Model درگاه کار با کالکشن است. نکته کلیدی در Next.js: مدل را با `models.X || model(...)` بسازید تا در Hot Reload دوباره تعریف نشود (خطای «Cannot overwrite model»:)

src/models/doctor.ts

```
import { Schema, model, models, InferSchemaType } from "mongoose";

const doctorSchema = new Schema(
  {
    name: { type: String, required: true, trim: true },
    specialty: { type: String, required: true, index: true }, // 🔍 indexed for filtering
    city: { type: String, required: true },
    rating: { type: Number, default: 0, min: 0, max: 5 },
    isActive: { type: Boolean, default: true },
  },
  {
```

```

{ timestamps: true }, // ⌚ auto createdAt / updatedAt
);

export type Doctor = InferSchemaType<typeof doctorSchema>; // 🧠 type from schema
// ⚠️ reuse the compiled model on hot reload, don't redefine it
export const DoctorModel = models.Doctor || model("Doctor", doctorSchema);

```

Mongoose کامل با CRUD |

حالا عملیات چهارگانه را می‌سازیم. فهرست و ساخت در `route.ts`، و تک‌رکورد در `[id]/route.ts`. توجه کنید که همیشه ورودی اعتبارسنجی و پاسخ استاندارد می‌شود:

```

src/app/api/doctors/route.ts - list + create

import { NextRequest, NextResponse } from "next/server";
import { z } from "zod";
import { dbConnect } from "@lib/mongoose";
import { DoctorModel } from "@models/doctor";

const createSchema = z.object({
  name: z.string().min(2),
  specialty: z.string().min(2),
  city: z.string().min(2),
});

// 🛠️ CREATE
export async function POST(req: NextRequest) {
  const parsed = createSchema.safeParse(await req.json().catch(() => null));
  if (!parsed.success)
    return NextResponse.json({ error: parsed.error.issues[0].message }, { status: 422 });

  await dbConnect(); // 🖱️ ensure connection
  const doctor = await DoctorModel.create(parsed.data);
  return NextResponse.json({ data: doctor }, { status: 201 });
}

// 📖 READ (list) - pagination/search/filter/sort added in the next section
export async function GET() {
  await dbConnect();
  const doctors = await DoctorModel.find({ isActive: true }).lean();
  return NextResponse.json({ data: doctors });
}

```

```

import { NextRequest, NextResponse } from "next/server";
import { isValidObjectId } from "mongoose";
import { dbConnect } from "@lib/mongoose";
import { DoctorModel } from "@models/doctor";

type Params = { params: Promise<{ id: string }> }; // ⌚ params is async in v16

export async function GET(_req: NextRequest, { params }: Params) {
  const { id } = await params;
  if (!isValidObjectId(id)) return NextResponse.json({ error: "شناسه نامعتبر" }, { status: 400 });

  await dbConnect();
  const doctor = await DoctorModel.findById(id).lean();
  if (!doctor) return NextResponse.json({ error: "پزشک یافت نشد" }, { status: 404 });
  return NextResponse.json({ data: doctor });
}

export async function PATCH(req: NextRequest, { params }: Params) {
  const { id } = await params;
  await dbConnect();
  // { new: true } ⇒ return the UPDATED document; runValidators ⇒ enforce schema rules
  const updated = await DoctorModel.findByIdAndUpdate(id, await req.json(), {
    new: true, runValidators: true,
  }).lean();
  if (!updated) return NextResponse.json({ error: "پزشک یافت نشد" }, { status: 404 });
  return NextResponse.json({ data: updated });
}

export async function DELETE(_req: NextRequest, { params }: Params) {
  const { id } = await params;
  await dbConnect();
  const deleted = await DoctorModel.findByIdAndDelete(id).lean();
  if (!deleted) return NextResponse.json({ error: "پزشک یافت نشد" }, { status: 404 });
  return NextResponse.json({ data: { id } });
}

```

صفحه‌بندی، جستجو، فیلتر و مرتب‌سازی

یک API واقعی هرگز «همه رکوردها» را برنمی‌گرداند. این پارامترها را از query string بخوانید و به کوئری Mongoose تبدیل کنید. این الگو در فهرست پزشکان، محصولات، مقالات — همه‌جا یکسان است:

```

export async function GET(req: NextRequest) {
  await dbConnect();
  const sp = req.nextUrl.searchParams;

  const page = Math.max(1, Number(sp.get("page") ?? "1"));
  const limit = Math.min(50, Number(sp.get("limit") ?? "10")); // 🚗 cap page size
  const q = sp.get("q")?.trim();
  const city = sp.get("city");
  const sort = sp.get("sort") ?? "-createdAt"; // "-" prefix = descending

  // 🔧 build the filter dynamically from provided params
  const filter: Record<string, unknown> = { isActive: true };
  if (q) filter.name = { $regex: q, $options: "i" }; // 🔍 case-insensitive search
  if (city) filter.city = city; // 🏠 exact filter

  const [items, total] = await Promise.all([ // 🚀 run both in parallel
    DoctorModel.find(filter).sort(sort).skip((page - 1) * limit).limit(limit).lean(),
    DoctorModel.countDocuments(filter),
  ]);

  return NextResponse.json({
    data: items,
    meta: { page, limit, total, pages: Math.ceil(total / limit) },
  });
}

```

چرا این‌طور؟

سقف‌گذاری روی `limit` از حمله «یک صفحه با ۱ میلیون رکورد» جلوگیری می‌کند. اجرای موازی `find` و `countDocuments` با `Promise.all` تأخیر را نصف می‌کند. و `lean()` اسناد ساده‌ی JS (نه document سنگین Mongoose) برمی‌گرداند که برای فقط‌خواندن، سریع‌تر است.

احراز هویت و کنترل دسترسی در API

اندپوینت‌هایی که داده می‌سازند یا تغییر می‌دهند باید محافظت شوند. session را از کوکی (یا توکن Bearer را از هدر) بخوانید و پیش از هر کاری بررسی کنید؛ نقش‌ها را هم همین‌جا کنترل کنید:

src/app/api/doctors/route.ts – protected POST

```
import { cookies } from "next/headers";
import { verifySession } from "@/features/auth/session";

export async function POST(req: NextRequest) {
  // 🛡 authentication: who is this?
  const token = (await cookies()).get("session")?.value;
  const user = token ? await verifySession(token) : null;
  if (!user) return NextResponse.json({ error: "ابتدا وارد شوید" }, { status: 401 });

  // ⛔ authorization: are they allowed?
  if (user.role !== "ADMIN")
    return NextResponse.json({ error: "دسترسی ندارید" }, { status: 403 });

  const parsed = createSchema.safeParse(await req.json().catch(() => null));
  if (!parsed.success)
    return NextResponse.json({ error: parsed.error.issues[0].message }, { status: 422 });

  await dbConnect();
  const doctor = await DoctorModel.create(parsed.data);
  return NextResponse.json({ data: doctor }, { status: 201 });
}
```

۴۰۱ در برابر ۴۰۳

401 Unauthorized یعنی «نمی‌دانم تو کیستی» (وارد نشده‌ای). **403 Forbidden** یعنی «می‌دانم کیستی، اما اجازه نداری». برای اندپوینت‌های حساس مثل ورود، **rateLimit** (فصل امنیت) را هم اضافه کنید.

قابلیت‌های پیشرفته API

گاهی به جای JSON ساده، به پاسخ زنده یا دریافت رویداد از سرویس دیگر نیاز دارید. این ابزارها را در جعبه‌ابزار داشته باشید:

۱) **پاسخ استریم**: برای پاسخ‌های بزرگ یا تدریجی (مثل خروجی هوش مصنوعی) داده را تکه‌تکه بفرستید:

src/app/api/stream/route.ts

```
export async function GET() {
  const encoder = new TextEncoder();
  const stream = new ReadableStream({
    async start(controller) {
```

```

    for (const chunk of ["دنیا", " ", "سلام"]) {
      controller.enqueue(encoder.encode(chunk)); // 🌊 send piece by piece
      await new Promise((r) => setTimeout(r, 300));
    }
    controller.close();
  },
});
return new Response(stream, { headers: { "Content-Type": "text/plain; charset=utf-8" } });
}

```

۲) **Server-Sent Events** برای push زنده سرور → مرورگر (اعلان‌ها، نوار پیشرفت)؛ سبک‌تر از WebSocket:

```

src/app/api/notifications/route.ts

export async function GET() {
  const encoder = new TextEncoder();
  const stream = new ReadableStream({
    start(controller) {
      const send = (d: object) =>
        controller.enqueue(encoder.encode(`data: ${JSON.stringify(d)}\n\n`));
      send({ msg: "متصل شدید" });
      setInterval(() => send({ time: Date.now() }), 1000); // 🔔 push every 1s
    },
  });
  return new Response(stream, {
    headers: { "Content-Type": "text/event-stream", "Cache-Control": "no-cache" },
  });
}

```

۳) **Webhook با تأیید امضا**: هنگام دریافت رویداد از درگاه پرداخت/گیت‌هاب، حتماً امضا را روی بدنه خام تأیید کنید:

```

src/app/api/webhook/route.ts

import crypto from "crypto";

export async function POST(req: Request) {
  const raw = await req.text(); // 🔑 verify against the RAW body
  const signature = req.headers.get("x-signature") ?? "";
  const expected = crypto
    .createHmac("sha256", process.env.WEBHOOK_SECRET!)
    .update(raw).digest("hex");
}

```

```
const a = Buffer.from(signature), b = Buffer.from(expected);
// 🛡️ length check first, then constant-time compare (avoids timing attacks)
if (a.length !== b.length || !crypto.timingSafeEqual(a, b))
  return new Response("Invalid signature", { status: 401 });

await handleEvent(JSON.parse(raw));
return Response.json({ received: true });
}
```

۴) CORS برای API های عمومی که از دامنه دیگر فراخوانی می‌شوند، و نسخه‌بندی برای تغییر بدون شکستن کلاینت‌ها:

```
CORS + versioned structure

const cors = {
  "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Methods": "GET,POST,OPTIONS",
  "Access-Control-Allow-Headers": "Content-Type, Authorization",
};

export async function OPTIONS() { return new Response(null, { status: 204, headers: cors }); }

// 📁 versioned routes: app/api/v1/doctors/route.ts → /api/v1/doctors
//                       app/api/v1/doctors/[id]/route.ts → /api/v1/doctors/:id
```

❌ اشتباهات رایج / ✅ روش درست

اشتباه ❌	درست ✅
اعتماد به ورودی کلاینت بدون اعتبارسنجی	هر بدنه/کوئری را با zod اعتبارسنجی کنید
همیشه 200 برگرداندن	کد وضعیت معنادار: ۲۰۱، ۴۰۰، ۴۰۱، ۴۰۳، ۴۰۴، ۴۲۲
ساخت اتصال جدید در هر درخواست	الگوی dbConnect کش‌شده
بازگرداندن همه رکوردها	صفحه‌بندی با سقف limit
تعریف دوباره Model در Hot Reload	models.X model(...)
لو دادن جزئیات خطای داخلی	پیام عمومی به کاربر، لاگ کامل روی سرور

چه زمانی Route Handler، چه زمانی Server Action؟

هر دو روی سرور اجرا می‌شوند، اما هدف فرق دارد: **Server Action** برای تغییر داده داخلی همین اپ از دل فرم‌ها (بدون ساختن API دستی). **Route Handler** وقتی یک *API عمومی/قراردادی* می‌خواهید: مصرف توسط اپ موبایل، سرویس ثالث، webhook، یا نیاز به کنترل کامل HTTP (هدر، استریم، CORS).

🚀 مینی‌پروژه: API واقعی CRUD پزشکان

یک API کامل بسازید: `GET /api/doctors` با صفحه‌بندی + جستجو (`?q=`) + فیلتر شهر (`?city=`) + مرتب‌سازی؛ `POST` محافظت‌شده با نقش ADMIN؛ و `GET/PATCH/DELETE /api/doctors/[id]`. از الگوی `dbConnect`، مدل با `guard`، و اعتبارسنجی `zod` استفاده کنید. سپس با یک ابزار مثل Thunder Client هر پنج مسیر را تست کنید و مطمئن شوید کدهای وضعیت درست‌اند.

🎤 سؤالات مصاحبه

Junior: تفاوت `GET` و `POST` چیست؟ کد وضعیت ۴۰۴ و ۲۰۱ یعنی چه؟ `Route Handler` کجا تعریف می‌شود؟

Mid: چرا در `Next.js` اتصال `Mongoose` را کش می‌کنیم؟ چرا مدل را با `models.X || model()` می‌سازیم؟ صفحه‌بندی و جستجو را چطور از `query string` پیاده می‌کنید؟

Senior: `Route Handler` در برابر `Server Action` — چه زمانی کدام؟ چطور یک API را در برابر ورودی مخرب، نشت خطا و بار زیاد مقاوم می‌کنید (`lean()`، `rate limit`، `status codes`، `validation`، سقف `limit`)؟ نسخه‌بندی API را چطور مدیریت می‌کنید؟

20 خطاهای Hydration – مرجع کامل

یکی از گیج‌کننده‌ترین خطاهای SSR. این فصل کامل توضیح می‌دهد Hydration چیست، پشت‌صحنه چه می‌گذرد، چرا mismatch رخ می‌دهد، و برای هر علت چه راه‌حل دقیقی وجود دارد.

Hydration یعنی چه؟

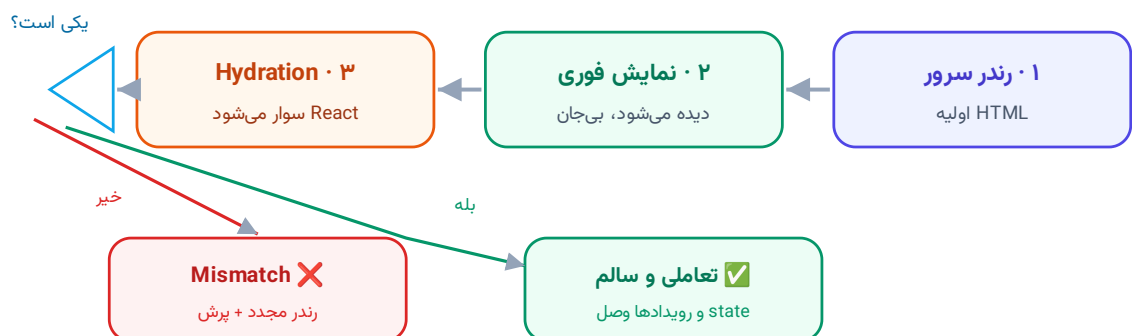
Next.js هر صفحه را دو بار رندر می‌کند: یک بار روی سرور (HTML اولیه که برای سرعت و سئو ارسال می‌شود) و یک بار در مرورگر، جایی که React آن HTML را «آبرسانی» (hydrate) می‌کند تا تعاملی شود. اگر آنچه React در کلاینت می‌سازد با HTML سرور یکسان نباشد، خطای Hydration رخ می‌دهد.

چرا خطرناک است؟

نشانه‌ها: فلش سفید، پرش چیدمان، یا رندر مجدد کل صفحه. در Production اغلب بی‌صدا اتفاق می‌افتد اما به **Core Web Vitals** و تجربه کاربر آسیب می‌زند. پیام‌های رایج: «Text content does not match server-rendered HTML» و «Hydration failed because the server rendered HTML didn't match the client».

Mental Model – مدل ذهنی

سرور یک «اسکلت HTML بی‌جان» می‌فرستد که فوری دیده می‌شود اما هنوز تعاملی نیست. سپس React همان درخت را در مرورگر دوباره می‌سازد و روی HTML موجود «سوار» می‌شود (رویدادها و state را وصل می‌کند). شرط موفقیت این سوارشدن: **درخت کلاینت باید موبه‌مو با HTML سرور یکی باشد.** کوچک‌ترین تفاوت = mismatch.



Behind the Scenes – پشت صحنه چه می‌گذرد؟

- ♦ سرور درخت React را به HTML رشته‌ای تبدیل و ارسال می‌کند؛ مرورگر آن را فوری نقاشی می‌کند (هنوز JS اجرا نشده).
- ♦ سپس باندل JS می‌رسد و React همان درخت را در حافظه می‌سازد و گره‌به‌گره با DOM موجود مقایسه می‌کند (به جای ساختن دوباره، فقط رویدادها را «می‌چسبانند»).
- ♦ اگر گرهی در کلاینت با HTML سرور فرق داشته باشد، React نمی‌تواند امن سوار شود؛ آن زیرشاخه را دور می‌ریزد و از نو رندر می‌کند (منشأ پرش/فلش) و در dev هشدار می‌دهد.

اصل طلایی

هر مقداری که بین «زمان رندر سرور» و «زمان رندر کلاینت» تغییر کند، mismatch می‌سازد. راه حل اساسی: سرور همیشه یک خط پایه قطعی و ثابت رندر کند، و محتوای واقعی پس از hydration در کلاینت اعمال شود.

علت ۱: زمان و مقادیر تصادفی

در سرور و کلاینت مقدار متفاوت می‌دهند: `Math.random()` و `new Date()` ، `Date.now()`

the problem

```
"use client";
// ❌ Server time ≠ client time → always mismatches
export function Clock() {
  return <p>{new Date().toLocaleTimeString()}</p>;
}
```

the fix: render after hydration

```
"use client";
import { useState, useEffect } from "react";

export function Clock() {
  const [time, setTime] = useState<string | null>(null);
  // 🕒 useEffect runs only on the client, after hydration
  useEffect(() => {
    const tick = () => setTime(new Date().toLocaleTimeString());
    tick();
    const id = setInterval(tick, 1000);
  }, []);
}
```

```

    return () => clearInterval(id);
  }, []);
  return <p>{time ?? "-"}</p>; // 🏠 stable baseline "-" matches the server
}

```

علت ۲: API های فقط-مرورگر

خواندن `localStorage`، `window` یا `navigator` در حین رندر، روی سرور وجود ندارد و mismatch (یا crash) می‌سازد. آن‌ها را به `useEffect` منتقل کنید:

```

safe browser API access

"use client";
import { useState, useEffect } from "react";

export function Greeting() {
  const [name, setName] = useState("");
  useEffect(() => {
    // 🌐 localStorage is only read on the client
    setName(localStorage.getItem("username") ?? "");
  }, []);
  return <p>سلام {name || "مهمان"}</p>;
}

```

علت ۳: شرط بر اساس typeof window

این الگو فریبنده است: روی سرور `window` تعریف نشده و روی کلاینت تعریف شده، پس دو خروجی متفاوت تولید می‌کند. به جایش از الگوی `mounted` استفاده کنید:

```

mounted pattern

"use client";
import { useState, useEffect } from "react";

export function ClientOnly({ children }: { children: React.ReactNode }) {
  const [mounted, setMounted] = useState(false);
  useEffect(() => setMounted(true), []);
  if (!mounted) return null; // 🏠 server + first client render both: null
  return <{children}>; // ✅ real content only after hydration
}

```

| علت ۴: شرط بر اساس اندازه صفحه با JS

روی سرور «صفحه‌ای» وجود ندارد، پس `mismatch` `window.innerWidth` می‌سازد. به جای JS از CSS برای نمایش/پنهان‌سازی استفاده کنید تا HTML در دو طرف یکسان بماند:

```
● ● ● use CSS, not JS, for responsive

// ❌ const isMobile = window.innerWidth < 768; ...
// ✅ identical HTML on server and client; CSS decides visibility
export function Nav() {
  return (
    <
      <MobileNav className="block md:hidden" />
      <DesktopNav className="hidden md:block" />
    </>
  );
}
```

| علت ۵: تودرتویی نامعتبر HTML

قرار دادن عناصر در جای نامعتبر (مثل `<div>` داخل `<p>`، یا `<a>` داخل `<a>`، یا `<button>` داخل `<button>`) باعث می‌شود مرورگر HTML را «اصلاح» کند و با خروجی React نخواند. ساختار را معتبر کنید. این خطا اغلب با پیام «Expected server HTML to contain a matching ...» ظاهر می‌شود.

| علت ۶: افزونه‌های مرورگر و اسکریپت‌های ثالث

افزونه‌هایی مثل مدیریت رمز یا مترجم، و اسکریپت‌های تبلیغاتی/چت، ممکن است پیش از hydration گره‌های DOM اضافه کنند. برای تأیید، در پنجره ناشناس (بدون افزونه) تست کنید. اگر علت افزونه باشد، در کد شما اشکالی نیست؛ راه‌حل پذیرفته‌شده گذاشتن `suppressHydrationWarning` روی `<body>` است. برای اسکریپت‌های ثالث هم از `next/script` با استراتژی مناسب استفاده کنید:

```
● ● ● third-party script loading

import Script from "next/script";

// 📄 lazyOnload: load after the page is interactive, away from hydration
<Script src="https://widget.example.com/chat.js" strategy="lazyOnload" />
```

راه فرار: suppressHydrationWarning

برای محتوایی که ناگزیر متفاوت است (مثل یک timestamp)، می‌توانید هشدار را روی همان عنصر خاموش کنید. این فقط یک سطح عمق کار می‌کند و یک «در پشتی» است؛ از آن زیاد استفاده نکنید:

```
intentional, unavoidable mismatch

// ⚠️ Escape hatch – one level deep, use sparingly
<time dateTime="2026-01-01" suppressHydrationWarning>
  {new Date().toLocaleString("fa-IR")}
</time>
```

سرکوب ≠ رفع

`suppressHydrationWarning` فقط هشدار را پنهان می‌کند، نه ریشه مشکل را. React همچنان آن زیرشاخه را در کلاینت دوباره رندر می‌کند و می‌تواند پرش چیدمان بسازد. اولویت همیشه رفع (الگوی `mounted/useEffect`) است، نه سرکوب.

گزینه `ssr: false` برای کامپوننت‌های ناسازگار

اگر یک کتابخانه ذاتاً فقط در مرورگر کار می‌کند (نقشه، نمودار وابسته به window)، آن را با `next/dynamic` و `ssr: false` فقط در کلاینت بارگذاری کنید تا اصلاً روی سرور رندر نشود:

```
client-only component

import dynamic from "next/dynamic";

// ⚠️ never rendered on the server → no hydration mismatch possible
const Map = dynamic(() => import("@components/map"), { ssr: false });
```

چه زمانی رفع، چه زمانی سرکوب؟ (Trade-off)

وضعیت	تصمیم درست
mismatch از کد خودتان (زمان، window، شرط)	✓ رفع کنید (mounted/useEffect/CSS)
محتوای ذاتاً متفاوت و بی‌اهمیت (timestamp نمایشی)	⚠ <code>suppressHydrationWarning</code> روی همان عنصر
افزونه مرورگر گره اضافه می‌کند	⚠ <code>suppressHydrationWarning</code> روی <code><body></code>
کتابخانه فقط-مرورگر	✓ <code>dynamic(..., { ssr: false })</code>

جدول مرجع سریع

علت	راه‌حل
<code>new Date()</code> / <code>Math.random()</code>	انتقال به <code>useEffect</code> + خط پایه ثابت
<code>window</code> / <code>localStorage</code>	<code>useEffect</code> یا الگوی <code>mounted</code>
شرط <code>typeof window</code>	الگوی <code>mounted</code>
ریسپانسیو با JS	استفاده از <code>hidden md:block</code> (CSS)
تودرتویی نامعتبر HTML	اصلاح ساختار عناصر
افزونه مرورگر	<code>suppressHydrationWarning</code> روی <code>body</code>
اسکرپت ثالث	<code>next/script</code> با <code>lazyOnload</code>
کتابخانه فقط-مرورگر	<code>dynamic(..., { ssr: false })</code>

Best Practices و نکات طلایی ✓

۱) قانون طلایی: «سرور یک خط پایه قطعی رندر کند، تفاوت‌ها بعد از ۲. «mount» هر چیز وابسته به مرورگر/زمان → داخل ۳. `useEffect` برای ریسپانسیو، CSS به‌جای `suppressHydrationWarning` (۴. JS. آخرین راه است، نه اولین. ۵) همیشه در `next dev` دیباگ کنید؛ component stack محل دقیق mismatch را می‌گوید. ۶) کامپوننت‌های ذاتاً کلاینتی را با `ssr: false` از رندر سرور کنار بگذارید.

نکته دیباگ

خطاهای Hydration در حالت `next dev` همراه با رد کامپوننت (component stack) نمایش داده می‌شوند که محل دقیق mismatch را نشان می‌دهد. همیشه ابتدا در `dev` دیباگ کنید، نه روی `build production` که خطا بی‌صداست.

مینی‌پروژه: ساعت زنده بدون خطای Hydration 🚀

یک کامپوننت `<LiveClock />` بسازید که زمان فعلی را هر ثانیه نشان دهد، بدون هیچ هشدار Hydration. ابتدا نسخه اشتباه (رندر مستقیم `new Date()`) را بنویسید و خطا را در کنسول `dev` ببینید؛ سپس با الگوی «خط پایه ثابت + `useEffect`» رفعش کنید. در گام بعد یک تم‌سوییچر بسازید که مقدار اولیه را از `localStorage` می‌خواند و با الگوی `mounted` از `mismatch` جلوگیری می‌کند.

خلاصه فصل 📌

Hydration یعنی «سوارشدن React روی HTML سرور». شرط سلامت: درخت کلاینت موبه‌مو با HTML سرور یکی باشد. هر تفاوت (زمان، تصادف، API‌های مرورگر، شرط `typeof window`، ریسپانسیو با HTML، JS، نامعتبر، افزونه‌ها) `mismatch` می‌سازد. راه‌حل اصلی «رفع» است (خط پایه ثابت + `useEffect` / `mounted` / `CSS`)؛ `suppressHydrationWarning` و `ssr:false` ابزارهای خاص و آخرین چاره‌اند.

تمرین 📝

۱) یک کامپوننت بنویسید که `Math.random()` را رندر می‌کند، خطا را ببینید، سپس رفعش کنید. ۲) یک بنر «فقط دسکتاپ» را یک‌بار با `window.innerWidth` (اشتباه) و یک‌بار با `CSS` (درست) پیاده کنید. ۳) یک کتابخانه نقشه فرضی را با `dynamic(..., {ssr:false})` بارگذاری کنید و ببینید چرا دیگر `mismatch` ممکن نیست.

سؤالات مصاحبه (با پاسخ)

Hydration – Junior چیست؟ فرایندی که در آن React روی HTML ازپیش‌رندر شده سرور «سوار» می‌شود و آن را با وصل کردن رویدادها و state تعاملی می‌کند؛ بدون ساختن دوباره کل DOM.

Mid – چرا خطای mismatch رخ می‌دهد و چند علت رایج بگویید. چون درخت رندر شده در کلاینت با HTML سرور فرق دارد. علت‌های رایج: API، `new Date()` / `Math.random()`، های فقط-مرورگر (`window` / `localStorage`)، شرط `typeof window`، ریسپانسیو با JS، و HTML نامعتبر. راه‌حل: خط پایه قطعی روی سرور و اعمال تفاوت‌ها بعد از mount در `useEffect`.

Senior – `suppressHydrationWarning` در برابر `ssr:false` در برابر رفع واقعی؛ کدام و چرا؟ رفع واقعی همیشه اولویت است (ریشه را برطرف می‌کند). `suppressHydrationWarning` فقط هشدار یک عنصر ناگزیر-متفاوت را خاموش می‌کند و mismatch را واقعاً حل نمی‌کند (پرش باقی می‌ماند). `ssr:false` برای کامپوننت‌هایی است که ذاتاً فقط در مرورگر کار می‌کنند؛ آن‌ها را کلاً از رندر سرور کنار می‌گذارد تا mismatch غیرممکن شود – اما هزینه‌اش از دست رفتن HTML اولیه و سئوی آن بخش است.

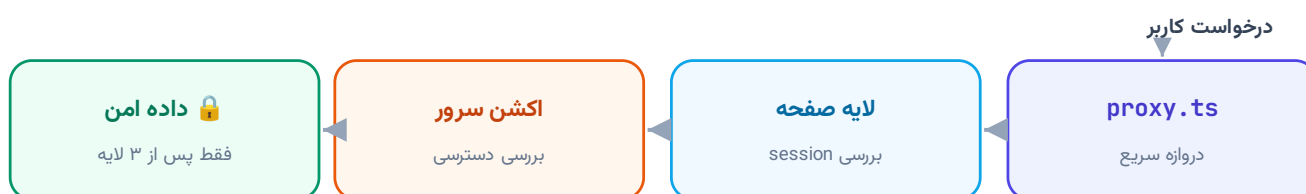
21 احراز هویت و کنترل دسترسی

امنیت کاربر و داده، پایه هر اپلیکیشن جدی است. اینجا الگوهای session، محافظت چندلایه و کنترل دسترسی نقش محور را می‌بینیم.

سه لایه محافظت

امنیت را هرگز به یک نقطه نسپارید. سه لایه را با هم به کار ببرید:

- ♦ **proxy.ts**: ریدایرکت سریع کاربر مهمان از مسیرهای محافظت‌شده (دروازه اول، نه قفل نهایی).
- ♦ **صفحه/Layout**: بررسی session هنگام رندر صفحه محافظت‌شده.
- ♦ **Server Action / Route Handler**: بررسی دسترسی پیش از هر عملیات حساس روی داده.



session با کوکی httpOnly

توکن session را در کوکی `httpOnly` نگه دارید تا JavaScript کلاینت به آن دسترسی نداشته باشد (محافظت در برابر XSS):

src/lib/session.ts

```
import { cookies } from "next/headers";
import { cache } from "react";

// 🛡️ cache(): de-dupe session lookup across a single request
export const getSession = cache(async () => {
  const token = (await cookies()).get("session")?.value;
  if (!token) return null;
  try {
    return await verifyToken(token); // returns { userId, role } or throws
  } catch {
```

```

    return null;
  }
});

```

محافظت از صفحه

در یک Layout یا صفحه محافظت‌شده، session را بخوانید و در نبودش به ورود هدایت کنید:

src/app/(dashboard)/layout.tsx

```

import { redirect } from "next/navigation";
import { getSession } from "@lib/session";

export default async function DashboardLayout({ children }: { children: React.ReactNode }) {
  const session = await getSession();
  if (!session) redirect("/login"); // 🚪 guests out
  return <div>{children}</div>;
}

```

کنترل دسترسی نقش‌محور (RBAC)

برای عملیات مدیریتی، نقش کاربر را بررسی کنید. این بررسی باید در همان **Server Action** باشد، نه فقط در UI:

src/app/admin/actions.ts

```

"use server";
import { getSession } from "@lib/session";

export async function deleteDoctor(id: string) {
  const session = await getSession();
  // 🛡️ authorize INSIDE the action – UI hiding is not security
  if (session?.role !== "admin") {
    throw new Error("دسترسی غیرمجاز");
  }
  await db.doctor.delete({ where: { id } });
}

```

src/app/login/actions.ts

```

"use server";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export async function login(formData: FormData) {
  const user = await authenticate(formData);
  if (!user) return { error: "نام کاربری یا رمز اشتباه است" };

  (await cookies()).set("session", await createToken(user), {
    httpOnly: true, secure: true, sameSite: "lax",
    path: "/", maxAge: 60 * 60 * 24 * 7, // 📅 7 days
  });
  redirect("/dashboard");
}

export async function logout() {
  (await cookies()).delete("session");
  redirect("/login");
}

```

از کتابخانه آماده استفاده کنید

پیاده‌سازی دستی احراز هویت پر از ریزه‌کاری‌های امنیتی است. برای پروژه‌های واقعی، کتابخانه‌های بالغی مثل **Auth.js (NextAuth)** یا **Lucia** بسیاری از این موارد (OAuth، توکن، rotation، CSRF) را امن مدیریت می‌کنند. این فصل مدل ذهنی را می‌سازد تا بدانید زیر کاپوت چه می‌گذرد.

یادآوری امنیتی نسخه ۱۶

به‌خاطر آسیب‌پذیری‌های دور زدن proxy که در نسخه‌های اخیر وصله شد، هرگز تنها لایه امنیتی‌تان `proxy.ts` نباشد. بررسی session را در صفحه و در هر اکشن حساس تکرار کنید و همیشه روی آخرین نسخه پایدار بمانید.

🚀 مینی پروژه: جریان کامل احراز هویت

یک جریان کامل بسازید: (۱) اکشن `login` که پس از احراز، کوکی `session` با پرچم‌های `httpOnly + secure + sameSite` ست کند؛ (۲) یک `(dashboard)/layout.tsx` که مهمان را به `/login` بفرستد؛ (۳) یک اکشن `deleteDoctor` که فقط برای نقش `admin` کار کند؛ و (۴) اکشن `logout`. سپس تلاش کنید اکشن `admin` را با یک کاربر عادی صدا بزنید و ببینید بررسی داخل اکشن جلوی شما را می‌گیرد.

🎤 سؤالات مصاحبه (با پاسخ)

Junior – چرا توکن `session` را در کوکی `httpOnly` می‌گذاریم و نه در `localStorage`؟ چون `httpOnly` از دسترس JavaScript خارج است و در برابر سرقت توکن با XSS محافظت می‌کند؛ `localStorage` با هر اسکریپت مخرب قابل خواندن است.

Mid – «دفاع لایه‌ای» یعنی چه و چرا `proxy.ts` به‌تنهایی کافی نیست؟ یعنی بررسی دسترسی در چند لایه (`proxy`، صفحه، اکشن) تکرار شود تا اگر یکی دور زده شد، بقیه جلوی نشت را بگیرند. `proxy` فقط یک «دروازه سریع» برای ریدایرکت است و آسیب‌پذیری‌های دور زدنش نشان داد نباید تنها خط دفاع باشد؛ بررسی واقعی باید کنار داده (اکشن/DAL) باشد.

Senior – تفاوت `Authentication` و `Authorization` و جای درست هر کدام؟ `Authentication` یعنی «تو کیستی» (اعتبارسنجی `session/توکن`) و `Authorization` یعنی «اجازه داری» (بررسی نقش/مالکیت). هر دو باید سمت سرور و نزدیک به داده اعمال شوند؛ پنهان کردن دکمه در UI امنیت نیست. بررسی نقش را داخل `Server Action/DAL` انجام دهید، نه فقط در رندر صفحه.

22 امنیت اپلیکیشن – فصل کامل

امنیت یک ویژگی نیست، یک لایه در سراسر اپ است. این فصل، تهدیدهای واقعی و دفاع عملی در برابر هرکدام را در Next.js 16 پوشش می‌دهد.

۱ | اعتبارسنجی سمت سرور (خط اول دفاع)

هر داده‌ای که از کلاینت می‌آید «آلوده» فرض می‌شود؛ محدودیت‌های الی هیچ ارزش امنیتی ندارند چون قابل دور زدن‌اند. هر Server Action و Route Handler باید ورودی را **روی سرور** با یک schema اعتبارسنجی کند:

src/features/appointments/schema.ts

```
import { z } from "zod";

// 🛡️ single source of truth for shape + validation
export const createAppointmentSchema = z.object({
  doctorId: z.string().uuid(),
  startsAt: z.coerce.date().min(new Date(), "زمان باید در آینده باشد"),
  note: z.string().max(500).optional(),
});
export type CreateAppointmentInput = z.infer<typeof createAppointmentSchema>;
```

using it inside a Server Action

```
"use server";
import { createAppointmentSchema } from "../schema";

export async function createAppointment(_prev: unknown, formData: FormData) {
  const parsed = createAppointmentSchema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) {
    return { error: parsed.error.issues[0].message }; // ❌ reject invalid input
  }
  // ...safe to use parsed.data from here
}
```

۲ | بررسی دسترسی: نزدیک به داده، نه UI

پنهان کردن یک دکمه در UI امنیت نیست. مجوز باید همان جا که داده لمس می شود بررسی شود. الگوی درست: یک تابع `requireUser` و یک `requireRole` که در لایه داده صدا زده می شوند:

```
src/features/auth/guard.ts

import "server-only";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { verifySession } from "../session";

export async function requireUser() {
  const token = (await cookies()).get("session")?.value;
  const user = token ? await verifySession(token) : null;
  if (!user) redirect("/login"); // 🛑 no session → out
  return user;
}

export async function requireRole(role: "ADMIN" | "DOCTOR") {
  const user = await requireUser();
  if (user.role !== role) redirect("/403"); // 🚫 wrong role → forbidden
  return user;
}
```

دفاع لایه‌ای (Defense in Depth)

فقط یک «دروازه سریع» برای ریدایرکت است و به تنهایی قابل اعتماد نیست (آسیب پذیری های اخیر `proxy.ts` همین را نشان دادند). بررسی واقعی دسترسی را همیشه در Server Action / DAL تکرار کنید. اگر `proxy` دور زده شود، این لایه جلوی نشت داده را می گیرد.

۳ | کوکی های امن و مدیریت session

توکن session را در یک کوکی با پرچم های امنیتی درست بگذارید تا از سرقت (XSS) و ارسال بین سایتی (CSRF) در امان بماند:

secure session cookie

```
(await cookies()).set("session", token, {
  httpOnly: true, // 🛡️ JavaScript can't read it (blocks XSS token theft)
  secure: true, // 🛡️ sent only over HTTPS
  sameSite: "lax", // 🛡️ not sent on cross-site requests (CSRF mitigation)
  path: "/",
  maxAge: 60 * 60 * 24 * 7,
});
```

XSS و CSRF (۴ |

تهدید	دفاع در Next.js
XSS (تزریق اسکریپت)	React به صورت پیش فرض خروجی را escape می کند. هرگز از <code>dangerouslySetInnerHTML</code> با ورودی کاربر استفاده نکنید؛ اگر لازم شد، اول با کتابخانه ای مثل DOMPurify پاک سازی کنید.
CSRF (درخواست جعلی)	Server Actions به صورت داخلی در برابر CSRF محافظت شده اند (بررسی Origin). کوکی session را <code>sameSite: "lax"</code> بگذارید. برای Route Handler های حساس، Origin/توکن را بررسی کنید.
سرقت توکن	کوکی <code>httpOnly</code> تا JS نتواند بخواندش؛ توکن را در <code>localStorage</code> نگذارید.

(۵ | محدودسازی نرخ (Rate Limiting)

اندپوینت های حساس (ورود، ثبت نام، ارسال OTP) را در برابر حمله brute-force با محدودسازی نرخ مبتنی بر IP محافظت کنید:

rate limit a login action

```
import { Redis } from "@upstash/redis";
import { headers } from "next/headers";

const redis = Redis.fromEnv();

export async function rateLimit(key: string, max = 5, windowSec = 60) {
  const ip = (await headers()).get("x-forwarded-for") ?? "anon";
  const bucket = `rl:${key}:${ip}`;
  const hits = await redis.incr(bucket); // 📈 count attempts
```

```
if (hits === 1) await redis.expire(bucket, windowSec);
if (hits > max) throw new Error("تلاش بیش از حد؛ کمی بعد دوباره امتحان کنید");
}
```

۶ | امنیت متغیرهای محیطی

- اسرار (کلید API، رشته اتصال) را هرگز با `NEXT_PUBLIC_` اکسپورت نکنید؛ این‌ها داخل باندل کلاینت می‌روند و برای همه قابل مشاهده‌اند.
- ماژول‌های حاوی اسرار را با `import "server-only"` علامت بزنید تا اگر به‌اشتباه در کلاینت `import` شدند، بیلد خطا دهد.
- متغیرهای محیطی را هنگام بالا آمدن اپ با `zod` اعتبارسنجی کنید تا پیکربندی غلط زود لو برود.

```
src/lib/env.ts - validated env

import "server-only";
import { z } from "zod";

const schema = z.object({
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32), // 🔑 enforce a strong secret
});

// 💥 fails fast at startup if the environment is misconfigured
export const env = schema.parse(process.env);
```

۷ | هدرهای امنیتی و CSP

- یک لایه دفاعی مهم که اغلب فراموش می‌شود: هدرهای امنیتی HTTP. این‌ها را یک‌بار برای همه پاسخ‌ها در `next.config` تنظیم کنید. مهم‌ترینشان **Content-Security-Policy** است که قوی‌ترین لایه دفاع در برابر XSS به‌شمار می‌رود:

```
next.config.ts - baseline security headers

import type { NextConfig } from "next";

// 🛡 applied to every response
const securityHeaders = [
  { key: "X-Frame-Options", value: "DENY" }, // 🚫 clickjacking
```

```

{ key: "X-Content-Type-Options", value: "nosniff" }, // 🛡️ no MIME sniff
{ key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
{ key: "Strict-Transport-Security", value: "max-age=63072000; includeSubDomains" }, // 🛡️
force HTTPS
{ key: "Content-Security-Policy", value: "default-src 'self'; img-src 'self' data;;
script-src 'self'" }, // 🛡️ strongest XSS defense
];

const nextConfig: NextConfig = {
  async headers() {
    return [{ source: "/*", headers: securityHeaders }]; // 🛡️ every route
  },
};
export default nextConfig;

```

CSP را با احتیاط تنظیم کنید

سیاست بالا یک نقطه شروع سخت‌گیرانه است؛ باید متناسب با منابع واقعی اپ (اسکریپت‌های ثالث، فونت، تصویر) تنظیم شود. با یک سیاست سخت شروع کنید و فقط منابع لازم را اضافه کنید – نه برعکس.

چک‌لیست امنیتی سریع

ورودی روی سرور اعتبارسنجی شده؟ دسترسی در DAL بررسی می‌شود (نه فقط UI)؟ کوکی session `httpOnly + secure + sameSite` است؟ اسرار خارج از `NEXT_PUBLIC_` مانده؟ اندپوینت‌های ورود rate-limit دارند؟ اگر پاسخ همه «بله» است، از اکثر اپ‌های واقعی امن‌ترید.

تمرین

یک Server Action ورود بنویسید که: (۱) ورودی را با zod اعتبارسنجی کند، (۲) قبل از بررسی رمز، `rateLimit("login")` را صدا بزند، و (۳) در صورت موفقیت یک کوکی session امن ست کند. سپس تلاش کنید با یک اسکریپت ساده آن را brute-force کنید و ببینید rate limit جلوی شما را می‌گیرد.

سؤالات مصاحبه (با پاسخ)

XSS – Junior و CSRF چه فرقی دارند؟ XSS یعنی تزریق و اجرای اسکریپت مخرب در مرورگر قربانی (سرقت داده/توکن)؛ CSRF یعنی وادار کردن مرورگر کاربر وارد شده به ارسال یک درخواست ناخواسته به سایتی که در آن لاگین است. XSS درباره «اجرای کد» است، CSRF درباره «سوءاستفاده از session معتبر».

Mid – چرا اعتبارسنجی سمت کلاینت کافی نیست و کجا باید انجام شود؟ چون کلاینت کاملاً قابل دور زدن است (درخواست دستی، تغییر DevTools). اعتبارسنجی کلاینت فقط برای تجربه کاربری است؛ اعتبارسنجی واقعی باید همیشه سمت سرور و نزدیک به داده (اکشن/Route Handler با zod) باشد.

Senior – چند لایه دفاعی کلیدی برای یک اپ Next.js نام ببرید و بگویید injection (به ویژه NoSQL) را چگونه مهار می‌کنید. اعتبارسنجی و پاک‌سازی ورودی با zod، کنترل دسترسی داخل اکشن/DAL، کوکی‌های `httpOnly/secure/sameSite`، محدودسازی نرخ روی اندپوینت‌های حساس، هدرهای امنیتی و CSP، و مدیریت اسرار با `server-only`. برای NoSQL injection، هرگز ورودی خام را مستقیم به کوئری ندهید؛ با schema نوع و شکل داده را اجبار کنید و از اپراتورهای تزریقی (مثل ورودی آجکتی که به `$gt` تبدیل می‌شود) با اعتبارسنجی نوع جلوگیری کنید.

23 فرم‌ها و اعتبارسنجی پیشرفته

فرم‌ها قلب اپلیکیشن‌های واقعی‌اند. اینجا الگوهای حرفه‌ای فرم با Server Actions، وضعیت ارسال، اعتبارسنجی دوطرفه و بهبود تدریجی را می‌بینیم.

useFormStatus برای وضعیت دکمه

به‌جای پاس‌دادن دستی `pending`، دکمه submit را در کامپوننت جدا بگذارید و از `useFormStatus` استفاده کنید؛ این هوک وضعیت فرم والد را خودکار می‌خواند:

```
src/components/submit-button.tsx

"use client";
import { useFormStatus } from "react-dom";

export function SubmitButton({ label }: { label: string }) {
  const { pending } = useFormStatus(); // 📡 reads parent <form> status
  return (
    <button type="submit" disabled={pending}>
      {pending ? "در حال ارسال" : label}
    </button>
  );
}
```

اعتبارسنجی دوطرفه با یک schema مشترک

یک schema بنویسید و آن را هم در کلاینت (UX سریع) و هم در سرور (امنیت) استفاده کنید. هرگز فقط به اعتبارسنجی کلاینت اعتماد نکنید:

```
src/lib/schemas.ts

import { z } from "zod";

// 🔄 Shared schema: import in both client and server
export const appointmentSchema = z.object({
  patient: z.string().min(2, "نام را کامل وارد کنید"),
  phone: z.string().regex(/^09\d{9}$/, "موبایل معتبر نیست"),
});
```

```

    date: z.coerce.date(),
  });
export type AppointmentInput = z.infer<typeof appointmentSchema>;

```

src/app/book/actions.ts

```

"use server";
import { appointmentSchema } from "@lib/schemas";
import { revalidatePath } from "next/cache";

export type FormState = { ok?: boolean; errors?: Record<string, string[]> };

export async function book(_prev: FormState, formData: FormData): Promise<FormState> {
  const parsed = appointmentSchema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) {
    // 📄 return field-level errors back to the form
    return { errors: parsed.error.flatten().fieldErrors };
  }
  await db.appointment.create({ data: parsed.data });
  revalidatePath("/book");
  return { ok: true };
}

```

src/app/book/form.tsx

```

"use client";
import { useActionState } from "react";
import { book, type FormState } from "./actions";
import { SubmitButton } from "@components/submit-button";

export function BookingForm() {
  const [state, action] = useActionState<FormState, FormData>(book, {});
  return (
    <form action={action} className="space-y-3">
      <div>
        <label htmlFor="patient">نام بیمار</label>
        <input
          id="patient" name="patient"
          aria-invalid={!!state.errors?.patient} // 🚫 flag invalid field
          aria-describedby={state.errors?.patient ? "patient-err" : undefined}
        />
        {state.errors?.patient && (
          <p id="patient-err" className="text-red-600">{state.errors.patient[0]}</p>

```

```

    })
  </div>

  <div>
    <label htmlFor="phone">موبایل</label>
    <input
      id="phone" name="phone"
      aria-invalid={!state.errors?.phone}
      aria-describedby={state.errors?.phone ? "phone-err" : undefined}
    />
    {state.errors?.phone && (
      <p id="phone-err" className="text-red-600">{state.errors.phone[0]}</p>
    )}
  </div>

  <div>
    <label htmlFor="date">تاریخ</label>
    <input id="date" name="date" type="date" />
  </div>

  <SubmitButton label="ثبت نوبت" />
  {/  role=status is announced by screen readers */}
  {state.ok && <p role="status" className="text-green-600">نوبت ثبت شد </p>}
</form>
);
}

```

بهبود تدریجی (Progressive Enhancement)

چون اکشن مستقیم به `action` فرم وصل است، فرم حتی پیش از بارگذاری کامل JavaScript (یا با JS غیرفعال) کار می‌کند. این یک مزیت بزرگ دسترس‌پذیری و پایداری است که با فرم‌های صرفاً کلاینتی به‌دست نمی‌آید.

فرم‌های پیچیده

برای فرم‌های بسیار بزرگ با منطق پیچیده (آرایه‌های پویا، مراحل متعدد)، ترکیب `react-hook-form` در کلاینت برای UX + ارسال نهایی به یک Server Action برای ثبت و اعتبارسنجی سروری، الگوی بسیار رایج و قدرتمندی است.

اشتباهات رایج / روش درست

اشتباه ❌	درست ✅
اتکا فقط به اعتبارسنجی کلاینت	schema مشترک؛ اعتبارسنجی نهایی روی سرور
input بدون <code><label></code> و فقط با <code>label</code> یا <code>htmlFor</code> + پیوند خطا با <code>aria-</code> <code>placeholder</code>	<code>describedby</code>
مدیریت دستی <code>pending</code> با <code>useState</code>	<code>useFormStatus</code> در دکمه جدا
نمایش خطا بدون بازگرداندن خطای فیلد از سرور	بازگرداندن <code>fieldErrors</code> و نمایش زیر هر فیلد

Best Practices و نکات طلایی ✅

(۱) یک `schema`، دو مصرف (کلاینت برای UX، سرور برای امنیت). (۲) هر فیلد `label` واقعی داشته باشد و خطا با `aria-invalid` / `aria-describedby` به آن پیوند بخورد. (۳) دکمه `submit` را جدا کنید تا `useFormStatus` وضعیت را بگیرد. (۴) پیام موفقیت را با `role="status"` اعلام کنید. (۵) اکشن را مستقیم به `action` فرم وصل کنید تا بدون JS هم کار کند.

🚀 مینی پروژه: فرم ثبت نوبت دسترس پذیر

فرم «ثبت نوبت» را با یک `schema.ts` مشترک بسازید که هم تایپها را می‌دهد و هم روی سرور اعتبارسنجی می‌کند. خطاهای فیلدی را از اکشن برگردانید و زیر هر ورودی نشان دهید؛ هر فیلد `label` و پیوند `aria-describedby` داشته باشد. در آخر با غیرفعال کردن JavaScript مرورگر، مطمئن شوید فرم هنوز ثبت می‌شود (بهبود تدریجی).

سؤالات مصاحبه (با پاسخ)

Junior – تفاوت `useFormStatus` و `useActionState` چیست؟ `useFormStatus` فقط وضعیت «در حال ارسال» فرم والد را می‌دهد (برای دکمه)؛ `useActionState` نتیجه/خطای بازگشتی از Server Action و تابع اکشن پیچیده‌شده را مدیریت می‌کند (برای کل فرم).

Mid – چرا یک `schema` مشترک بین کلاینت و سرور؟ تا قواعد اعتبارسنجی یک منبع حقیقت داشته باشند: کلاینت برای بازخورد سریع و سرور برای امنیت. هم از تکرار جلوگیری می‌کند، هم تایپ‌ها را از همان `schema` می‌گیریم (`z.infer`).

Senior – یک فرم `Production` چه ویژگی‌های دسترس‌پذیری و پایداری باید داشته باشد؟ `label` مرتبط با هر فیلد، `aria-invalid` و `aria-describedby` برای پیوند خطا، اعلام نتیجه با `role="status"`، اعتبارسنجی سرور اجباری، و اتصال مستقیم اکشن به فرم برای کارکرد بدون Progressive JavaScript (Enhancement). برای فرم‌های پیچیده، `react-hook-form` در کلاینت + Server Action در انتها.

24 مدیریت state و داده در کلاینت

بیشتر داده‌ها باید روی سرور واکنشی شوند، اما گاهی به state کلاینتی و واکنشی سمت مرورگر نیاز دارید. اینجا می‌بینیم چه زمانی و با چه ابزاری.

قاعده تصمیم: سرور یا کلاینت؟

وضعیت	راه‌حل
داده اولیه صفحه	Server Component + <code>await</code>
داده زنده متناوب (polling)	SWR یا TanStack Query در کلاینت
state رابط کاربری (مودال، تب)	<code>useState</code>
state اشتراکی بین کامپوننت‌ها	Context یا Zustand

هوک `use()` برای مصرف Promise

هوک جدید `use()` اجازه می‌دهد یک Promise را که از Server Component پاس داده‌اید، در Client Component «باز» کنید – همراه با `<Suspense>`:

```
server passes a promise, client unwraps it

// Server Component
export default function Page() {
  const dataPromise = getDoctors(); // 🚀 don't await – pass the promise down
  return (
    <Suspense fallback=<p>بارگذاری...</p>>
      <DoctorList promise={dataPromise} />
    </Suspense>
  );
}
```

src/components/doctor-list.tsx

```
"use client";
import { use } from "react";

export function DoctorList({ promise }: { promise: Promise<Doctor[]> }) {
  const doctors = use(promise); // 📺 unwrap the promise (suspends until ready)
  return <ul>{doctors.map((d) => <li key={d.id}>{d.name}</li>)}</ul>;
}
```

SWR برای واکنشی کلاینتی

برای داده‌هایی که باید در مرورگر تازه بمانند (موجودی نوبت، اعلان‌ها)، کشینگ `SWR`، revalidation خودکار و حالت loading/error را مدیریت می‌کند:

src/components/slots.tsx

```
"use client";
import useSWR from "swr";

const fetcher = (url: string) => fetch(url).then((r) => r.json());

export function AvailableSlots({ doctorId }: { doctorId: string }) {
  // 🔄 auto-revalidates on focus/reconnect; refresh every 30s
  const { data, error, isLoading } = useSWR(
    `/api/slots?doctor=${doctorId}`,
    fetcher,
    { refreshInterval: 30_000 }
  );

  if (isLoading) return <p>در حال بررسی نوبت‌ها</p>;
  if (error) return <p>خطا در دریافت</p>;
  return <ul>{data.slots.map((s: string) => <li key={s}>{s}</li>)}</ul>;
}
```

App Router در Context Provider

Providerها باید Client Component باشند. آن‌ها را در یک فایل جدا با `"use client"` بسازید و در Root Layout دور `children` بپیچید — این الگو اجازه می‌دهد بقیه درخت همچنان Server بماند:

src/components/providers.tsx

```
"use client";
import { ThemeProvider } from "@lib/theme";

export function Providers({ children }: { children: React.ReactNode }) {
  return <ThemeProvider>{children}</ThemeProvider>;
}
```

src/app/layout.tsx

```
import { Providers } from "@components/providers";

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="fa" dir="rtl">
      { /* 🌱 children stay Server Components; only Providers is client */ }
      <body><Providers>{children}</Providers></body>
    </html>
  );
}
```

برای state سراسری سبک

اگر فقط چند تکه state اشتراکی دارید، Zustand یا Jotai از Context سبک‌تر و بدون re-render اضافی‌اند. اما همیشه اول بپرسید: آیا این واقعاً باید client-state باشد؟ بسیاری از چیزهایی که قبلاً در state سراسری بودند، حالا بهتر است روی سرور یا در URL نگهداری شوند.

سؤالات مصاحبه (با پاسخ)

Mid – یک state را کجا نگه می‌دارید: کلاینت، سرور یا URL؟ اگر باید قابل اشتراک/بوکمارک باشد (فیلتر، شماره صفحه، تب فعال) در URL؛ اگر منبع حقیقت داده است، روی سرور؛ و فقط UI-state گذرا (باز/بسته منو، مقدار در حال تایپ) در کلاینت. قانون عملی: کمترین state کلاینتی ممکن.

Senior – Context در برابر یک کتابخانه state مثل Zustand؛ Trade-off؟ Context برای مقادیر کم تغییر و کم عمق (تم، زبان، کاربر) مناسب است، اما هر تغییر مقدار همه مصرف‌کننده‌ها را re-render می‌کند. برای state پرتغییر و سراسری، کتابخانه‌ای با selector که فقط اجزای وابسته را re-render می‌کند بهتر است. اما پرسش اول همیشه این است: آیا اصلاً به state کلاینتی نیاز است یا سرور/URL کافی است؟

Senior – فرم چند مرحله‌ای با رفرش، وضعیت کاربر را پاک می‌کند؛ چه می‌کنید؟ وضعیت را در URL (query) یا sessionStorage نگه می‌دارم و هنگام mount بازخوانی می‌کنم (با الگوی mounted برای پرهیز از mismatch در Hydration)، و ثبت نهایی را به یک Server Action می‌سپارم تا اعتبارسنجی و پایداری سمت سرور تضمین شود.

Senior – Context در ریشه اپ باعث شده کل درخت با هر تغییر کوچک re-render شود و تایپینگ کند شده؛ ریشه‌یابی و رفع؟ علت این است که مقدار Context در هر رندر یک آبجکت تازه است و همه مصرف‌کننده‌ها را invalidate می‌کند. مقدار را با useMemo پایدار می‌کنم، Context را به چند Context کوچک‌تر (state جدا از dispatch) می‌شکنم، و برای state پرتغییر سراغ کتابخانه‌ای با selector می‌روم تا فقط اجزای وابسته re-render شوند. اگر state «در حال تایپ» است، آن را محلی نگه می‌دارم نه در ریشه.

Staff – در یک اپلیکیشن بزرگ با ده‌ها تیم، می‌خواهید از رشد بی‌رویه state سراسری جلوگیری کنید؛ چه اصول و مرزهایی تعریف می‌کنید؟ یک «سلسله‌مراتب state» را به عنوان استاندارد معماری تثبیت می‌کنم: پیش‌فرض سرور/URL، سپس state محلی کامپوننت، و state سراسری فقط برای داده واقعاً میان‌بخشی (کاربر، تم). دسترسی به store سراسری را پشت یک لایه feature (hookهای اختصاصی) می‌گذارم تا وابستگی‌ها کنترل شده بماند، از selectorهای ریزدانه برای جلوگیری از re-render آبخاری استفاده می‌کنم، و در بازبینی کد، افزودن به state سراسری را نیازمند توجیه می‌کنم. هدف: state سراسری کوچک، عمدی و قابل ردیابی بماند.

25 تسلط بر کشینگ: لایه‌ها و باطل‌سازی

کشینگ قوی‌ترین اهرم کارایی است و در عین حال رایج‌ترین منبع باگ. اینجا مدل ذهنی کامل کش در نسخه ۱۶ و ابزارهای دقیق باطل‌سازی را می‌بینیم.

لایه‌های کش را بشناسید

Next.js چند لایه کش مستقل دارد. شناختن این‌ها تفاوت بین «چرا تازه نمی‌شود؟» و کنترل کامل است:

لایه	محل	چه چیزی	طول عمر
Request Memoization	سرور	حذف کوئری تکراری در یک درخواست	یک درخواست
Data Cache (use cache)	سرور	نتیجه توابع/کامپوننت‌های کش‌شده	تا revalidate
Full Route Cache	سرور	HTML/RSC صفحات استاتیک	تا revalidate
Router Cache	مرورگر	سگمنت‌های بازدیدشده برای ناوبری سریع	طول session

سه گونه use cache

گونه	دامنه	کاربرد
"use cache"	اشتراکی بین همه	محتوای عمومی (لیست مقالات، تخصص‌ها)
"use cache: private"	per-user	داشبورد یا داده شخصی کاربر
"use cache: remote"	کش پلتفرمی	اشتراک کش بین چند نمونه سرور

دام شماره یک

هرگز داده شخصی را با "use cache" اشتراکی کش نکنید؛ ممکن است داده یک کاربر به کاربر دیگر نشان داده شود. برای داده شخصی حتماً "use cache: private" یا اصلاً بدون کش (پویا) استفاده کنید.

پروفايل‌هاى سفارشى cacheLife

علاوه بر پروفايل‌هاى آماده، مى‌توانيد پروفايل دلخواه با کنترل دقيق تعريف كنيد:

```
next.config.ts

const nextConfig = {
  cacheComponents: true,
  cacheLife: {
    // 🕒 custom profile: fresh 5m, serve stale up to 1h while revalidating
    feed: { stale: 300, revalidate: 60, expire: 3600 },
  },
};
export default nextConfig;
```

```
usage

import { cacheLife } from "next/cache";
export async function getFeed() {
  "use cache";
  cacheLife("feed"); // 🏷 use the custom profile by name
  return db.post.findMany();
}
```

باطل‌سازى: ابزار درست براى هر كار

تابع	چه مى‌كند	كى
<code>revalidateTag("t")</code>	همه كش‌هاى داراى برچسب t	بعد از تغيير گسترده يك نوع داده
<code>revalidatePath("/x")</code>	كش يك مسير مشخص	وقتي فقط يك صفحه تغيير كرده
<code>updateTag("t")</code>	به‌روزرسانى فوري در همان پاسخ	وقتي كاربر بايد نتيجه را بلافاصله ببيند
<code>router.refresh()</code>	تازه‌سازى Router Cache مرورگر	بعد از تغيير كوئى/سشن در كلاينت

```
"use server";
import { revalidateTag, updateTag } from "next/cache";

export async function publishPost(data: PostInput) {
  await db.post.create({ data });
  // 🔄 Background refresh of the public list...
  revalidateTag("posts");
  // ⚡ ...or immediate update so the author sees it right away:
  // updateTag("posts");
}
```

دام‌های رایج کش

- هر تابع `use cache` باید `async` باشد و خروجی‌اش قابل‌سریال‌سازی؛ تابع، کلاس یا Promise‌های پویا را نمی‌توان کش کرد.
- داخل `scope` کش‌شده مستقیم `cookies()` / `headers()` نخوانید؛ بیرون بخوانید و به‌عنوان آرگومان پاس دهید.
- Cache Components با Edge Runtime پشتیبانی نمی‌شود؛ روی Node اجرا کنید.
- اگر `revalidateTag` اثر نکرد: املائی برچسب را چک کنید (حساس به حروف)، مطمئن شوید `cacheTag` داخل همان تابع کش صدا زده شده، و باطل‌سازی **بعد** از موفقیت `mutation` انجام شده.

دیباگ کش

هنگام `build`، خروجی Next.js کنار هر مسیر نشان می‌دهد استاتیک است یا پویا (○ استاتیک، f پویا). اگر مسیری ناخواسته پویا شده، معمولاً به‌خاطر خواندن `cookies()` / `headers()` در سطح صفحه است؛ آن دسترسی را به یک کامپوننت کوچک‌تر و داخل `Suspense` منتقل کنید.

سؤالات مصاحبه (با پاسخ)

Mid – کاربر می‌گوید داده‌اش «کهنه» است؛ چطور دیباگ می‌کنید؟ لایه‌ها را از بالا بررسی می‌کنم: آیا آن تابع/صفحه `use cache` دارد؟ `cacheLife` چقدر است؟ آیا بعد از تغییر، `revalidateTag/Path` صدا زده شده؟ آیا داده اشتبهاً اشتراکی کش شده؟ و آیا Router Cache سمت کلاینت نسخه قدیمی نگه داشته است؟

Senior – استراتژی باطل‌سازی برای فروشگاه‌های با هزاران محصول؟ از باطل‌سازی مبتنی بر برچسب ریزدانه استفاده می‌کنم (`cacheTag("product:"+id)`) تا با تغییر یک محصول فقط همان باطل شود نه کل فهرست؛ `cacheLife` را متناسب با نرخ تغییر واقعی تنظیم می‌کنم و برای تغییرات دسته‌ای یک برچسب سطح بالاتر می‌گذارم. باطل‌سازی بیش‌ازحد، خود کش را بی‌اثر می‌کند.

Senior – بعد از انتشار، بعضی کاربران قیمت قدیمی می‌بینند؛ ریشه‌یابی؟ بررسی می‌کنم که آیا mutation مسیر/برچسب درست را باطل می‌کند، سپس لایه CDN/Edge را چک می‌کنم. برای داده واقعاً لحظه‌ای مثل قیمت، آن بخش را از کش خارج و در `<Suspense>` استریم می‌کنم تا پوسته صفحه کش‌شده بماند اما قیمت همیشه تازه باشد (روح PPR).

Senior – یک داشبورد شخصی کاربر به‌اشتباه بین کاربران «نشت» کرده و هرکس داده دیگری را می‌بیند؛ چه اتفاقی افتاده؟ تقریباً همیشه یعنی داده per-user با `use cache` اشتراکی کش شده یا یک تابع کش‌شده به‌جای دریافت شناسه کاربر به‌عنوان آرگومان، آن را از یک منبع سراسری خوانده است. رفع: کلید کش باید شامل هویت کاربر باشد یا از `use cache: private` استفاده شود؛ و داده حساس اصلاً نباید اشتراکی کش شود. این یک باگ امنیتی است، نه فقط کارایی.

Staff – برای یک پلتفرم چند-instance با ترافیک بالا، یک استراتژی کش end-to-end طراحی کنید و لایه‌ها را نام ببرید. لایه‌ها را از کاربر تا داده می‌چینم: CDN/Edge برای دارایی‌ها و صفحات کاملاً استاتیک، کش داده برنامه (`cacheTag` + `use cache`) با یک Cache Handler مشترک مثل Redis تا بین instance‌ها یکسان و قابل باطل‌سازی باشد، PPR برای صفحات ترکیبی، و Router Cache کلاینت برای ناوبری نرم. کلید طراحی: هر لایه `cacheLife` متناسب با نرخ تغییر داده و یک مسیر باطل‌سازی رویدادمحور (بعد از mutation) دارد؛ و مرز «داده عمومی در برابر شخصی» در همه لایه‌ها حفظ می‌شود.

26 کارایی و Core Web Vitals

سرعت، هم تجربه کاربر و هم رتبه سئو را می‌سازد. اینجا معیارهای کلیدی و ابزارهای Next.js برای بهینه‌سازی هرکدام را می‌بینیم.

سه معیار حیاتی

معیار	چه می‌سنجد	هدف
LCP	زمان نمایش بزرگ‌ترین عنصر	کمتر از ۲.۵ ثانیه
CLS	میزان پرش چیدمان	کمتر از ۰.۱
INP	سرعت پاسخ به تعامل کاربر	کمتر از ۲۰۰ms

بهبود LCP

- تصویر اصلی بالای صفحه را با `priority` بارگذاری کنید تا زودتر بیاید.
- پوسته استاتیک را با `use cache` کش کنید تا فوری سرو شود (PPR).
- فونت را با `next/font` خودمیزبان کنید تا متن بدون تأخیر نمایش یابد.

● ● ● priority hero image

```
import Image from "next/image";  
// 🚀 priority → preloaded, improves LCP for the main image  
<Image src={hero} alt="" priority sizes="100vw" />
```

حذف پرش چیدمان (CLS)

- همیشه برای تصاویر `width` / `height` یا `fill` بدهید تا فضا رزرو شود.
- برای فونت از `display: "swap"` و در صورت امکان `adjustFontFallback` استفاده کنید.
- برای محتوای تأخیری، اسکلت (skeleton) هم‌اندازه نمایش دهید تا چیدمان نپرد.

بهبود INP و کاهش JS

- ◆ کامپوننت را Server نگه دارید مگر تعامل لازم باشد؛ این بزرگ‌ترین کاهش JS است.
- ◆ کامپوننت‌های سنگین تعاملی را با `next/dynamic` به تنبلی بارگذاری کنید.
- ◆ به‌روزرسانی‌های غیرفوری را در `useTransition` بپیچید تا UI پاسخ‌گو بماند.

```
useTransition keeps UI responsive

"use client";
import { useTransition, useState } from "react";

export function Filter() {
  const [isPending, startTransition] = useTransition();
  const [query, setQuery] = useState("");
  return (
    <input
      onChange={(e) => {
        const v = e.target.value;
        setQuery(v); // urgent: keep the input responsive
        // ⌚ non-urgent heavy update is deferred, never blocks typing
        startTransition(() => runExpensiveFilter(v));
      }}
    />
  );
}
```

React Compiler و prefetch

در نسخه ۱۶، **React Compiler** به‌صورت پایدار، memoization خودکار انجام می‌دهد و نیاز به `useMemo` / `useCallback` دستی را کم می‌کند. همچنین `Link` مسیرهای داخل دید را خودکار prefetch می‌کند تا ناوبری آبی شود.

تحلیل و سنجش

```
Terminal

# 📊 See per-route bundle sizes (o static, f dynamic) right in the build output
npm run build
```

```
# 🔍 Deep bundle analysis to find heavy dependencies
npm i -D @next/bundle-analyzer
ANALYZE=true npm run build
```

سه حرکت با بیشترین بازده

۱) Server-first بودن (کمترین JS). ۲) کش کردن پوسته استاتیک + استریم بخش پویا (LCP عالی). ۳) بهینه سازی تصویر و فونت با ابزارهای داخلی (CLS صفر). همین سه، اغلب سایتها را به نمره سبز Core Web Vitals می‌رساند.

سؤالات مصاحبه (با پاسخ)

Mid – معیار INP (کندي تعامل) بالا گزارش شده؛ علتها و رفع؟ معمولاً JS سنگین کلاینت است که ریسمان اصلی را بلاک می‌کند. راه حل: انتقال منطق به سرور (RSC)، code-splitting، استفاده از `startTransition` / `useDeferredValue` برای به روزرسانی‌های سنگین، و حذف re-renderهای غیرضروری.

LCP – Senior روی شبکه ۴G کند است؛ رویکرد سیستماتیک؟ اول المان LCP را شناسایی و `priority` می‌کنم، TTFB را با کش و استریم پوسته (PPR) کم می‌کنم، منابع بلاک کننده را حذف و به مبدأ داده `preconnect` می‌کنم، و payload اولیه را کوچک می‌کنم (JS کمتر با تکیه بر RSC).

Senior – نمره لوکال سبز است اما داده میدانی (RUM) قرمز؛ چرا و چه می‌کنید؟ چون کاربران واقعی دستگاه/شبکه/جغرافیای ضعیف تری دارند. به داده میدانی (CrUX/Speed Insights) تکیه می‌کنم، در تست، شبکه و CPU را throttle می‌کنم، و یک بودجه کارایی تعیین می‌کنم تا رگرسیون‌ها پیش از انتشار گرفته شوند.

Senior – باندل JS کلاینت بی دلیل بزرگ شده؛ چطور پیدا و کوچکش می‌کنید؟ با تحلیل گر باندل، بزرگ ترین مازول‌ها را می‌یابم و دنبال نشت‌های رایج می‌گردم: یک کتابخانه سنگین که کل آن import شده (به جای import نقطه ای)، کدی که بی جهت `"use client"` شده و می‌توانست Server Component بماند، یا یک وابستگی که فقط برای سرور لازم بوده اما به کلاینت رفته. رفع: مرز کلاینت را کوچک نگه می‌دارم، dynamic import برای بخش‌های سنگین/غیرحیاتی، و انتقال منطق به سرور.

Staff – مسئول کارایی یک محصول بزرگ هستید که به مرور کند شده؛ چه چارچوبی برای شناسایی و جلوگیری از رگرسیون تعریف می‌کنید؟ کارایی را از یک تلاش موردی به یک فرایند تبدیل می‌کنم: تعریف بودجه صریح برای هر مسیر بحرانی (LCP/INP/CLS و اندازه باندل)، اجرای این بودجه در CI تا PR ناقض رد شود، مانیتورینگ میدانی مستمر با هشدار روی رگرسیون، و «رهگیری منشأ» تا هر افت به یک تغییر مشخص نسبت داده شود. در کنارش، الگوهای پیش فرض سریع (RSC-first، تصویر/فونت استاندارد، استریم) را در معماری پایه می‌گذارم تا مسیر درست، مسیر ساده باشد.

27 Metadata و سئو – کامل

از تگ‌های پایه تا داده ساختاریافته و تصویر OG پویا. سئوی فنی درست، تفاوت دیده‌شدن یا نشدن سایت شماست.

metadataBase و قالب عنوان

در Root Layout یک `metadataBase` ست کنید تا آدرس‌های نسبی (OG image، canonical) به آدرس مطلق درست تبدیل شوند:

```
src/app/layout.tsx

import type { Metadata } from "next";

export const metadata: Metadata = {
  metadataBase: new URL("https://doctorrezro.ir"), // 🌐 base for all relative URLs
  title: { default: "دکتر رزرو", template: "%s | دکتر رزرو" },
  description: "رزرو آنلاین نوبت پزشک",
  alternates: { canonical: "/" }, // 🔗 canonical to avoid duplicate content
};
```

Twitter Card و Open Graph

برای پیش‌نمایش زیبا هنگام اشتراک‌گذاری در شبکه‌های اجتماعی:

```
src/app/doctors/[id]/page.tsx

export async function generateMetadata({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const doctor = await getDoctor(id);
  return {
    title: doctor.name,
    description: `رزرو نوبت با ${doctor.name}`,
    openGraph: {
      title: doctor.name,
      images: [`/og/${id}`], // 🖼️ resolved against metadataBase
      type: "profile",
    },
  },
}
```

```

    twitter: { card: "summary_large_image" },
  };
}

```

تصویر OG پویا با ImageResponse

به جای ساخت دستی تصویر برای هر صفحه، یک `opengraph-image.tsx` بسازید که تصویر را در لحظه با JSX تولید کند:

```

src/app/doctors/[id]/opengraph-image.tsx

import { ImageResponse } from "next/og";

export const size = { width: 1200, height: 630 };
export const contentType = "image/png";

export default async function OG({ params }: { params: { id: string } }) {
  const doctor = await getDoctor(params.id);
  // 🐛 Generates a PNG on the fly from JSX
  return new ImageResponse(
    (
      <div style={{ display: "flex", width: "100%", height: "100%",
        alignItems: "center", justifyContent: "center",
        fontSize: 64, background: "#1e1b4b", color: "#fff" }}>
        {doctor.name}
      </div>
    ),
    size
  );
}

```

داده ساختاریافته (JSON-LD)

برای نمایش بهتر در نتایج گوگل (rich results)، داده ساختاریافته را به صورت یک اسکریپت JSON-LD در صفحه بگذارید:

```

structured data

export default async function DoctorPage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const doctor = await getDoctor(id);

```

```
const jsonLd = {
  "@context": "https://schema.org",
  "@type": "Physician",
  name: doctor.name,
  medicalSpecialty: doctor.specialty,
};
return (
  <
    { /* 🕒 Helps Google render a rich result */}
    <script type="application/ld+json"
      dangerouslySetInnerHTML={{ __html: JSON.stringify(jsonLd) }} />
    <Profile doctor={doctor} />
  </>
);
}
```

robots و sitemap

دو فایل ویژه بسازید تا Next.js خروجی استانداردشان را تولید کند:

```
src/app/robots.ts

import type { MetadataRoute } from "next";
export default function robots(): MetadataRoute.Robots {
  return {
    rules: { userAgent: "*", allow: "/", disallow: "/dashboard" },
    sitemap: "https://doctorrezro.ir/sitemap.xml",
  };
}
```

دام سئوی پرتکرار

اگر `metadataBase` یا `NEXT_PUBLIC_BASE_URL` در Production اشتباه (مثلاً روی `localhost`) بماند، همه `canonical` ها و `OG image` ها خراب می‌شوند. این مقدار را در محیط Production حتماً درست تنظیم کنید.

Mid – صفحه محصول باید عنوان/توضیح و OG داینامیک داشته باشد؛ چطور؟ از `generateMetadata` `async` استفاده می‌کنم که داده را واکنشی و عنوان/توضیح/تصویر OG را بر اساس همان محصول می‌سازد، و `metadataBase` را تنظیم می‌کنم تا URL های OG مطلق و درست تولید شوند.

Senior – یک SPA قدیمی سئوی ضعیفی داشت؛ چرا **App Router** بهتر است و چه چیزهایی را باید پوشش داد؟ چون HTML سمت سرور تولید می‌شود و خزنده بدون اجرای JS محتوا را می‌بیند. باید `per-metadata` `route`، `sitemap` و `robots` داینامیک، تگ `canonical`، و داده ساخت یافته (JSON-LD) را پوشش داد و مطمئن شد محتوای کلیدی سمت سرور رندر می‌شود، نه پشت `useEffect`.

Senior – صفحات فیلترشده (`?color=red`) محتوای تکراری و مشکل `canonical` ساخته‌اند؛ راهکار؟ برای نسخه‌های فیلترشده `canonical` را به صفحه اصلی تنظیم می‌کنم، درباره `index/noindex` پارامترها تصمیم می‌گیرم، و مراقبم که خواندن `searchParams` کل صفحه را بی‌جهت پویا/CSR نکند تا هم سئو و هم کارایی حفظ شود.

Senior – بعد از انتشار، گوگل صفحات را ایندکس نمی‌کند و در منبع صفحه، محتوا خالی دیده می‌شود؛ ریشه‌یابی؟ بررسی می‌کنم که محتوای کلیدی سمت سرور رندر شود، نه پشت `useEffect` در یک `Client Component`. سپس `robots` /متادیتا را برای `noindex` ناخواسته چک می‌کنم، وضعیت واقعی را با «مشاهده به عنوان گوگل» (URL Inspection) می‌سنجم، و مطمئن می‌شوم صفحه به خاطر خطای زمان اجرا یا ریدایرکت اشتباه، HTML خالی برنمی‌گرداند.

Staff – برای یک سایت بزرگ با میلیون‌ها صفحه پویا (مثل مارکت پلیس)، معماری سئوی فنی و `sitemap` را چطور طراحی می‌کنید؟ `sitemap` را تکه‌ای (`sitemap index` + چند `sitemap` فرزند) و به صورت پویا از داده تولید می‌کنم تا از سقف اندازه عبور نکند، متادیتا را `per-route` با `generateMetadata` و کش متناسب می‌سازم تا فشار داده کنترل شود، داده ساخت یافته (JSON-LD) و `canonical` را استاندارد می‌کنم، و برای صفحات پرارزش رندر سروری/ISR و برای دُم بلند کم‌بازدید تولید به درخواست می‌گذارم. کلید مقیاس: سئو باید از داده منبع مشتق شود، نه دستی نگه‌داری شود.

28 TypeScript پیشرفته و امنیت تایپ

تایپ‌های درست، خطاها را از زمان اجرا به زمان کامپایل منتقل می‌کنند. اینجا الگوهای امنیت تایپ سرتاسری در Next.js را می‌بینیم.

تایپ صحیح Props صفحه و Layout

چون `params` و `searchParams` در نسخه ۱۶ Promise هستند، آن‌ها را دقیق تایپ کنید:

```
typed page

type PageProps = {
  params: Promise<{ id: string }>;
  searchParams: Promise<Record<string, string | string[] | undefined>>;
};

export default async function Page({ params, searchParams }: PageProps) {
  const { id } = await params;
  const { sort } = await searchParams;
  return <View id={id} sort={typeof sort === "string" ? sort : undefined} />;
}
```

مسیرهای تایپ‌شده (Typed Routes)

با فعال‌سازی این گزینه، هر `href` اشتباه در `Link` همان لحظه خطای کامپایل می‌دهد:

```
next.config.ts

const nextConfig = {
  typedRoutes: true, // 🔗 type-safe <Link href> across the whole app
};

export default nextConfig;
```

اکشن‌های تایپ‌شده با useActionState

نوع state و ورودی اکشن را با generic مشخص کنید تا کل چرخه فرم type-safe بماند:

● ● ● typed action state

```
type State = { ok?: boolean; error?: string };

// ✅ State and FormData types flow through end-to-end
const [state, action] = useActionState<State, FormData>(submit, {});
```

استنتاج تایپ از zod با infer

به جای نوشتن دوباره تایپ‌ها، آن‌ها را از schema استخراج کنید تا منبع حقیقت یکی بماند:

● ● ● single source of truth

```
import { z } from "zod";

const userSchema = z.object({ name: z.string(), age: z.number() });

// 🕒 Type derived from the schema – never drifts out of sync
export type User = z.infer<typeof userSchema>;
```

satisfies برای پیکربندی‌های امن

عملگر `satisfies` هم تایپ را اعتبارسنجی می‌کند و هم نوع دقیق را حفظ می‌کند – عالی برای آجکت‌های پیکربندی:

● ● ● satisfies operator

```
import type { Metadata } from "next";

// ✅ Checked against Metadata, but keeps the precise literal types
export const metadata = {
  title: "دکتر رزرو",
  description: "رزرو نوبت",
} satisfies Metadata;
```

عادت طلایی

قبل از هر تحویل، `npx tsc --noEmit` بزنید و آن را در CI اجباری کنید. صفر خطای تایپ پیش از merge، یکی از ارزان‌ترین و مؤثرترین تضمین‌های کیفیت است.

سؤالات مصاحبه (با پاسخ)

Mid – یک API دو شکل پاسخ دارد (موفق/خطا)؛ چطور **type-safe** مدل می‌کنید؟ با یک «اتحاد تفکیک‌شده» (discriminated union) و فیلد نشانه مثل `ok: true | false`؛ آنگاه بعد از `if (res.ok)` تایپ‌اسکرپت نوع را باریک می‌کند و دسترسی به فیلدهای هر حالت امن و بدون `any` می‌شود.

Senior – `satisfies` چه مشکلی را حل می‌کند که `annotation` ساده حل نمی‌کند؟ با `obj satisfies T` هم انطباق با نوع `T` بررسی می‌شود و هم نوع دقیق literal حفظ می‌ماند؛ در حالی که `const obj: T` نوع را به `T` گشاد می‌کند و اطلاعات دقیق کلیدها/مقادیر از دست می‌رود. برای آبجکت‌های `config` که می‌خواهید کلیدهایشان دقیق بماند، ایده‌آل است.

Senior – **type-safety** داده را از دیتابیس تا UI چطور یکپارچه حفظ می‌کنید؟ یک منبع نوع واحد از `schema` می‌سازم (با `z.infer` از `Zod` یا `InferSchemaType` از `Mongoose`)، در مرز سرور/کلاینت DTOهای سریالایزپذیر و بدون `any` رد می‌کنم، و همان تایپ‌ها را تا کامپوننت‌ها می‌برم تا هر تغییر مدل، خطای تایپ در کل مسیر بدهد.

Senior – تیمی برای «ساکت‌کردن» خطاها از `any` و `as` زیاد استفاده می‌کند؛ چرا خطرناک است و جایگزین‌ها چیست؟ `any` بررسی نوع را کاملاً خاموش می‌کند و باگ را به زمان اجرا موکول می‌کند؛ `as` یک ادعای بدون تضمین است که می‌تواند دروغ باشد. جایگزین‌ها: `unknown` + باریک‌سازی با `type guard`، اعتبارسنجی زمان‌اجرا با `Zod` در مرزهای ورودی (تا نوع تضمین‌شده شود)، و `generics` به جای `as`. فقط `cast`. جایی مجاز است که واقعاً بیش از کامپایلر می‌دانیم و آن را مستند می‌کنیم.

Staff – می‌خواهید در یک **monorepo** بزرگ، قرارداد نوع بین چند اپ/پکیج پایدار و مقاوم به تغییر بماند؛ چه رویکردی؟ تایپ‌های مشترک را در یک پکیج مرزی واحد متمرکز می‌کنم و بقیه از آن مصرف می‌کنند (منبع حقیقت نوع)، در APIها به جای تایپ دستی از تایپ مشتق‌شده از `schema` استفاده می‌کنم تا سرور و کلاینت هم‌زمان به‌روز شوند، `strict` و قواعد سخت‌گیرانه را در کل ریپو اجباری می‌کنم، و تغییرات ناسازگار نوع را مثل تغییر API عمومی مدیریت می‌کنم (نسخه‌بندی و بازبینی). هدف: خطای قرارداد در «زمان کامپایل» ظاهر شود، نه در `Production`.

29 مدیریت خطا – کامل

یک اپلیکیشن حرفه‌ای هیچ‌وقت با صفحه سفید نمی‌شکند. اینجا تمام لایه‌های مدیریت خطا در App Router را می‌بینیم.

سلسله‌مراتب فایل‌های خطا

فایل	چه خطایی را می‌گیرد
<code>error.tsx</code>	خطای رندر در همان سگمنت و فرزندانش
<code>global-error.tsx</code>	خطای Root Layout (آخرین خط دفاع)
<code>not-found.tsx</code>	حالت ۴۰۴ (فراخوانی <code>notFound()</code>)

`error.tsx` برای بازیابی سگمنت

این فایل باید Client Component باشد و یک تابع `reset` می‌گیرد که اجازه می‌دهد کاربر بدون رفرش کل صفحه، همان بخش را دوباره تلاش کند:

```
src/app/doctors/error.tsx

"use client";
import { useEffect } from "react";

export default function Error({
  error, reset,
}: { error: Error & { digest?: string }; reset: () => void }) {
  useEffect(() => {
    // 📄 log to your monitoring service (digest helps correlate server logs)
    reportError(error);
  }, [error]);

  return (
    <div className="p-6 text-center">
      <p>متأسفیم، بارگذاری این بخش با خطا مواجه شد</p>
      <button onClick={() => reset()} className="mt-2 underline">تلاش دوباره</button>
    </div>
  );
}
```

```

    </div>
  );
}

```

خطاهای «مورد انتظار» در برابر «غیرمنتظره»

دو نوع خطا را متفاوت مدیریت کنید:

- ♦ **مورد انتظار** (ورودی نامعتبر، نبود دسترسی): به جای `throw`، یک مقدار برگردانید و در `error.tsx` نشان دهید. اینها بخشی از جریان عادی اند.
- ♦ **غیرمنتظره** (قطع دیتابیس، باگ): اجازه دهید `throw` شوند تا `error.tsx` آنها را بگیرد.

```

● ● ● expected errors: return, don't throw

"use server";
export async function transfer(amount: number) {
  const session = await getSession();
  if (!session) return { error: "ابتدا وارد شوید" }; // ● expected → return
  if (amount ≤ 0) return { error: "مبلغ نامعتبر" }; // ● expected → return

  await db.tx.create({ data: { amount } }); // ● if DB fails, it throws → error.tsx
  return { ok: true };
}

```

notFound و صفحه ۴۰۴

وقتی منبعی وجود ندارد، `notFound()` را صدا بزنید تا نزدیکترین `not-found.tsx` رندر شود:

```

● ● ● triggering not-found

import { notFound } from "next/navigation";

export default async function DoctorPage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const doctor = await getDoctor(id);
  if (!doctor) notFound(); // ⚠ renders not-found.tsx, returns 404 status
  return <Profile doctor={doctor} />;
}

```

try/catch در واکنشی داده

برای کنترل دقیق‌تر (مثلاً نمایش حالت جایگزین به جای شکستن کل صفحه)، خطا را همان‌جا بگیرید:

```
graceful degradation

async function Weather() {
  try {
    const data = await getWeather(); // external API that may fail
    return <WeatherCard data={data} />;
  } catch {
    // ☁️ degrade gracefully instead of crashing the page
    return <p className="text-gray-500">اطلاعات آب‌وهوا فعلاً در دسترس نیست.</p>;
  }
}
```

معماری خطای پیشنهادی

یک `error.tsx` در سطح بالا برای حالت کلی + `error.tsx` های محلی برای بخش‌های مستقل (تا خطای یک ویجت کل صفحه را نشکند) + `global-error.tsx` به‌عنوان آخرین تور ایمنی + لاگ‌کردن هر خطای غیرمنتظره به یک سرویس مانیتورینگ. این ترکیب، اپلیکیشن را مقاوم می‌کند.

سؤالات مصاحبه (با پاسخ)

Mid – تفاوت `error.tsx` و `not-found.tsx` و کی هرکدام؟ `not-found` برای حالت «منبع وجود ندارد» است (۴۰۴)، با فراخوانی `notFound()`، و `error.tsx` برای خطای غیرمنتظره رندر یا واکنشی داده که یک Error Boundary سمت کلاینت با تابع `reset` است.

Senior – استراتژی خطا برای یک اپ Production چندلایه چگونه است؟ Error Boundary را نزدیک هر بخش می‌گذارم (نه فقط ریشه) تا خطای یک ویجت کل صفحه را نیندازد، `global-error` را برای خطای خود Root Layout اضافه می‌کنم، به کاربر پیام عمومی می‌دهم و `digest` را به سرویس مانیتورینگ لاگ می‌کنم، و خطای قابل‌بازيابی (retry) را از غیرقابل‌بازيابی تفکیک می‌کنم.

Senior – یک `fetch` در Server Component گاهی تایم‌اوت می‌شود و کل صفحه می‌افتد؛ راهکار؟ آن واکنشی را در یک بخش جدا با `<Suspense>` و Error Boundary مستقل ایزوله می‌کنم، در لایه داده تایم‌اوت و retry می‌گذارم و یک fallback معنادار نشان می‌دهم؛ خطا لاگ می‌شود اما بقیه صفحه سالم می‌ماند.

Senior – یک Server Action گاهی شکست می‌خورد اما کاربر هیچ بازخوردی نمی‌گیرد؛ چطور خطا را تا UI به‌درستی می‌رسانید؟ اکشن باید به‌جای `throw` خام، یک نتیجه ساخت‌یافته `{ ok, error }` برگرداند تا در `useActionState` نمایش داده شود؛ خطای غیرمنتظره را داخل اکشن می‌گیرم، پیام عمومی به کاربر و جزئیات را به مانیتورینگ می‌دهم، و برای خطاهای قابل‌بازيابی امکان retry می‌گذارم. تفاوت کلیدی: خطای «قابل‌انتظار» (اعتبارسنجی) را داده برمی‌گردانم، خطای «سیستمی» را به Error Boundary می‌سپارم.

Staff – برای یک اپلیکیشن بزرگ، یک استراتژی مشاهده‌پذیری (observability) و مدیریت خطای end-to-end طراحی کنید. خطاها را دسته‌بندی می‌کنم (اعتبارسنجی، دسترسی، وابستگی بیرونی، غیرمنتظره) و برای هرکدام رفتار مشخص تعریف می‌کنم؛ Error Boundary ها را لایه‌ای می‌گذارم تا شعاع خرابی محدود بماند و `global-error` آخرین تور نجات باشد. هر خطا با یک شناسه (`digest` / correlation id) به سرویس مانیتورینگ می‌رود تا قابل‌ردیابی باشد، نرخ خطا مانیتور و روی آستانه هشدار داده می‌شود، و پیام کاربر همیشه عمومی و بدون نشت جزئیات است. هدف: خرابی‌ها زودتر از کاربر، توسط تیم دیده شوند.

30 تست‌نویسی

تست، شبکه ایمنی شما هنگام تغییر کد است. اینجا سه سطح تست و ابزار مناسب هرکدام را در Next.js می‌بینیم.

هرم تست

سطح	چه می‌سنجد	ابزار پیشنهادی
واحد (Unit)	توابع و منطق خالص	Vitest
کامپوننت	رندر و تعامل UI	Testing Library
سرتاسری (E2E)	جریان واقعی کاربر در مرورگر	Playwright

تست واحد با Vitest

```
src/lib/slug.test.ts

import { describe, it, expect } from "vitest";
import { toSlug } from "../slug";

describe("toSlug", () => {
  it("converts spaces to dashes", () => {
    // ✏️ pure function → easiest, fastest tests
    expect(toSlug("hello world")).toBe("hello-world");
  });
});
```

تست کامپوننت با Testing Library

کامپوننت را رندر کنید و رفتارش را از دید کاربر بسنجید (نه جزئیات پیاده‌سازی):

src/components/counter.test.tsx

```
import { render, screen, fireEvent } from "@testing-library/react";
import { Counter } from "../counter";

it("increments on click", () => {
  render(<Counter />);
  const btn = screen.getByRole("button");
  fireEvent.click(btn);
  // 🐛 assert what the user actually sees
  expect(btn).toHaveTextContent("1");
});
```

تست سرتاسری با Playwright

برای اطمینان از کارکرد جریان‌های حیاتی (ورود، ثبت نوبت) در یک مرورگر واقعی:

e2e/booking.spec.ts

```
import { test, expect } from "@playwright/test";

test("user can book an appointment", async ({ page }) => {
  await page.goto("/book");
  await page.fill('input[name="patient"]', "علی رضایی");
  await page.fill('input[name="phone"]', "09120000000");
  await page.click('button[type="submit"]');
  // ✅ verify the success state appears
  await expect(page.getByText("نوبت ثبت شد")).toBeVisible();
});
```

تست Server Component ها

Server Component های async را بهتر است با تست E2E (Playwright) پوشش دهید، چون رفتار واقعی واکنشی داده و رندر سروری را می‌سنجند. منطق خالص (توابع کمکی، اعتبارسنجی، تبدیل داده) را با تست واحد جدا کنید — این‌ها سریع‌ترین و باارزش‌ترین تست‌هایند.

از کجا شروع کنم؟

اگر وقت محدودی دارید، با چند تست E2E روی «مسیرهای پولساز» (ورود، خرید، ثبت فرم اصلی) شروع کنید؛ بیشترین اطمینان را با کمترین تعداد تست می‌دهند. سپس منطق پیچیده را با تست واحد پوشش دهید.

سؤالات مصاحبه (با پاسخ)

Mid – چه چیزی را unit و چه چیزی را e2e تست می‌کنید؟ منطق خالص، توابع، hookها و مرزهای مهم را unit؛ و جریان‌های کاربر حیات مثل ثبت‌نام یا پرداخت را e2e. از تست‌کردن جزئیات پیاده‌سازی پرهیز می‌کنم و رفتار قابل‌مشاهده را تست می‌کنم.

Senior – تست Server Components و Server Actions چه چالشی دارد؟ Server Component async را می‌توان مستقیم فراخواند و خروجی‌اش را بررسی کرد یا با e2e پوشش داد؛ Server Actionها را مثل توابع سرور unit تست می‌کنم (اعتبارسنجی، بررسی دسترسی، نتیجه) و جریان کامل فرم را e2e. وابستگی دیتابیس را با یک دیتابیس تست یا تست‌دوئل جدا می‌کنم.

Senior – زمان محدود است؛ کجا تست بگذارید تا بیشترین اطمینان با کمترین هزینه باشد؟ روی مرزهای پریسک و پرتکرار تمرکز می‌کنم: اعتبارسنجی ورودی، منطق پول و دسترسی، و یکی‌دو مسیر e2e طلایی. به‌جای هدف پوشش ۱۰۰٪، پوشش هدفمند ریسک ارزش واقعی می‌دهد.

Senior – تست‌های شما گاهی سبز و گاهی قرمزند (flaky) و به CI اعتماد از بین رفته؛ چطور پایدارشان می‌کنید؟ علل رایج را حذف می‌کنم: انتظارهای مبتنی بر زمان ثابت را با انتظار برای وضعیت (await for state) جایگزین می‌کنم، وابستگی به شبکه/زمان/ترتیب اجرا را با تست‌دوئل و داده‌های ایزوله قطع می‌کنم، و هر تست را مستقل و بدون state مشترک می‌سازم. تست flaky بدتر از نبود تست است چون اعتماد را از بین می‌برد؛ پس آن را یا پایدار می‌کنم یا حذف.

Staff – برای یک تیم بزرگ، یک استراتژی تست مقیاس‌پذیر تعریف کنید که هم سریع باشد و هم مانع رگرسیون شود. هرم تست را به‌عنوان سیاست تثبیت می‌کنم: پایه عریضی از unitهای سریع، لایه میانی integration برای مرزهای واقعی (اکشن‌ها، لایه داده با دیتابیس تست)، و قله باریکی از e2e فقط برای جریان‌های حیاتی درآمدزا. این‌ها را در CI با اجرای موازی و سریع نگه می‌دارم، پوشش را روی «کد پریسک» هدف‌گذاری می‌کنم نه عدد کلی، و تست را بخشی از تعریف «انجام‌شده» می‌کنم تا کیفیت به فرایند گره بخورد، نه به افراد.

31 استقرار حرفه‌ای و Self-Hosting (Deploying)

از بیلد Production تا انتشار: روی Vercel با صفر پیکربندی، یا میزبانی شخصی با خروجی standalone و Docker. این فصل همه مسیرها را پوشش می‌دهد.

بیلد و اجرای Production

صرف‌نظر از مقصد، اول یک بیلد بهینه بسازید و آن را به‌صورت محلی تست کنید تا مطمئن شوید همه‌چیز کار می‌کند:

```
Terminal

npm run build # 🛠 optimized production build (Turbopack)
npm run start # ✅ run that build locally before deploying
```

گزینه ۱ – Vercel (صفر پیکربندی)

ساده‌ترین مسیر، Vercel (سازنده Next.js) است: مخزن Git را وصل می‌کنید و هر `push` به‌صورت خودکار بیلد و دیپلوی می‌شود؛ Preview برای هر Pull Request و Production برای شاخه اصلی.

دام رایج: متغیرهای محیطی Production

متغیرها را علاوه بر `.env.local` حتماً در داشبورد (Settings → Environment Variables) Vercel هم تعریف کنید. مراقب باشید `NEXT_PUBLIC_BASE_URL` در Production روی `localhost` نماند؛ این یکی از رایج‌ترین دلایل خراب‌شدن لینک‌های sitemap و متادیتای سئو پس از دیپلوی است.

گزینه ۲ – خروجی standalone و Docker

برای میزبانی شخصی، حالت `standalone` یک پوشه مستقل و سبک می‌سازد که فقط فایل‌های لازم برای اجرا (و نه کل `node_modules`) را دارد؛ ایده‌آل برای ساخت image کوچک Docker:

next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  output: "standalone", // 📦 emit a minimal, self-contained server build
};

export default nextConfig;
```

پس از `build`، خروجی در `.next/standalone` قرار می‌گیرد و با `node server.js` اجرا می‌شود. یک Dockerfile چند مرحله‌ای استاندارد:

Dockerfile

```
# 1) Build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build          # 📦 needs output: "standalone"

# 2) Minimal runtime image
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
# 📦 copy only what the standalone server needs
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
EXPOSE 3000
CMD ["node", "server.js"]  # 🚀 start the standalone server
```

متغیرهای محیطی: زمان بیلد در برابر زمان اجرا

یک نکته کلیدی هنگام میزبانی شخصی و Docker: متغیرهای `NEXT_PUBLIC_*` در زمان بیلد داخل کد کلاینت جاسازی می‌شوند، اما متغیرهای فقط-سروری در زمان اجرا خوانده می‌شوند. پس مقادیر عمومی باید هنگام `build` موجود باشند، و مقادیر سروری را می‌توانید هنگام اجرای کانینر تزریق کنید:

run the container with runtime env

```
docker build -t my-app .
# 🗝 server-only secrets injected at runtime
docker run -p 3000:3000 \
  -e DATABASE_URL="..." \
  -e JWT_SECRET="..." \
  my-app
```

گزینه ۳ – خروجی کاملاً استاتیک (Static Export)

اگر سایت شما هیچ قابلیت سمت سرور پویا (Route Handler، SSR، پویا، ISR) ندارد و فقط محتوای استاتیک است، می‌توانید آن را به HTML خالص اکسپورت کنید و روی هر میزبان استاتیک (مثل GitHub Pages یا یک CDN) بگذارید:

next.config.ts

```
const nextConfig = {
  output: "export", // 📄 produces static HTML in the `out/` folder
};
export default nextConfig;
```

محدودیت Static Export

در حالت `export` قابلیت‌های وابسته به سرور کار نمی‌کنند: Route Handler، Server Actions، پویا، `cookies()` / `headers()` و ISR. فقط برای سایت‌های واقعاً استاتیک مناسب است.

کش و ISR در میزبانی شخصی

هنگام self-host، کش و بازتولید تدریجی (ISR) به صورت پیش فرض روی فایل سیستم همان نمونه ذخیره می‌شود. اگر چند نمونه (instance) پشت یک load balancer دارید، باید یک **Cache Handler** مشترک (مثلاً مبتنی بر Redis) تنظیم کنید تا کش بین نمونه‌ها هماهنگ بماند – جزئیات در فصل «تسلط بر کشینگ».

چک‌لیست قبل از Production

(۱) `tsc --noEmit` بدون خطا. (۲) `next build` موفق و تست محلی با `next start`. (۳) همه متغیرهای محیطی Production تنظیم شده (به‌خصوص `NEXT_PUBLIC_BASE_URL` و اتصال دیتابیس). (۴) یک اندپوینت سلامت مثل `/api/health` برای مانیتورینگ. (۵) Node نسخه ۲۰.۹ یا بالاتر روی سرور.

سؤالات مصاحبه (با پاسخ)

Mid – تفاوت دیپلوی روی Vercel و Self-Host چیست و چه چیزهایی را باید مدیریت کنید؟ Vercel بیشتر زیرساخت (کش، CDN، توابع) را خودکار می‌کند؛ در Self-Host خودتان مسئول خروجی `output: "standalone"`، مدیریت کش، CDN، مقیاس‌پذیری و متغیرهای محیطی هستید.

Senior – در Self-Host، کش ISR / `use cache` چطور کار می‌کند و چه باید کرد؟ به‌طور پیش‌فرض کش روی فایل‌سیستم همان instance ذخیره می‌شود؛ در محیط چند-instance باید یک Cache Handler سفارشی (مثلاً Redis) تعریف کنید تا کش بین instance‌ها به‌اشتراک گذاشته و باطل‌سازی به‌درستی منتشر شود، وگرنه کاربران نسخه‌های ناهم‌هنگ می‌بینند.

Senior – بعد از دیپلوی، یک متغیر محیطی جدید در مرورگر `undefined` است؛ چرا؟ فقط متغیرهای با پیشوند `NEXT_PUBLIC_` به کلاینت می‌رسند و در زمان `build` درون باندل تزریق می‌شوند؛ پس بعد از افزودن باید دوباره `build` کرد. ضمناً اسرار سروری هرگز نباید `NEXT_PUBLIC_` شوند تا لو نروند.

Senior – بعد از دیپلوی نسخه جدید، کاربران قدیمی با خطای «چانک بارگذاری نشد» مواجه می‌شوند؛ علت و راهکار؟ علت این است که کاربر HTML نسخه قبل را باز دارد اما فایل‌های `chunk` با هش جدید جایگزین شده‌اند و نسخه قدیمی دیگر روی سرور نیست. راهکار: نگه‌داشتن دارایی‌های نسخه قبل برای مدتی (عدم حذف فوری)، سرو دارایی‌ها از CDN با کش پایدار، و مدیریت خطای بارگذاری چانک با یک `reload` کنترل‌شده تا کاربر به نسخه تازه منتقل شود.

Staff – مسئول انتقال یک اپلیکیشن پرتراфик از یک پلتفرم مدیریت‌شده به Self-Host هستید؛ چه چیزهایی را باید پوشش دهید تا پایداری Production حفظ شود؟ ابتدا خروجی `output: "standalone"` و یک ایمیج قابل‌تکرار می‌سازم، سپس لایه‌هایی که پلتفرم قبلی خودکار می‌داد را صریحاً بازطراحی می‌کنم: یک Cache Handler مشترک (Redis) برای ISR / `use cache` در محیط چند-CDN، برای دارایی‌ها و بهینه‌سازی تصویر (با `sharp` یا `loader` بیرونی)، `health-check` و `graceful shutdown` برای استقرار بدون قطعی، مدیریت اسرار و متغیرهای محیطی، و مانیتورینگ/لاگ متمرکز. استقرار را ابتدا کنار سیستم قدیم (canary) اجرا می‌کنم و با معیارهای میدانی می‌سنجم تا انتقال بدون رگرسیون باشد.

32 مهاجرت و ارتقا به نسخه ۱۶ (Upgrading)

اگر پروژه‌ای روی نسخه ۱۴ یا ۱۵ دارید، این فصل مسیر امن ارتقا به ۱۶ را نشان می‌دهد: ابزار خودکار، فهرست تغییرات شکننده، و راهبرد مهاجرت تدریجی.

ارتقای خودکار با codemod

Next.js یک ابزار رسمی دارد که بسیاری از تغییرات لازم را خودکار اعمال می‌کند (تغییر نام‌ها، async کردن API‌ها و ...). ابتدا همین را اجرا کنید:

```
Terminal

# 🤖 official upgrade tool: bumps versions + applies codemods
npx @next/codemod@latest upgrade latest

# then verify nothing broke
npx tsc --noEmit # ✅ type-check
npm run build    # 🏗️ full production build
```

کانال‌های نسخه

latest نسخه پایدار است (پیشنهاد برای Production). اگر می‌خواهید جدیدترین قابلیت‌های در حال آزمایش را امتحان کنید، نسخه **canary** را نصب کنید: `npm i next@canary`. روی Production هرگز **canary** نگذارید.

فهرست تغییرات شکننده ۱۵ → ۱۶

codemod بیشتر این‌ها را خودکار انجام می‌دهد، اما باید بدانید چه چیزی تغییر کرده تا موارد دستی را بررسی کنید:

تغییر	اقدام لازم
+Node 20.9	نسخه Node را بالا ببرید؛ روی ۱۸ اجرا نمی‌شود.
async های API	<code>headers()</code> ، <code>cookies()</code> ، <code>searchParams</code> ، <code>params</code> را <code>await</code> کنید.
middleware → proxy	فایل <code>middleware.ts</code> را به <code>proxy.ts</code> و تابع <code>middleware</code> را به <code>proxy</code> تغییر دهید.
مدل کشینگ	کش خودکار حذف شد؛ همه چیز پویاست مگر اینکه <code>cacheComponents</code> را روشن و <code>use cache</code> بگذارید.
Turbopack پیش فرض	اگر پیکربندی سفارشی webpack داشتید، باید معادل Turbopack را تنظیم کنید.
حذف next lint	ESLint یا Biome را مستقیم اجرا کنید (مثلاً اسکریپت <code>"lint": "eslint ."</code>).
حذف AMP	هر کد مربوط به AMP را بردارید.
props قدیمی next/image	<code>lazyBoundary</code> ، <code>onLoadingComplete</code> و <code>lazyRoot</code> حذف شدند؛ جایگزین کنید.
React 19.2	وابستگی <code>react</code> و <code>react-dom</code> باید نسخه ۱۹.۲ باشند.

رایج ترین خطای دستی: async params

پرتکرارترین موردی که codemod ممکن است در کدهای پیچیده جا بیندازد، همین async شدن است. الگوی قبل و بعد:

```

before (v15) → after (v16)

// ❌ v15: params was a plain object
export default function Page({ params }: { params: { id: string } }) {
  return <h1>{params.id}</h1>;
}

// ✅ v16: params is a Promise — type it and await it
export default async function Page({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  return <h1>{id}</h1>;
}

```

راهنمای مهاجرت امن

- ♦ یک شاخه جدا بسازید و روی همان مهاجرت کنید، نه روی شاخه اصلی.
- ♦ اول `codemod` را اجرا کنید، سپس `tsc --noEmit`؛ خطاهای تایپ، دقیقاً جاهایی را که `async` نشده‌اند نشان می‌دهند.
- ♦ بعد از سبز شدن تایپ‌ها، یک `build` کامل بگیرید و مسیرهای کلیدی (ورود، فرم‌ها، صفحات پویا) را دستی تست کنید.
- ♦ اگر روی مدل کشینگ جدید گیر کردید، ابتدا بدون `cacheComponents` اجرا کنید (همه چیز پویا، درست اما کندتر)، و بعد به تدریج `use cache` اضافه کنید.
- ♦ قبل از `merge`، روی یک محیط `staging` دیپلوی و دود-تست کنید.

جمع‌بندی

مهاجرت به ۱۶ معمولاً مکانیکی است: `codemod` + `async` کردن API‌ها + تغییر نام `proxy`. سخت‌ترین بخش، مفهومی مدل کشینگ جدید است که در فصل‌های ۸ و ۲۲ کامل پوشش داده شده. با مهاجرت تدریجی و تکیه بر `tsc`، این کار کم‌ریسک است.

33 نکات و ترفندهای طلایی

این فصل، مجموعه‌ای از تکنیک‌های کمترگفته‌شده و حرفه‌ای است؛ همان چیزهایی که فاصله یک پروژه معمولی با یک پروژه سریع، امن و حرفه‌ای را می‌سازند.

۱ | به‌روزرسانی خوش‌بینانه با `useOptimistic`

وقتی کاربر روی «لایک» یا «افزودن» می‌زند، نباید تا پایان درخواست سرور منتظر بماند. هوک `useOptimistic` فوراً UI را به‌روز نشان می‌دهد و اگر سرور خطا داد، خودکار به حالت قبل برمی‌گردد:

```
src/components/like-button.tsx

"use client";
import { useOptimistic, startTransition } from "react";
import { toggleLike } from "@app/actions";

export function LikeButton({ postId, likes }: { postId: string; likes: number }) {
  // ✨ optimistic = what the user sees instantly
  const [optimistic, addOptimistic] = useOptimistic(likes, (cur, d: number) => cur + d);

  return (
    <button
      onClick={() => {
        startTransition(async () => {
          addOptimistic(1); // ⚡ UI updates immediately
          await toggleLike(postId); // 🔄 reverts automatically on error
        });
      }}
    >
      ❤️ {optimistic}
    </button>
  );
}
```

۲ | کار سنگین بعد از پاسخ با `after()`

گاهی می‌خواهید کاری انجام دهید (مثل لاگ، آنالیتیکس، ارسال ایمیل) که نباید پاسخ کاربر را کند کند. تابع `after()` آن کار را به بعد از ارسال پاسخ موکول می‌کند:

src/app/actions.ts

```
"use server";
import { after } from "next/server";

export async function trackView(postId: string) {
  // 🚀 Respond to the user right away...
  after(async () => {
    // ⌚ ...then run this in the background, off the critical path
    await analytics.record({ type: "view", postId });
  });
}
```

۳ | حذف درخواست‌های تکراری با React.cache

اگر یک تابع واکنشی در چند جای درخت کامپوننت‌ها (مثلاً هم در `page` و هم در `generateMetadata`) صدا زده شود، با `cache` از React در طول یک درخواست فقط یک‌بار اجرا می‌شود:

src/lib/get-doctor.ts

```
import { cache } from "react";

// 🧠 De-duped within a single request: call it anywhere, runs once
export const getDoctor = cache(async (id: string) => {
  return db.doctor.findUnique({ where: { id } });
});
```

تفاوت با use cache

`React.cache` فقط در طول همان درخواست نتیجه را به‌خاطر می‌سپارد (memoization)، اما `use cache` نتیجه را بین درخواست‌ها و کاربران مختلف کش می‌کند. اولی برای جلوگیری از کوئری تکراری، دومی برای کارایی بلندمدت.

۴ | الگوی درست اتصال دیتابیس (Hot Reload)

یک باگ کلاسیک: در حالت توسعه، هر بار که فایل تغییر می‌کند یک اتصال جدید به دیتابیس ساخته می‌شود و خیلی زود به سقف اتصال می‌رسید. راه‌حل، نگه‌داشتن اتصال روی `globalThis` است:

src/lib/db.ts

```
import mongoose from "mongoose";

// 🌐 Reuse one connection across hot reloads in development
const globalForDb = globalThis as unknown as { conn?: typeof mongoose };

export async function connectDB() {
  if (globalForDb.conn) return globalForDb.conn; // ♻️ reuse existing
  const conn = await mongoose.connect(process.env.DATABASE_URL!);
  if (process.env.NODE_ENV !== "production") globalForDb.conn = conn;
  return conn;
}
```

۵ | محافظت از کد سرور با server-only

برای اینکه به اشتباه یک ماژول حاوی اسرار به باندل کلاینت نشت نکند، پکیج کوچک `server-only` را در بالای آن import کنید؛ اگر کسی آن را در Client Component وارد کند، زمان **build** خطا می‌دهد:

src/lib/secrets.ts

```
import "server-only"; // 🛡️ build fails if imported into client code

export const apiKey = process.env.PAYMENT_SECRET!;
```

۶ | مسیرهای تایپ‌شده (Typed Routes)

با فعال‌کردن این گزینه، اگر در `href` یک آدرس اشتباه بنویسید، TypeScript همان لحظه خطا می‌دهد؛ دیگر لینک خراب در Production نخواهید داشت:

next.config.ts

```
const nextConfig = {
  typedRoutes: true, // 🔗 type-checked <Link href> across the whole app
};

export default nextConfig;
```

۷ | state در URL به جای useState

برای فیلتر، جستجو و صفحه‌بندی، state را در `searchParams` نگه دارید نه در حافظه کامپوننت. مزیت‌ها: قابل‌اشتراک‌گذاری بودن لینک، بازگشت مرورگر، و کارکرد با Server Component. در کلاینت با `useRouter` و `usePathname` آن را به‌روز کنید:

```
update URL state without reload

"use client";
import { usePathname, useRouter, useSearchParams } from "next/navigation";

export function SortSelect() {
  const router = useRouter();
  const pathname = usePathname();
  const params = useSearchParams();

  function setSort(value: string) {
    const next = new URLSearchParams(params);
    next.set("sort", value);
    // 🔗 shallow update: server re-fetches, no full reload
    router.push(`${pathname}?${next.toString()}`);
  }

  return (
    <select onChange={(e) => setSort(e.target.value)}>
      <option value="newest">جدیدترین</option>
      <option value="rating">بیشترین امتیاز</option>
    </select>
  );
}
```

۸ | بارگذاری تتبل کامپوننت‌های سنگین

کامپوننت‌های سنگینی که فوراً لازم نیستند (نمودار، ویرایشگر متن، نقشه) را با `next/dynamic` جدا کنید تا از باندل اولیه خارج شوند:

```
lazy-load a heavy client component

import dynamic from "next/dynamic";

// 📊 Chart code is fetched only when actually rendered
```

```
const Chart = dynamic(() => import("@/components/chart"), {
  loading: () => <p>در حال آماده‌سازی نمودار</p>,
});
```

۹ | placeholder محو برای تصاویر (بدون پرش)

با `placeholder="blur"` تا قبل از بارگذاری کامل، یک نسخه محو نمایش داده می‌شود و تجربه روان‌تر می‌شود. برای تصاویر import شده، blur به‌صورت خودکار ساخته می‌شود:

```
import Image from "next/image";
import hero from "@/../public/hero.jpg"; //  static import enables auto-blur

export function Hero() {
  return <Image src={hero} alt="" placeholder="blur" priority />;
}
```

۱۰ | خطای سراسری با global-error.tsx

فقط خطای همان سگمنت را می‌گیرد. برای گرفتن خطایی که حتی در **Root Layout** رخ دهد، فایل `error.tsx` بسازید (باید تگ‌های `html` و `body` خودش را داشته باشد):

```
src/app/global-error.tsx

"use client";

export default function GlobalError({ reset }: { reset: () => void }) {
  return (
    <html lang="fa" dir="rtl">
      <body>
        <h2>خطای غیرمنتظره‌ای رخ داد</h2>
        <button onClick={() => reset()}>تلاش دوباره</button>
      </body>
    </html>
  );
}
```

۱۱ | ساخت استاتیک مسیرهای پویا با generateStaticParams

اگر آدرس‌های پویا (مثل صفحه هر تخصص) را از قبل می‌شناسید، با `generateStaticParams` آن‌ها را در زمان build بسازید تا فوق‌العاده سریع سرو شوند:

```
src/app/specialties/[slug]/page.tsx

// 🏠 Pre-render these pages at build time (SSG for dynamic routes)
export async function generateStaticParams() {
  const items = await getSpecialties();
  return items.map((s) => ({ slug: s.slug }));
}
```

۱۲ | ترفندهای کوچک ولی پرتکرار

نیاز	ترفند طلایی
ادغام کلاس‌های شرطی Tailwind	یک تابع <code>cn()</code> با <code>tailwind-merge</code> + <code>clsx</code> بسازید تا کلاس‌های متضاد درست merge شوند.
وضعیت «در حال ارسال» فرم	در دکمه داخل فرم از هوک <code>useFormStatus</code> استفاده کنید؛ بدون نیاز به prop دادن دستی.
ریدایرکت دائمی (سئو)	برای ۳۰۱ از <code>permanentRedirect()</code> به جای <code>redirect()</code> استفاده کنید.
آدرس مطلق در متادیتا	در <code>metadataBase</code> ، Root را ست کنید تا OG image و canonical درست بسازند.
تحلیل حجم باندل	با <code>@next/bundle-analyzer</code> ببینید کدام پکیج باندل را سنگین کرده.
اعتبارسنجی متغیرهای محیطی	هنگام بالا آمدن اپ، <code>process.env</code> را با zod اعتبارسنجی کنید تا خطاها زود لو بروند.

یک قاعده ذهنی برای کارایی

سه سؤال طلایی پیش از هر کامپوننت بپرسید: آیا واقعاً باید Client باشد؟ آیا این داده باید کش شود؟ آیا این بخش باید رندر کل صفحه را بلاک کند یا می‌تواند استریم شود؟ پاسخ آگاهانه به همین سه سؤال، بیشتر مشکلات کارایی را حل می‌کند.

34 کارگاه مینی پروژه‌ها

هشت پروژه کوچک اما کامل. هر پروژه یک آموزش مستقل «صفر تا صد» است: ابتدا می‌بینید چه می‌سازید و چه فایل‌هایی لازم است، سپس گام به گام آن را می‌سازید و در پایان نتیجه را می‌فهمید.

پیش‌نیاز مشترک

فرض می‌کنیم یک پروژه Next.js 16 (طبق فصل ۲) دارید. در مثال‌ها `db` همان ابزار اتصال دیتابیس از فصل «ترفندهای طلایی» است. مسیر هر فایل بالای بلوک کد نوشته شده؛ کافی است همان ساختار را بسازید و کد را در جای درست قرار دهید.

پروژه ۱ – تم تاریک/روشن بدون پرش

چه می‌سازیم؟ یک سوئیچ تم که انتخاب کاربر را در کوکی نگه می‌دارد و حتی در همان اولین بارگذاری صفحه، «پرش رنگ» (flash of wrong theme) ندارد؛ چون تم روی سرور خوانده و اعمال می‌شود.

مفاهیم کلیدی: `cookies()` `Server Action` `router.refresh`

گام ۰ – ساختاری که می‌سازید

```
src/
├─ app/
│  ├─ layout.tsx      # reads the theme cookie on the server
│  └─ theme-actions.ts # server action to persist the choice
└─ components/
   └─ theme-toggle.tsx # the client button
```

گام ۱ – اکشن ذخیره تم

یک `Server Action` که مقدار تم را در کوکی می‌نویسد. چون فقط روی سرور اجرا می‌شود، امن و ساده است:

src/app/theme-actions.ts

```
"use server";
import { cookies } from "next/headers";

export async function setTheme(theme: "light" | "dark") {
  const store = await cookies(); // ⌚ cookies() is async in v16
  // 🍪 keep the choice for a year
  store.set("theme", theme, { path: "/", maxAge: 60 * 60 * 24 * 365 });
}
```

گام ۲ – خواندن تم در Root Layout

چون layout یک Server Component است، تم را پیش از ارسال HTML می‌خوانیم و روی `<html>` می‌گذاریم. به همین دلیل هیچ پرشی رخ نمی‌دهد:

src/app/layout.tsx

```
import { cookies } from "next/headers";
import { ThemeToggle } from "@components/theme-toggle";
import "./globals.css";

export default async function RootLayout({ children }: { children: React.ReactNode }) {
  const theme = ((await cookies()).get("theme")?.value ?? "light") as "light" | "dark";
  // 🍪 theme is known on the server → correct colors on first paint
  return (
    <html lang="fa" dir="rtl" data-theme={theme}>
      <body>
        <header className="p-4"><ThemeToggle current={theme} /></header>
        {children}
      </body>
    </html>
  );
}
```

گام ۳ – دکمه سوئیچ (Client)

دکمه اکشن را صدا می‌زند و سپس با `router.refresh()` درخت سرور را با کوکی جدید دوباره می‌سازد:

src/components/theme-toggle.tsx

```
"use client";
import { setTheme } from "@app/theme-actions";
import { useRouter } from "next/navigation";

export function ThemeToggle({ current }: { current: "light" | "dark" }) {
  const router = useRouter();
  return (
    <button
      onClick={async () => {
        await setTheme(current === "dark" ? "light" : "dark");
        router.refresh(); // ↺ re-render the server tree with the new cookie
      }}
      className="rounded border px-3 py-1"
    >
      {current === "dark" ? "🌙 تاریک" : "☀ روشن"}
    </button>
  );
}
```

گام ۴ – رنگ‌ها در CSS

src/app/globals.css

```
@import "tailwindcss";
/* 🌙 colors react to the data-theme attribute on <html> */
:root { --bg: #ffffff; --fg: #0f172a; }
[data-theme="dark"] { --bg: #0b1220; --fg: #e2e8f0; }
body { background: var(--bg); color: var(--fg); }
```

نتیجه

کاربر روی دکمه می‌زند، تم عوض می‌شود و در کوکی می‌ماند. حالا اگر صفحه را رفرش کند یا فردا برگردد، صفحه از همان اولین رنگ‌آمیزی با تم درست بالا می‌آید؛ بدون فلش سفید. این دقیقاً همان چیزی است که با تم‌های مبتنی بر `localStorage` به‌سختی به‌دست می‌آید.

پروژه ۲ – لیست کارها با بهروزرسانی خوش بینانه

چه می‌سازیم؟ یک Todo که وقتی آیتم اضافه می‌کنید، فوراً در فهرست ظاهر می‌شود (بدون انتظار برای سرور) و در پس‌زمینه ذخیره و همگام می‌شود.

مفاهیم کلیدی: `revalidatePath` `useOptimistic` `Server Action`

گام ۰ – ساختاری که می‌سازید

```
structure

src/app/todos/
├─ page.tsx      # Server Component: fetches initial todos
├─ actions.ts    # server action to create a todo
└─ todo-list.tsx # Client Component: optimistic UI
```

گام ۱ – اکشن افزودن

```
src/app/todos/actions.ts

"use server";
import { revalidatePath } from "next/cache";
import { db } from "@lib/db";

export async function addTodo(title: string) {
  await db.todo.create({ data: { title, done: false } });
  revalidatePath("/todos"); // 🔄 sync the real list after the optimistic update
}
```

گام ۲ – صفحه (واکشی اولیه روی سرور)

صفحه یک Server Component است که فهرست اولیه را می‌خواند و به کامپوننت کلاینت می‌سپارد:

```
src/app/todos/page.tsx

import { db } from "@lib/db";
import { TodoList } from "../todo-list";

export default async function TodosPage() {
  const initial = await db.todo.findMany({ orderBy: { createdAt: "desc" } });
  return (
```

```

<main className="mx-auto max-w-md p-6">
  <h1 className="mb-4 text-xl font-bold">کارهای من</h1>
  <TodoList initial={initial} />
</main>
);
}

```

گام ۳ – UI خوش بینانه (Client)

هسته پروژه: `useOptimistic` یک نسخه «موقت» از فهرست می‌سازد که آیتم جدید را بلافاصله نشان می‌دهد؛ پس از اتمام اکشن و `revalidatePath`، فهرست واقعی جای آن را می‌گیرد:

```

src/app/todos/todo-list.tsx

"use client";
import { useOptimistic, useState, startTransition } from "react";
import { addTodo } from "../actions";

type Todo = { id: string; title: string };

export function TodoList({ initial }: { initial: Todo[] }) {
  const [text, setText] = useState("");
  const [items, addOptimistic] = useOptimistic(
    initial,
    (cur, t: Todo) => [t, ...cur] // ✨ show the new item immediately, on top
  );

  function submit() {
    const title = text.trim();
    if (!title) return;
    startTransition(async () => {
      addOptimistic({ id: "temp-" + Date.now(), title }); // ⚡ instant
      await addTodo(title); // 💾 persist + revalidate
    });
    setText("");
  }

  return (
    <div>
      <div className="flex gap-2">
        <input
          value={text}
          onChange={(e) => setText(e.target.value)}
          onKeyDown={(e) => e.key === "Enter" && submit()}

```

```

        placeholder="یک کار جدید"
        className="flex-1 rounded border p-2"
      />
      <button onClick={submit} className="rounded bg-blue-600 px-4 text-
white">افزودن</button>
    </div>
    <ul className="mt-4 space-y-1">
      {items.map((t) => <li key={t.id} className="rounded border p-2">• {t.title}</li>)}
    </ul>
  </div>
);
}

```

نتیجه

هنگام افزودن، آیتم بدون هیچ تأخیری بالای فهرست ظاهر می‌شود؛ کاربر حس «آنی بودن» می‌گیرد. اگر ذخیره در سرور شکست بخورد، React به‌صورت خودکار آیتم خوش‌بینانه را برمی‌گرداند. این الگو پایه تجربه کاربری حرفه‌ای در اپ‌های مدرن است.

۱ پروژه ۳ – جستجوی URL-محور با Suspense

چه می‌سازیم؟ صفحه جستجویی که عبارت جستجو را در آدرس (`?q=...`) نگه می‌دارد؛ لینک قابل‌اشتراک است، دکمه Back مرورگر کار می‌کند، و نتایج روی سرور با حالت بارگذاری استریم می‌شوند.

مفاهیم کلیدی: `Server render` `Suspense` `searchParams`

گام ۰ – ساختاری که می‌سازید

```

structure

src/app/search/
└─ page.tsx # everything lives here: form + streamed results

```

گام ۱ – صفحه جستجو

عبارت جستجو از `searchParams` (که در v16 یک Promise است) خوانده می‌شود. فرم با ارسال ساده، مقدار را در URL می‌گذارد؛ بخش نتایج داخل `<Suspense>` با `key` قرار می‌گیرد:

```

import { Suspense } from "react";
import { db } from "@lib/db";

// 🕒 The slow part: fetch results on the server
async function Results({ q }: { q: string }) {
  if (!q) return <p className="text-gray-500">عبارتی برای جستجو وارد کنید.</p>;
  const doctors = await db.doctor.findMany({
    where: { name: { contains: q, mode: "insensitive" } },
    take: 20,
  });
  if (doctors.length === 0) return <p>نتیجه‌ای یافت نشد.</p>;
  return <ul className="space-y-1">{doctors.map((d) => <li key={d.id}>{d.name}</li>)}</ul>;
}

export default async function SearchPage({
  searchParams,
}: {
  searchParams: Promise<{ q?: string }>;
}) {
  const { q = "" } = await searchParams; // ⌚ async searchParams in v16
  return (
    <main className="mx-auto max-w-lg p-6">
      </* a plain form puts ?q=... into the URL on submit */>
      <form className="mb-4 flex gap-2">
        <input name="q" defaultValue={q} placeholder="نام پزشک" className="flex-1 rounded border p-2" />
        <button className="rounded bg-blue-600 px-4 text-white">جستجو</button>
      </form>
      </* 🔑 key={q} restarts the fallback on every new query */>
      <Suspense key={q} fallback={<p>در حال جستجو...</p>}>
        <Results q={q} />
      </Suspense>
    </main>
  );
}

```

نتیجه و درک عمیق

چون state در URL است، نتیجه جستجو قابل اشتراک و قابل بوک‌مارک است و دکمه Back کار می‌کند. کلید `key={q}` باعث می‌شود با هر جستجوی جدید، حالت «در حال جستجو...» دوباره دیده شود؛ بدون آن، نتایج قبلی تا آماده شدن نتایج تازه روی صفحه می‌مانند و کاربر فکر می‌کند چیزی کار نمی‌کند.

پروژه ۴ – صفحه‌بندی سروری

چه می‌سازیم؟ یک فهرست بزرگ که به‌صورت صفحه‌بندی‌شده و کاملاً روی سرور بارگذاری می‌شود؛ شماره صفحه در URL است و ناوبری با لینک انجام می‌شود (بدون JavaScript اضافه).

مفاهیم کلیدی: `Link` `skip/take` `searchParams`

گام ۰ – ساختاری که می‌سازید

structure

```
src/app/doctors/  
└─ page.tsx  # reads ?page=N, queries a slice, renders nav links
```

گام ۱ – صفحه فهرست با صفحه‌بندی

src/app/doctors/page.tsx

```
import Link from "next/link";  
import { db } from "@lib/db";  
  
const PER_PAGE = 10;  
  
export default async function DoctorsPage({  
  searchParams,  
}: {  
  searchParams: Promise<{ page?: string }>;  
}) {  
  const page = Math.max(1, Number((await searchParams).page ?? "1"));  
  
  // 📄 fetch one slice + the total count in parallel  
  const [doctors, total] = await Promise.all([  
    db.doctor.findMany({ skip: (page - 1) * PER_PAGE, take: PER_PAGE, orderBy: { name:  
"asc" } })),  
    db.doctor.count(),  
  ]);  
  const lastPage = Math.ceil(total / PER_PAGE);  
  
  return (  
    <main className="mx-auto max-w-lg p-6">  
      <ul className="space-y-1">{doctors.map((d) => <li key={d.id}>{d.name}</li>)}</ul>  
      <nav className="mt-4 flex items-center justify-between">  
        {page > 1
```

```

      ? <Link href={`/${doctors}?page=${page - 1}`} className="text-blue-600">قبلی</Link>
      : <span />
    <span className="text-sm text-gray-500">صفحه {page} از {lastPage}</span>
    {page < lastPage
      ? <Link href={`/${doctors}?page=${page + 1}`} className="text-blue-600">بعدی</Link>
      : <span />
    }
  </nav>
</main>
);
}

```

نتیجه

هر صفحه فقط داده همان صفحه را از دیتابیس می‌گیرد (نه همه را)، پس حتی با میلیون‌ها رکورد سریع می‌ماند. شماره صفحه در URL است، بنابراین قابل‌اشتراک و سازگار با دکمه Back است. دکمه‌های «قبلی/بعدی» در مرز اول و آخر به‌درستی پنهان می‌شوند.

پروژه ۵ – شمارنده بازدید با Redis و محدودسازی نرخ

چه می‌سازیم؟ یک اندپوینت که بازدید هر مقاله را می‌شمارد، اما با محدودسازی نرخ مبتنی بر IP جلوی شمارش تکراری/سوءاستفاده را می‌گیرد.

مفاهیم کلیدی: `rate limit` `Upstash Redis` `Route Handler`

گام ۰ – ساختار و پیش‌نیاز

```

src/app/api/views/[slug]/
└─ route.ts      # POST: count a view (rate-limited per IP)

# .env.local → from your Upstash dashboard
UPSTASH_REDIS_REST_URL="https://...upstash.io"
UPSTASH_REDIS_REST_TOKEN="..."

```

پکیج لازم را نصب کنید: `npm i @upstash/redis`. سپس اندپوینت را بسازید:

src/app/api/views/[slug]/route.ts

```
import { Redis } from "@upstash/redis";
import { headers } from "next/headers";

const redis = Redis.fromEnv(); // 🐛 reads UPSTASH_* env vars automatically

export async function POST(_req: Request, { params }: { params: Promise<{ slug: string }> }) {
  const { slug } = await params;
  const ip = (await headers()).get("x-forwarded-for") ?? "anon";

  // 🛡️ Rate limit: one count per IP per article per hour.
  // SET with nx (only-if-absent) + ex (expiry) returns true the first time only.
  const fresh = await redis.set(`rl:${slug}:${ip}`, "1", { nx: true, ex: 3600 });
  if (fresh) await redis.incr(`views:${slug}`); // 👁️ count only the first hit

  const total = (await redis.get<number>(`views:${slug}`)) ?? 0;
  return Response.json({ views: total });
}
```

گام ۱ – فراخوانی از سمت صفحه

در کامپوننت مقاله، هنگام نمایش یک بار بازدید را ثبت کنید:

src/components/view-counter.tsx

```
"use client";
import { useEffect, useState } from "react";

export function ViewCounter({ slug }: { slug: string }) {
  const [views, setViews] = useState<number | null>(null);
  useEffect(() => {
    // ✅ fire once on mount; server enforces the real rate limit
    fetch(`/api/views/${slug}`, { method: "POST" })
      .then((r) => r.json())
      .then((d) => setViews(d.views));
  }, [slug]);
  return <span className="text-sm text-gray-500">{views ?? "-"} بازدید</span>;
}
```

نتیجه

شمارش واقعی و مقاوم در برابر رفرش پیاپی است: هر IP در هر ساعت فقط یکبار شمرده می‌شود. منطق محدودسازی روی سرور و در Redis اعمال می‌شود، پس از سمت کلاینت قابل دور زدن نیست. همین الگو پایه rate-limit برای فرم‌ها و API‌های حساس هم هست.

پروژه ۶ – فرم چندمرحله‌ای (ویزارد)

چه می‌سازیم؟ یک فرم ثبت‌نام سه‌مرحله‌ای که هر مرحله را روی سرور اعتبارسنجی می‌کند و فقط پس از معتبر بودن، به مرحله بعد می‌رود.

مفاهیم کلیدی: `useActionState` `Server Action` `state machine`

گام ۰ – ساختاری که می‌سازید

structure

```
src/app/signup/  
├─ actions.ts  # validates + advances the step  
└─ wizard.tsx  # renders the right fields for the current step
```

گام ۱ – اکشن پیشروی مرحله

اکشن، «مرحله بعدی» را برمی‌گرداند؛ همین مقدار بازگشتی تعیین می‌کند کدام فیلدها نمایش داده شوند:

src/app/signup/actions.ts

```
"use server";  
import { db } from "@/lib/db";  
  
export type WizardState = { step: number; error?: string };  
  
export async function nextStep(prev: WizardState, formData: FormData): Promise<WizardState>  
{  
  // 🚧 validate the current step before moving forward  
  if (prev.step === 1 && !String(formData.get("name"))?.trim()) {  
    return { step: 1, error: "نام را وارد کنید" };  
  }  
  if (prev.step === 2 && !/^09\d{9}$/.test(String(formData.get("phone"))?.trim())) {  
    return { step: 2, error: "شماره موبایل معتبر نیست" };  
  }  
}
```

```

if (prev.step === 3) {
  await db.user.create({ data: { /* ... collected fields ... */ } }); // 📁 final submit
  return { step: 4 }; // 4 = done
}
return { step: prev.step + 1 }; // ➡ advance
}

```

گام ۲ – کامپوننت ویزارد

```

src/app/signup/wizard.tsx

"use client";
import { useActionState } from "react";
import { nextStep, type WizardState } from "./actions";

const initial: WizardState = { step: 1 };

export function Wizard() {
  const [state, action, pending] = useActionState(nextStep, initial);

  if (state.step === 4) return <p className="p-6">✔ ثبت نام کامل شد </p>;

  return (
    <form action={action} className="mx-auto max-w-sm space-y-3 p-6">
      <p className="text-sm text-gray-500">مرحله {state.step} از ۳</p>
      {state.step === 1 && <input name="name" placeholder="نام و نام خانوادگی" className="w-full rounded border p-2" /> }
      {state.step === 2 && <input name="phone" placeholder="شماره موبایل" className="w-full rounded border p-2" /> }
      {state.step === 3 && <p>اطلاعات را بررسی کردید؟ برای ثبت نهایی ادامه دهید</p> }
      {state.error && <p className="text-red-600">{state.error}</p> }
      <button disabled={pending} className="rounded bg-blue-600 px-4 py-2 text-white">
        {state.step < 3 ? "ثبت نهایی" : "مرحله بعد"}
      </button>
    </form>
  );
}

```

نتیجه و درک عمیق

کل «وضعیت مرحله» در همان state بازگشتی اکشن زندگی می‌کند؛ نه نیازی به مدیریت پیچیده state داریم و نه به کتابخانه بیرونی. هر مرحله روی سرور اعتبارسنجی می‌شود، پس کاربر نمی‌تواند با دستکاری UI از اعتبارسنجی فرار کند. این همان الگوی «ماشین حالت سمت سرور» است.

پروژه ۷ – آپلود آواتار

چه می‌سازیم؟ فرم آپلود تصویر که فایل را در یک Server Action دریافت، اعتبارسنجی (نوع و حجم) و ذخیره می‌کند.

مفاهیم کلیدی: `Server Action` `File API` `FormData`

گام ۰ – ساختاری که می‌سازید

```
structure
src/app/profile/
├─ upload.ts      # server action: receive + validate + store
└─ avatar-form.tsx # client form with a file input
```

گام ۱ – اکشن آپلود

فایل از `FormData` به صورت یک شیء `File` می‌آید؛ آن را به Buffer تبدیل و ذخیره می‌کنیم:

```
src/app/profile/upload.ts

"use server";
import { revalidatePath } from "next/cache";
import { storage } from "@lib/storage"; // your storage (S3, disk, etc.)

export async function uploadAvatar(_prev: unknown, formData: FormData) {
  const file = formData.get("avatar") as File | null;
  if (!file || file.size === 0) return { error: "فایلی انتخاب نشده" };

  // 🛡️ validate type + size (max 2MB) on the server
  if (!file.type.startsWith("image/")) return { error: "فقط فایل تصویری مجاز است" };
  if (file.size > 2 * 1024 * 1024) return { error: "حجم نباید از ۲ مگابایت بیشتر باشد" };

  const bytes = Buffer.from(await file.arrayBuffer()); // 📦 File → Buffer
  const url = await storage.save(`avatars/${crypto.randomUUID()}`, bytes);
```

```

revalidatePath("/profile");
return { ok: true, url };
}

```

گام ۲ – فرم آپلود (Client)

src/app/profile/avatar-form.tsx

```

"use client";
import { useActionState } from "react";
import { uploadAvatar } from "../upload";

export function AvatarForm() {
  const [state, action, pending] = useActionState(uploadAvatar, null);
  return (
    <form action={action} className="space-y-3 p-6">
      { /* the input name must match formData.get("avatar") */ }
      <input type="file" name="avatar" accept="image/*" className="block" />
      <button disabled={pending} className="rounded bg-blue-600 px-4 py-2 text-white">
        {pending ? "آپلود" : "در حال آپلود..."}
      </button>
      {state?.error && <p className="text-red-600">{state.error}</p>}
      {state?.ok && <img src={state.url} alt="" className="h-20 w-20 rounded-full" />}
    </form>
  );
}

```

نتیجه

آپلود کاملاً روی سرور انجام و اعتبارسنجی می‌شود (نوع و حجم)، پس امن است. چون از Server Action استفاده کردیم، نیازی به ساختن دستی یک Route Handler و مدیریت `fetch` نبود. برای فایل‌های بزرگ‌تر، می‌توانید به جای ذخیره مستقیم، یک URL آپلود امضاشده (presigned) تولید کنید.

پروژه ۸ – احراز هویت با کوکی و proxy

چه می‌سازیم؟ یک جریان کامل ورود/خروج با کوکی `httpOnly`، به‌علاوه محافظت از مسیر `/dashboard` در دو لایه: `proxy.ts` برای ریدایرکت سریع، و بررسی دوباره در خود صفحه.

مفاهیم کلیدی: `defense in depth` `proxy.ts` `httpOnly cookie`

گام ۰ - ساختاری که می‌سازید

```
structure

src/
├─ proxy.ts           # fast guard: redirect guests away from /dashboard
└─ app/
  ├─ login/
  │   ├─ actions.ts    # login + logout server actions
  │   └─ page.tsx      # login form
  └─ dashboard/
      └─ page.tsx      # re-checks the session (never trust proxy alone)
```

گام ۱ - اکشن‌های ورود و خروج

```
src/app/login/actions.ts

"use server";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { verifyCredentials, createToken } from "@lib/auth";

export async function login(_prev: unknown, formData: FormData) {
  const user = await verifyCredentials(formData); // your auth logic
  if (!user) return { error: "نام کاربری یا رمز عبور نادرست است" };

  const store = await cookies();
  // 🛡️ httpOnly ⇒ not readable by client JS (protects against XSS token theft)
  store.set("session", await createToken(user), {
    httpOnly: true,
    secure: true,
    sameSite: "lax",
    path: "/",
    maxAge: 60 * 60 * 24 * 7,
  });
  redirect("/dashboard");
}

export async function logout() {
  (await cookies()).delete("session"); // 🗑️ sign out
  redirect("/login");
}
```

src/app/login/page.tsx

```

"use client";
import { useActionState } from "react";
import { login } from "../actions";

export default function LoginPage() {
  const [state, action, pending] = useActionState(login, null);
  return (
    <form action={action} className="mx-auto max-w-sm space-y-3 p-6">
      <input name="username" placeholder="نام کاربری" className="w-full rounded border p-2" />
      <input name="password" type="password" placeholder="رمز عبور" className="w-full rounded border p-2" />
      <button disabled={pending} className="w-full rounded bg-blue-600 py-2 text-white">ورود</button>
      {state?.error && <p className="text-red-600">{state.error}</p>}
    </form>
  );
}

```

گام ۳ – لایه اول محافظت: proxy

این لایه فقط یک «دروازه سریع» است که مهمان‌ها را به صفحه ورود می‌فرستد:

src/proxy.ts

```

import { NextResponse, type NextRequest } from "next/server";

export function proxy(request: NextRequest) {
  const hasSession = Boolean(request.cookies.get("session")?.value);
  if (!hasSession && request.nextUrl.pathname.startsWith("/dashboard")) {
    return NextResponse.redirect(new URL("/login", request.url)); // 🛡️ fast guard
  }
  return NextResponse.next();
}

export const config = { matcher: ["/dashboard/:path*"] };

```

گام ۴ – لایه دوم: بررسی در خود صفحه

هرگز امنیت را فقط به proxy نسپارید. در خود صفحه محافظت‌شده، session را واقعاً اعتبارسنجی کنید:

src/app/dashboard/page.tsx

```
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { getUserFromToken } from "@lib/auth";

export default async function DashboardPage() {
  const token = (await cookies()).get("session")?.value;
  const user = token ? await getUserFromToken(token) : null;
  if (!user) redirect("/login"); // 🛡️ real verification at the data layer

  return <h1 className="p-6">خوش آمدید {user.name}</h1>;
}
```

چرا دو لایه؟ (درک عمیق)

آسیب‌پذیری‌های اخیر لایه proxy نشان داد که اتکای صرف به آن خطرناک است؛ یک درخواست دستکاری‌شده می‌تواند از proxy عبور کند. اما اگر صفحه و اکشن‌ها هم session را مستقل بررسی کنند (Defense in Depth)، حتی با دور خوردن proxy، داده محافظت‌شده لو نمی‌رود. این الگوی استاندارد امنیت در Next.js است.

قدم بعدی شما

این هشت پروژه را در یک ریپوی تمرینی پیاده کنید، سپس آن‌ها را ترکیب کنید: یک «پنل پزشک» با احراز هویت + فهرست نوبت‌های صفحه‌بندی‌شده + جستجو + آپلود مدارک + تم تاریک. وقتی این الگوها در دستتان بیفتد، ساختن هر اپلیکیشن واقعی فقط کنار هم گذاشتن همین قطعات است.

35 بهترین شیوه‌ها، اشتباهات رایج و منابع

جمع‌بندی حرفه‌ای: عادت‌هایی که کیفیت پروژه را بالا می‌برند و دام‌هایی که باید از آن‌ها دوری کنید.

| بهترین شیوه‌ها

- ♦ با **Server Component** شروع کنید؛ `"use client"` را فقط روی کوچک‌ترین جزء تعاملی و در پایین‌ترین سطح ممکن بگذارید.
- ♦ واکنشی داده را نزدیک به محل مصرف انجام دهید و منابع مستقل را با `Promise.all` موازی کنید.
- ♦ کشینگ را آگاهانه انتخاب کنید: عمومی با `use cache`، شخصی با `use cache: private`، و باطل‌سازی با `revalidateTag`.
- ♦ هر ورودی کاربر را در سمت سرور اعتبارسنجی کنید (zod) و دسترسی را در لایه داده بررسی کنید، نه فقط در `.proxy`.
- ♦ برای رابط راست‌چین از خاصیت‌های منطقی (`ms-/me-`) و تنظیم درست `lang="fa" dir="rtl"` استفاده کنید.
- ♦ قبل از تحویل، `tsc --noEmit` و یک `build` کامل بگیرید.

| اشتباهات رایج

اشتباه	راه درست
فراموش کردن <code>await</code> روی <code>params</code>	در نسخه ۱۶ همیشه <code>await params</code> بنویسید
گذاشتن <code>"use client"</code> در بالای صفحه	فقط جزء تعاملی را Client کنید
انتظار کش خودکار مثل نسخه‌های قبل	در v16 صراحتاً <code>use cache</code> بنویسید
import از <code>next/router</code>	در App Router از <code>next/navigation</code>
اکسپورت اسرار با <code>NEXT_PUBLIC_</code>	اسرار را فقط سمت سرور نگه دارید
اتکای کامل به proxy برای احراز هویت	بررسی دسترسی را در لایه داده تکرار کنید

منابع برای ادامه یادگیری

- ♦ مستندات رسمی: nextjs.org/docs – مرجع کامل و به‌روز.
- ♦ دوره رسمی: nextjs.org/learn – آموزش گام‌به‌گام تعاملی.
- ♦ وبلاگ نسخه‌ها: nextjs.org/blog – برای دنبال کردن تغییرات هر نسخه.
- ♦ مخزن گیت‌هاب: برای مطالعه مثال‌ها و دنبال کردن تغییرات و آسیب‌پذیری‌های امنیتی.

سخن پایانی

Next.js 16 با مدل «پویا به‌صورت پیش‌فرض» و کشینگ صریح، کنترل و شفافیت بیشتری به شما می‌دهد. اگر این چند اصل را درونی کنید – Server-first بودن، کشینگ آگاهانه، اعتبارسنجی سمت سرور و استفاده از ابزارهای بهینه‌سازی داخلی – می‌توانید اپلیکیشن‌هایی سریع، امن و مقیاس‌پذیر بسازید. حالا بهترین کار، ساختن یک پروژه واقعی و تمرین همین مفاهیم است.

پایان • موفق باشید 🚀

36 پیوست الف – پرسش‌های مصاحبه

پرباربردترین پرسش‌های مصاحبه 16 Next.js، از سطح Junior تا Senior، با پاسخ‌های کوتاه و دقیق. اگر به همه این‌ها مسلط باشید، در بیشتر مصاحبه‌های فنی موفق خواهید بود.

سطح Junior

تفاوت App Router و Pages Router چیست؟ App Router معماری مدرن مبتنی بر پوشه `app` است که از `Server Components`، `layout`، `Server Components`، `Streaming` و `Cache Components` پشتیبانی می‌کند؛ `Pages Router` سیستم قدیمی‌تر مبتنی بر `pages` است. برای پروژه‌های جدید `App Router` توصیه می‌شود.

Server Component و Client Component چه فرقی دارند؟ `Server Component` پیش‌فرض است، فقط روی سرور اجرا می‌شود، به مرورگر ارسال نمی‌شود و می‌تواند مستقیماً داده بگیرد؛ اما `state` و رویداد ندارد. `Client Component` با `"use client"` مشخص می‌شود و برای تعامل، `state` و `API`‌های مرورگر لازم است.

چطور بین صفحات پیمایش می‌کنید؟ با کامپوننت `<Link>` (ناوبری نرم + `prefetch` خودکار)، و برای پیمایش برنامه‌نویسی‌شده با `useRouter` از `next/navigation`.

سطح Middle

در نسخه ۱۶ کشینگ چطور کار می‌کند؟ برخلاف نسخه‌های قبل، همه چیز پیش‌فرض «پویا» است. با فعال کردن `cacheComponents` و دایرکتیو `use cache` صراحتاً انتخاب می‌کنید چه چیزی کش شود؛ عمر آن با `cacheLife` و باطل‌سازی هدفمند با `revalidateTag` + `cacheTag`.

Server Action چیست و چه مزیتی دارد؟ تابعی با `"use server"` که روی سرور اجرا می‌شود و می‌توان مستقیماً به فرم وصلش کرد؛ بدون ساختن دستی `API`، با محافظت داخلی `CSRF` و پشتیبانی از `Progressive Enhancement`.

چطور از آبشار درخواست‌ها (waterfall) جلوگیری می‌کنید؟ منابع مستقل را با `Promise.all` موازی می‌کنم، درخواست‌های تکراری را با `React.cache` حذف می‌کنم، و برای واکنشی زود هنگام از الگوی `preload` استفاده می‌کنم.

Streaming و Suspense چه می‌کنند؟ اجازه می‌دهند بخش‌های سریع صفحه فوری نمایش داده شوند و بخش‌های کند به تدریج «استریم» شوند؛ کاربر به جای صفحه خالی، اسکلت را فوری می‌بیند.

سطح Senior

خطای Hydration چیست و چطور رفعش می‌کنید؟ وقتی HTML سرور با اولین رندر کلاینت فرق کند رخ می‌دهد؛ علت‌های رایج: API، `Date.now()` / `Math.random()` های مرورگر، و HTML نامعتبر. راه‌حل: خروجی قطعی، انتقال کد وابسته به مرورگر به `useEffect`، و در موارد ناگزیر `suppressHydrationWarning`.

معماری یک اپ بزرگ Next.js را چطور می‌چینید؟ ساختار feature-based، پوشه `app` نازک، و یک لایه دسترسی به داده (DAL) که تنها درگاه دیتابیس است و بررسی دسترسی در آن متمرکز می‌شود.

امنیت را کجا اعمال می‌کنید؟ اعتبارسنجی ورودی روی سرور (zod)، بررسی دسترسی در DAL نه UI، کوکی session با rate limiting، `httpOnly + secure + sameSite` روی اندپوینت‌های حساس، و اسرار خارج از proxy. `NEXT_PUBLIC_` را تنها خط دفاع نمی‌کنم (Defense in Depth).

PPR (Partial Prerendering) چیست؟ یک صفحه می‌تواند پوسته استاتیک فوری داشته باشد و بخش‌های پویا به تدریج استریم شوند؛ ترکیب بهترین‌های استاتیک و پویا در یک صفحه.

چطور کارایی را می‌سنجید و بهبود می‌دهید؟ با Core Web Vitals (LCP، CLS، INP)، کاهش JS کلاینت با Server-first بودن، `next/image` و `next/font`، بارگذاری تنبل، و کشینگ آگاهانه.

نکته مصاحبه

در مصاحبه، فقط تعریف ندهید؛ بگویید «چرا» و «چه زمانی». مثلاً برای کشینگ: «چون در v16 پیش‌فرض پویاست، من آگاهانه فقط داده عمومی کم‌تغییر را `use cache` می‌کنم و داده کاربر را نه.» این نشان‌دهنده درک عمیق است، نه حفظ‌کردن.

37 پیوست ب – واژه‌نامه فارسی-انگلیسی

مرجع سریع مهم‌ترین اصطلاحات 16 Next.js و React Server Components. برای مرور پیش از مصاحبه

اصطلاح	توضیح کوتاه
App Router	معماری روتینگ مدرن مبتنی بر پوشه <code>app</code> با پشتیبانی از Server Components.
Server Component (React RSC)	کامپوننتی که فقط روی سرور اجرا می‌شود و به مرورگر ارسال نمی‌شود.
Client Component	کامپوننت تعاملی با <code>"use client"</code> که در مرورگر اجرا می‌شود.
Hydration	فرایند «زنده‌کردن» HTML سرور در مرورگر و اتصال رویدادها.
Streaming	ارسال تدریجی بخش‌های صفحه به مرورگر به جای انتظار برای کل آن.
Suspense	مرزی که تا آماده‌شدن محتوا، یک fallback (اسکلت) نشان می‌دهد.
Cache Components	مدل کشینگ v16 مبتنی بر دایرکتیو صریح <code>use cache</code> .
cacheLife	تعیین مدت‌زمان تازه‌ماندن یک ورودی کش.
cacheTag / revalidateTag	برچسب‌زدن کش و باطل‌کردن هدفمند آن پس از تغییر.
Server Action	تابع سمت‌سرور با <code>"use server"</code> برای تغییر داده، قابل‌اتصال به فرم.
Route Handler	اندپوینت API با فایل <code>route.ts</code> برای پاسخ به متدهای HTTP.
proxy.ts	جانشین middleware؛ کد اجراشده پیش از رسیدن درخواست به مسیرها.
DAL (Data Access Layer)	تنها لایه‌ای که با دیتابیس حرف می‌زند و دسترسی را بررسی می‌کند.
RBAC	کنترل دسترسی نقش‌محور (Role-Based Access Control).
PPR (Partial Prerendering)	ترکیب پوسته استاتیک با بخش‌های پویای استریم‌شونده در یک صفحه.
ISR	بازتولید تدریجی صفحات استاتیک پس از انقضای عمر کش.
SSR / SSG / CSR	رندر سمت سرور / تولید استاتیک هنگام بیلد / رندر سمت کلاینت.
Turbopack	باندلر پیش‌فرض و سریع v16 (جایگزین Webpack).

Prefetch	واکشی زود هنگام مسیر مقصد در پس‌زمینه برای ناوبری آتی.
Core Web Vitals	معیارهای کلیدی تجربه کاربر: LCP، CLS، INP.
Progressive Enhancement	کارکرد پایه بدون JS و بهبود تدریجی با فعال‌شدن آن.
Server-only	نشانه‌گذاری مازول‌هایی که هرگز نباید به باندل کلاینت بروند.

پایان مرجع · موفق باشید 🚀